

从 C++ 到计算系统：第二册

硬件原理、多核并行、高性能计算、同步、无锁结构与分布式计算

CPU Performance Study

本册接续第一册的程序执行基础，系统讲解现代 CPU 硬件、多核心并发、高性能计算、锁与无锁数据结构、并行算法、运行时和分布式计算。AI 模型、算子开发和推理引擎放入第三册。

前言

第一册已经回答了基础问题：一个 C++ 程序如何变成进程、地址空间、二进制接口、汇编指令和可测量的执行现象。它负责打牢源码到二进制、x86-64 汇编、System V ABI、Linux 基础工具、虚拟地址空间和可信 benchmark 的底座。第二册不再把这些内容重新讲一遍，而是把它们当作读者已经掌握的工具：能看汇编，能确认二进制身份，能写基本实验报告，能理解进程和虚拟地址空间的基本边界。

本册继续向下和向外展开，但不提前进入 AI。真正要理解高性能程序，必须先把“计算本身”讲清楚：核心怎样取指、译码、调度和提交，乱序执行为什么能隐藏延迟，分支预测为什么会失败，缓存和 TLB 为什么决定很多程序的上限，多核心之间为什么需要一致性协议，NUMA 为什么让同一块内存存在不同核心看来成本不同。第一册把单个程序放到机器上，第二册追问这台机器怎样扩展为多核心计算系统，再怎样扩展为多进程、多节点和分布式计算系统。

软件层面同样不能跳步。多线程不是简单地把循环切成几份；锁不是只有“慢”一个性质；原子操作不是更快的锁；无锁数据结构也不是不用同步。并行算法要面对数据依赖、负载均衡、局部性、归约顺序和任务粒度。分布式计算更进一步：一旦机器之间只能通过网络通信，延迟、带宽、故障、重试、超时、复制和一致性就会成为计算模型的一部分。

因此第二册的主题是计算系统，而不是某个应用领域。它从硬件原理开始，讲到单机高性能计算、多线程并发、锁与无锁结构、并行算法、运行时系统、异步 I/O 和分布式计算。AI 模型、张量、算子开发和量化推理引擎会放到第三册。这样安排不是延后重点，而是先把地基打牢：没有硬件和并发基础，后续任何 AI 算子优化都会变成经验堆砌。

核心思想

第二册的核心问题是：给定真实硬件、操作系统和网络边界，怎样组织数据、线程、同步、任务和通信，使计算能够正确、可测量、可扩展地运行。

本册仍然沿用第一册的写法：原理先行，代码作为证据；实验必须记录输入、环境、命令和误差；源码阅读抓数据结构、热路径、同步边界和性能瓶颈。不过，本册的证据重心会改变。第一册主要用反汇编、ELF、GDB、`readelf`、`objdump` 和基础 `perfstat` 建立“这段程序真的这样执行”的证据；第二册会更多使用拓扑、PMU、调度事件、锁等待、cache line 迁移、NUMA 放置、队列长度、吞吐曲线和失败注入来解释“系统为什么不能继续扩展”。

读者读完后，应当能解释一个多核程序为什么没有线性加速，能判断锁竞争和 false sharing 的差异，能写出可测试的并行归约、有界队列和环形队列，能读懂 Linux 调度、futex、RCU、内存管理和 perf 相关源码入口，也能把单机计算原则扩展到分布式系统。

怎样阅读本册

本册默认目标平台是本地 Linux 机器。普通 x86-64 Linux 台式机、笔记本或服务器适合完成完整实验；Termux 可以用于阅读、构建小实验和运行基础测试，但 perf、NUMA、线程亲和性、硬件计数器、futex 观察、eBPF 观测和分布式网络实验可能受权限、内核和设备能力限制。所有性能结论都必须写清环境。

阅读本册前，应默认已经完成第一册的最低要求：能把源码编译成二进制并确认产物身份，能读基本 x86-64 汇编，能解释调用约定和栈帧，能写出隔离热路径的 benchmark，能使用 Linux 常见工具观察进程、地址空间和基础性能计数。第二册不会反复解释这些背景；遇到这些内容时，本册只会说明它们怎样服务多核、同步、运行时或分布式问题。

阅读顺序建议严格按章节推进。前四章讲硬件执行模型，这是后面所有内容的解释基础。单机高性能计算部分讲测量、数据布局、SIMD 和典型内核，帮助读者把“程序慢”分解成计算、内存、分支、同步或 I/O 问题。并发部分讲线程、锁、条件变量、原子、内存序和无锁数据结构，重点不是背 API，而是理解共享状态怎样被保护、发布和回收。并行算法和分布式计算部分再把这些原则扩展到任务图、运行时、网络通信、复制和一致性。

本册的贯穿实验叫 Compute Systems Labs。它不追求一开始很大，而是从小而真实的模块开始：顺序归约、并行归约、带锁计数器、原子计数器、有界队列、任务切分和简单 benchmark。后续扩写会继续加入线程池、工作窃取、SPSC 环形队列、MPMC 队列、并行扫描、分布式 shuffle 和故障注入。

心智模型

读本册时始终追问四件事：数据在哪里，谁会读写它，硬件会怎样移动它，失败或竞争发生时系统怎样保持正确。

每章至少做一个实验。硬件章节要配合 `lscpu`、`numactl`、`perfstat`、拓扑图和内存访问 microbenchmark；并发章节要写出可重复触发竞争、阻塞、false sharing、CAS 失败或内存序问题的小程序；运行时章节要记录队列长度、任务粒度、worker 空闲时间和背压；分布式章节即使只在本机多进程模拟，也要记录延迟、重试、超时、重复请求和消息顺序。

常见误区

不要把高性能计算理解成“把线程数开大”。线程数只是资源请求，性能来自数据复用、负载均衡、同步控制和测量证据。没有这些证据，加线程常常只是更快地制造争用。

本册的章节之间有明确承接关系。第一部分回答硬件成本从哪里来，第二部分回答怎样把成本变成可测假设，第三部分回答共享内存里怎样保持正确，第四部分回答怎样把多个线程组织成可扩展运行时，第五部分回答当共享内存消失、网络和故障进入模型后，计算原则怎样变化。不要把某章当成孤立技巧阅读；每次做实验，都要把结论回连到这条链上。

全书结构

本册分为五个部分，围绕“计算系统如何正确且高效地扩展”展开。第一册已经承担源码、进程、虚拟地址、汇编、ABI、工具链和基础测量，本册只在需要建立新成本模型时短暂回扣这些内容，不再重复铺陈。这样处理的目的，是把篇幅留给第一册没有充分展开的主题：多核心硬件、缓存一致性、NUMA、SIMD 和内存带宽、Linux 调度和同步、无锁结构、并行运行时、异步 I/O、分布式计算和故障模型。

第一部分是硬件执行模型。它讲核心流水线、乱序执行、分支预测、寄存器重命名、缓存层级、TLB、页表、预取、NUMA、互连和缓存一致性。读者要在这里建立硬件成本模型：一条指令不只是语法，背后有取指、依赖、执行端口、访存、提交和一致性流量。

第二部分是单机高性能计算。第一册已经讲过 benchmark 纪律，所以本部分不再停留在“怎样计时”这类基础问题，而是进入多核进阶测量、Roofline、PMU 归因、cache 和 TLB 事件、数据布局、局部性、SIMD、指令级并行和典型计算内核。目标是把“优化”从口号变成可验证的分析：到底是算力不够、内存带宽不够、cache miss 太多、分支预测差、TLB 覆盖不足、同步等待太长，还是 benchmark 边界仍然写错。

第三部分是多线程、同步与内存模型。它讲线程生命周期、调度、亲和性、锁、条件变量、信号量、原子操作、内存序、false sharing、无锁队列和内存回收。这里的重点是正确性先于性能：如果可见性、互斥、进度保证和对对象生命周期没有说清楚，所谓高性能就是不稳定的。

第四部分是并行算法与运行时。归约、扫描、分区、图遍历、任务图、线程池、工作窃取、异步 I/O 和背压，都是把多核硬件变成可用计算能力的桥梁。读者在这里要学会从算法依赖和数据移动角度设计并程序序。

第五部分是分布式计算。它把单机原则扩展到网络和集群：RPC、序列化、时间、故障、分片、复制、一致性、共识、shuffle、检查点和观测性。分布式不是“多几台机器”，而是把通信和失败纳入计算模型。

核心思想

第二册的主线是一条从硬件到集群的链：硬件决定局部成本，多线程决定共享内存内的协作，并行算法决定可扩展形状，分布式系统决定跨机器的通信和故障边界。

和第一册的边界

第一册已经讲清楚的内容，本册只作为前提使用：源码到目标文件和 ELF 的路径，进程和虚拟地址空间的基本概念，x86-64 汇编阅读，System V AMD64 ABI，C++ 与汇编互操作，基础 Linux 命令，基础 benchmark 设计，基础 **perfstat** 和 Linux 源码入口。第二册遇到这些内容时，重点不是重复定义，而是解释它们在可扩展计算中的新作用。

例如，第一册讲虚拟地址空间时，目标是让读者理解进程地址、映射和缺页；第二册讲 TLB 和页级性能时，目标是解释页大小、page walk、NUMA 放置和首次触页怎样改变吞吐。第一册讲 **perfstat** 时，目标是建立可信测量意识；第二册讲 PMU 时，目标是用事件组合区分前端、后端、内存、同步和调度瓶颈。第一册讲汇编控制流时，目标是读懂分支和跳转；第二册讲分支预测时，目标是理解输入分布、预测器状态和流水线回滚怎样限制单核吞吐。

本册扩展的重点

本册扩展不是简单增加术语，而是围绕可扩展计算的核心矛盾展开。硬件部分要能从 pipeline、ROB、load/store queue、store buffer、cache line、TLB、NUMA 和一致性协议解释成本。同步部分要能从不变量、等待谓词、futex、内存序、线性化点和内存回收解释正确性。运行时部分要能从任务粒度、局部性、工作窃取、背压和关闭语义解释工程稳定性。分布式部分要能从消息、重试、幂等、复制、共识、shuffle、检查点和观测性解释跨机器计算。

每个章节都要有三个出口：能画出机制图，能写出最小可运行例子，能设计一份可复查实验。只

有这样，第二册才不是第一册的复述，而是把第一册的基础能力推进到现代计算系统。

规模目标和章节配额

第二册按正式书籍标准写作，最终正文不少于 50 万单位。统计口径只看 `chapters/` 下的正文内容，以中文字符加英文单词计数；代码块、命令、反汇编、终端输出、配置、前言、目录、附录和 LaTeX 标记不计入。50 万只是最低验收线，不是上限；如果硬件、多核、运行时、无锁结构和分布式计算的知识链需要更大篇幅，就按完整性继续写，不能因为数字达到门槛而停在半成品。这个目标不是排版长度目标，而是知识密度目标：每一章都必须把原理、机制、案例、实验和源码阅读边界讲清楚。

按 18 章估算，平均每章需要 2.7 万到 2.9 万单位。考虑章节难度不同，硬件执行、缓存/TLB/NUMA、测量、锁与原子、无锁结构、任务运行时和分布式计算应承担更高篇幅；实验路线和源码索引只能辅助正文，不能替代正文。后续扩写时，每章都要围绕同一条链展开：先讲模型，再讲硬件或系统实现，再讲 C++ 代码形态，再讲测量证据，再讲反例和工程边界。

常见误区

本册不接受用重复定义、空泛口号、附录堆表、题目清单或模板化实验报告凑字数。若一个段落不能帮助读者理解现代计算系统的机制、判断或实践，就不应进入正文。

章节质量标准

每章必须按“机制、案例、代码、测量、源码、作业”六个维度写作。机制部分要解释底层原理，不只给术语定义；案例部分要从真实问题出发，展示为什么朴素方案不够；代码部分要给 baseline、改进版本和边界处理版本，不能只给最终答案；测量部分要说明实验矩阵、指标、预期现象和失败解释；源码部分要给 Linux 或生产系统的窄入口，并说明读源码要验证什么；作业部分要能训练读者独立复现、修改、测量和写报告。

本册的案例不能停留在“数组求和”“打印线程 ID”这种单点玩具上。简单例子可以作为入口，但必须继续推进到工程问题：线程数增加为什么不线性加速，锁竞争和 false sharing 怎样区分，TLB miss 和 cache miss 怎样分开，任务粒度怎样影响线程池，队列满时背压怎样传导，分布式 shuffle 为什么会被热点 key 拖垮。每个案例都要有问题背景、朴素方案、失败原因、改进方案、代价和实验验证。

贯穿大项目

第二册贯穿项目命名为 Compute Systems Engine。它不是一个孤立实验，而是整本书的工程主线。第一阶段实现顺序和并行归约、测量 harness、拓扑记录和结果校验；第二阶段加入数据布局实验、线程本地 partial、false sharing 对照和 NUMA 初始化策略；第三阶段加入 mutex 队列、条件变量、有界队列、原子计数器和内存序实验；第四阶段加入线程池、任务图、工作窃取和运行时指标；第五阶段加入 SPSC ring buffer、MPMC queue、内存回收实验和压力测试；第六阶段加入异步 I/O 模拟、背压策略、批处理和限流；第七阶段在本机多进程模拟分布式 shuffle、幂等提交、重试、检查点和故障注入。

每章扩写时都要回答两个问题：本章为 Compute Systems Engine 增加了什么能力；这个能力需要哪些测试和测量才能被认为可靠。若某章不能落到项目上，也必须说明它提供的是哪一种模型能力，例如分支预测帮助解释热循环，TLB 帮助解释大工作集，NUMA 帮助解释多 socket 扩展曲线。

目录

第一部分	硬件执行模型：计算为什么会快也会慢	
第 1 章	计算系统全景	2
第 2 章	核心流水线、乱序执行与分支预测	7
第 3 章	缓存、TLB 与页级性能	17
第 4 章	NUMA、互连与缓存一致性	25
第二部分	单机高性能计算：从测量到数据流	
第 5 章	可信测量、Roofline 与性能计数器	31
第 6 章	数据布局、局部性与预取	37
第 7 章	SIMD、指令级并行与向量化	43
第 8 章	典型计算内核模式	45
第三部分	多线程、同步与内存模型	
第 9 章	线程、调度与亲和性	48
第 10 章	锁、条件变量与信号量	53
第 11 章	原子操作与内存序	59
第 12 章	无锁数据结构	65
第四部分	并行算法与运行时	
第 13 章	归约、扫描与分区	72
第 14 章	任务运行时与工作窃取	74
第 15 章	I/O、异步与背压	78
第五部分	分布式计算：把单机原则扩展到集群	
第 16 章	分布式计算模型	81
第 17 章	分片、复制与共识	83
第 18 章	分布式计算工程实践	85
附录 A	实验路线	87
附录 B	源码阅读索引	88

第零部分

硬件执行模型：计算为什么会快也会慢

第 1 章

计算系统全景

第一册已经说明程序如何变成进程、虚拟地址、调用约定和机器指令。第二册继续追问：当一个程序需要利用多个核心、多个内存层级、多个线程甚至多台机器时，计算能力从哪里来，又会在哪里损失。这个问题不能只从代码出发，也不能只从硬件手册出发。真正的计算系统由硬件、操作系统、运行时、算法和通信协议共同构成。

1.1 从第一册到第二册

第一册的学习对象是一段程序如何成为一个可观察的执行实例。读者已经知道源文件、目标文件、链接、ELF、进程、虚拟地址、ABI、汇编指令和基础测量之间的关系。第二册的学习对象不是再把这条路径复述一遍，而是把这个执行实例放进更复杂的资源环境：多个核心同时执行，多个缓存层级同时保存数据，多个线程共享状态，多个任务争用队列，多个节点通过网络交换消息。

这意味着本册每次遇到第一册知识时，都要换一个问题问。看到汇编，不再只问“这条指令做什么”，还要问“它处在依赖链上还是可以并行发射”；看到虚拟地址，不再只问“它属于哪个映射”，还要问“这个访问是否会击穿 TLB、是否发生首次触页、页面是否放在远端 NUMA 节点”；看到 `perfstat`，不再只问“数字如何记录”，还要问“这些事件能否区分前端、后端、内存带宽、锁等待和调度噪声”。

核心思想

第一册把程序执行讲清楚，第二册把可扩展计算讲清楚。前者关心“程序如何被机器执行”，后者关心“多个执行实体怎样在硬件、操作系统和网络边界内协同前进”。

1.2 从一条指令到一个系统

一条加法指令看起来很简单，但它并不是孤立执行的。核心要取指、译码、重命名寄存器、等待操作数、选择执行端口、写回结果并提交状态。如果操作数在寄存器里，成本很低；如果来自 L1 cache，延迟增加；如果来自 L3、远端 NUMA 节点或磁盘，程序看到的时间会跨越几个数量级。并发程序还要考虑另一个核心是否正在写同一条 cache line，分布式程序还要考虑消息是否跨越网络。

因此本册把计算系统分成五层。第一层是硬件执行，包括核心、缓存、TLB、内存控制器和互连。第二层是操作系统，包括调度、页表、系统调用、I/O 和计数器。第三层是运行时，包括线程池、任务队列、同步原语和内存分配器。第四层是算法，包括归约、扫描、分区、图遍历和负载均衡。第五层是分布式协议，包括 RPC、复制、共识、shuffle 和检查点。

核心思想

高性能不是某一层单独决定的。硬件给出能力边界，操作系统分配资源，运行时组织执行，算法决定数据依赖，分布式协议处理通信和故障。

1.3 四种尺度

计算系统至少有四种尺度。第一种是单条指令尺度，讨论延迟、吞吐、依赖、端口和分支预测。第二种是单个核心尺度，讨论流水线、乱序窗口、寄存器重命名、load/store queue 和 SIMD。第三种是单台机器尺度，讨论缓存层级、TLB、NUMA、线程调度、锁、原子、队列和 I/O。第四种是集群尺度，讨论 RPC、分片、复制、共识、shuffle、检查点和故障恢复。

这四种尺度不能混用。单条指令尺度上，减少一条指令可能有效；单机多线程尺度上，减少一条指令可能完全被锁等待淹没；集群尺度上，减少一百万条指令也可能抵不过一次跨机重试。工程判断必须先确认当前瓶颈属于哪个尺度。第二册的核心训练之一，就是把“慢”定位到正确尺度，再选择对应证据。


```

source code
-> instructions and dependencies
-> core pipeline and local cache
-> shared-memory multicore machine
-> runtime queues and tasks
-> networked nodes and distributed state

```

这张文本图不是排版装饰，而是阅读本册的索引。越往右，通信成本越高，故障模式越复杂，确定性越弱。第一册主要覆盖左侧两层，本册从中间向右推进。

1.4 计算成本的基本单位

分析性能时，不能只看“执行了多少行代码”。更稳定的单位是指令数、周期数、访存字节数、cache miss、TLB miss、分支误预测、同步等待时间、上下文切换、系统调用次数和网络往返延迟。不同单位之间不能随便相加，但它们共同解释程序为什么慢。

例如，一个数组求和循环的算术量很小，主要成本是把数据从内存送到核心。一个矩阵乘如果分块得当，会重复使用缓存中的数据，瓶颈可能转向浮点执行端口。一个共享队列如果每次 push 都抢同一把锁，瓶颈不是算术，也不是内存带宽，而是临界区串行化和 cache line 所有权迁移。

1.5 把成本转成证据

第二册不鼓励凭经验猜瓶颈。每个成本单位都应该有可观察证据。依赖链问题可以用汇编、优化报告、IPC 和不同累加器版本比较；分支问题可以用输入分布、branch-misses 和 branchless 版本比较；cache 问题可以用工作集大小、访问步长和 cache 事件比较；TLB 问题可以用页数、大页和 dTLB 事件比较；锁问题可以用等待时间、futex、context switches 和吞吐曲线比较；NUMA 问题可以用 first-touch、绑定策略和远端访问计数比较；分布式问题可以用 trace、队列长度、重试次数和超时分布比较。

证据的作用不是一次性证明真理，而是排除错误方向。若一个归约程序在线程数增加后不再加速，可能原因很多：内存带宽到顶、false sharing、锁竞争、线程迁移、任务粒度太小、NUMA 放置错误、CPU 频率下降或 benchmark 把初始化混入热路径。没有证据时，这些解释都像对；有了证据，才能逐步缩小范围。

1.6 工作负载决定系统形状

同一台机器在不同工作负载下会呈现完全不同的系统形状。一个大数组顺序求和主要考验内存带宽、预取和 TLB 覆盖；一个哈希表查询主要考验随机访问、分支、缓存 miss 和分配器布局；一个日志写入服务主要考验队列、批处理、I/O 延迟和背压；一个并行图遍历主要考验负载均衡、前沿大小、随机邻接访问和同步开销。若不先描述工作负载，就谈不上性能分析。

工作负载至少要写清五件事。第一，输入规模和数据分布。例如数组长度、key 的倾斜程度、图的顶点度分布、请求大小分布。第二，读写比例。例如只读查询、读多写少、写多读少、读写混合。第三，局部性形状。例如顺序扫描、固定步长、随机访问、热点访问、分层访问。第四，依赖形状。例如每个元素独立、前缀依赖、图依赖、任务依赖、跨节点依赖。第五，失败和等待形状。例如是否会阻塞 I/O，是否会超时重试，是否会遇到队列满，是否会有节点失败。

经典教材式写法不应把“程序”当成孤立对象，而应把程序放在 workload 里看。一个函数在小输入上很快，不代表在大输入上快；在均匀 key 上扩展良好，不代表在热点 key 上可靠；在单线程上 cache 命中高，不代表多线程不会互相驱逐；在本机多进程模拟正常，不代表真实网络环境下重试和乱序没有影响。第二册后续所有实验都必须先写 workload，再写实现，再写测量。

1.7 瓶颈会迁移

优化不是把一个程序永久变快，而是把瓶颈从一个位置推到另一个位置。一个标量循环最初可能受长依赖链限制；使用多累加器后，依赖链缓解，瓶颈可能变成加载端口；再使用 SIMD 后，指令数下降，瓶颈可能变成内存带宽；再多线程后，瓶颈可能变成 NUMA、LLC 争用或内存控制器；

再把任务分布到多机后，瓶颈可能变成 shuffle、热点 key 或检查点。每次优化都应该重新判断瓶颈，而不是沿用上一次结论。

瓶颈迁移也是很多性能争论的根源。一个人说“这个优化没用”，另一个人说“这个优化提升很大”，可能他们测的是不同阶段的程序。在内存带宽已经打满的版本上减少几条整数指令，确实不会明显加速；在还受依赖链限制的版本上，多累加器可能非常有效。没有上下文的优化建议通常没有意义。

因此本册要求性能报告写出“优化前瓶颈假设”和“优化后瓶颈是否迁移”。例如优化并行归约时，可以先写：单线程版本 IPC 较低，怀疑依赖链；四累加器版本 IPC 提高但带宽接近上限；多线程版本 8 线程后不再加速，怀疑内存带宽或 NUMA；按 NUMA 节点初始化后带宽改善，说明原先包含远端访问因素。这样的叙述才像工程分析，而不是只列快了多少。

1.8 系统边界和责任边界

计算系统由多个层次组成，但工程修改必须有责任边界。硬件给出指令、缓存、TLB、互连和计数器；操作系统负责调度、地址空间、页表、I/O 和权限；运行时负责任务、队列、线程池、内存分配和同步封装；算法负责依赖、切分、合并和数据结构；业务协议负责语义、重试、幂等和恢复。性能问题经常跨层出现，但修复不能没有边界。

例如，一个服务尾延迟高，可能是锁竞争、页错误、磁盘抖动、网络重试、GC 或业务热点。排查时可以跨层收集证据，但修复时要明确到底改哪一层。如果把业务幂等问题硬塞进线程池，线程池会变得难以复用；如果把 NUMA 亲和性写死在算法库里，库就难以适配不同机器；如果把所有指标都放在业务日志里，底层运行时就无法单独诊断。好的系统设计会让层次之间有清晰接口，同时允许证据跨层关联。

本册后面讲线程池时，会强调 submit、shutdown、wait 的语义；讲无锁结构时，会强调线性化点和内存回收；讲分布式系统时，会强调请求 ID、重试预算和提交点。这些内容看起来分散，其实都在回答同一个问题：系统的责任边界在哪里，失败时谁负责恢复，性能结论应归因到哪一层。

1.9 延迟、吞吐和并发度

系统分析经常混淆三个词：延迟、吞吐和并发度。延迟是单个操作从开始到结束的时间，吞吐是单位时间完成多少操作，并发度是同时在系统中飞行的操作数量。一个内存 load 的延迟可能很高，但如果核心能同时挂起多个 miss，整体吞吐仍然可以接受；一个网络请求延迟很高，但如果客户端保持足够多的 in-flight 请求，吞吐可以接近带宽上限。

这三个量之间有一个朴素但非常有用的关系：要填满一个延迟很高的管道，需要足够并发度。磁盘、网络、内存和 CPU 后端都可以用这个心智模型理解。区别在于，CPU 的并发度来自乱序窗口、load buffer、SIMD lane 和多核心；网络系统的并发度来自连接数、请求队列和异步 I/O；分布式计算的并发度来自分片、任务和流水线。

心智模型

延迟回答“一个操作要等多久”，吞吐回答“系统每秒能做多少”，并发度回答“为了达到吞吐，需要同时容纳多少未完成工作”。

1.10 Amdahl 和 Gustafson

多核性能讨论离不开可并行比例。Amdahl 定律说明，如果程序有一部分必须串行执行，那么线程数增加到一定程度后，加速会被串行部分限制。Gustafson 定律从另一个角度看问题：如果随着核心数增加而扩大问题规模，可并行部分也随之增加，系统仍然能获得更高总吞吐。

这两个定律不是考试公式，而是工程判断工具。一个服务端请求路径中，如果日志锁、全局分配器、单分片元数据或单线程事件循环占比很高，增加 worker 不会线性加速。一个离线计算任务如果可以把输入切成更多分片，让每个节点处理独立数据，再做树形合并，扩大规模时就更接近 Gustafson 的场景。

1.11 观察系统的三张图

学习计算系统时，建议每遇到一个程序都画三张图。第一张是数据流图：输入从哪里来，经过哪些结构，输出到哪里。第二张是执行图：哪些线程、任务或节点执行哪些阶段，哪里并行，哪里等待。第三张是成本图：每个阶段主要消耗 CPU、内存、I/O、锁等待还是网络。

这三张图可以避免把问题混成一团。一个队列积压，可能是消费者计算慢，也可能是生产者太快，也可能是下游 I/O 阻塞，也可能是队列锁竞争。只有先画清数据和执行边界，测量才知道该放在哪里。

数据流图应标出数据的所有权和复制点。数组是只读输入，还是会被多个线程写回；任务队列里保存的是值、指针还是共享对象；网络请求里发送的是完整数据、增量还是引用；文件映射里的页面是否会被多个进程共享。所有权不清，优化很容易破坏正确性。比如把对象从共享指针改成裸指针可能减少引用计数开销，但如果生命周期没有重新证明，就可能引入悬垂访问。

执行图应标出并行区、串行区和等待区。很多程序源码看上去“用了线程”，执行图却显示所有线程都在等待一个队列、一把锁或一个单线程提交阶段。执行图还要标出调度关系：任务由谁提交，由谁执行，是否允许嵌套，是否可能在 worker 内等待 future，是否可能被取消。没有执行图，线程池和异步代码很容易变成看不见的控制流。

成本图应标出主成本和次成本。一个阶段可以同时消耗 CPU 和内存，但报告要判断主导因素。若主成本是内存带宽，减少日志字符串格式化不一定有用；若主成本是锁等待，改 SIMD 不会解决；若主成本是网络重试，优化本地解析函数意义有限。成本图不是精确模型，而是让团队讨论同一个对象。

1.12 正确性先于性能

并发和分布式计算最容易出现错误，是把速度放在正确性之前。数据竞争可能让程序偶尔错；错误的内存序可能只在某些 CPU 上触发；无锁结构如果没有处理对象回收，可能在压力测试里才出现悬垂访问；分布式系统如果没有处理超时和重试，可能在网络抖动时重复提交请求。

本册每次讨论优化，都要先给出语义边界。共享变量由谁写，谁读，什么时候发布，什么时候回收。任务是否可以重排，结果是否依赖顺序。消息是否幂等，故障后是否可以重试。只有这些问题说清楚，性能数字才有意义。

1.13 从单机到分布式的一条线

很多读者会把并发和分布式分成两门课：前者讲线程，后者讲网络。第二册刻意把它们放在一条线上，是因为它们面对的是同一种基本矛盾：多个执行实体共享某些状态，但共享状态越多，协调成本越高。在单机上，协调成本表现为锁、原子、cache line 迁移、调度和内存序；在集群中，协调成本表现为 RPC、复制、共识、重试、超时和数据迁移。

因此，一个好的单机设计原则常常能迁移到分布式系统。线程本地计数再合并，对应 map-side combine；分片锁对应按 key 分区；树形归约对应分层聚合；有界队列和背压对应限流和排队控制；线性化点对应协议中的提交点；对象生命周期对应消息幂等和状态恢复。反过来，分布式系统也会提醒单机程序：只要协调成本足够高，就必须减少共享、批量通信、明确失败语义。

1.14 贯穿项目：Compute Systems Engine

本册需要一个贯穿全书的大项目，否则知识会散成很多孤立技巧。这个项目命名为 Compute Systems Engine。它的目标不是做一个商业框架，而是用足够小、可测试、可测量的工程，把现代计算系统的核心问题串起来：数据怎样放置，线程怎样执行，任务怎样调度，同步怎样证明，队列怎样背压，失败怎样恢复，分布式阶段怎样保持幂等。

项目的第一条主线是计算内核。最开始只有顺序归约和并行归约，但它们不是为了演示“多线程加速”这么简单。顺序归约用于建立 reference 和测量基线；多累加器版本用于解释依赖链和乱序执行；线程本地 partial 用于解释 false sharing 和共享写；NUMA 分块初始化用于解释 first-touch；树形合并用于连接后面的分布式聚合。一个小小的归约，在第二册里要承担从核心流水线到集群聚合的教学任务。

项目的第二条主线是运行时。先实现一个 mutex 和 condition variable 保护的 bounded queue，用它讲清锁保护的不变量、等待谓词、关闭语义和背压。然后实现固定线程数线程池，用它讲清 submit、

wait、shutdown、异常传播和任务粒度。再加入任务图和工作窃取，用它讲清依赖、ready 队列、本地性和负载均衡。最后把队列和线程池用于分布式模拟，让读者看到单机运行时和分布式执行器之间的相似性。

项目的第三条主线是并发数据结构。先比较 mutex counter、atomic counter 和 sharded counter，解释原子热点和 cache line 所有权迁移。再实现 SPSC ring buffer，解释 head/tail 单写者、acquire/release 和环形容量边界。之后再讨论 MPMC 队列、ABA、hazard pointer、epoch 和 RCU，不让读者一上来就陷入复杂无锁代码。每一步都要有 reference、有压力测试、有错误注入。

项目的第四条主线是分布式计算模拟。它可以先在本机多进程或多线程中完成，不依赖真实集群。输入按分片读取，worker 本地聚合，shuffle 按 key 重分布，reduce 合并输出。随后加入重试、超时、重复请求、幂等提交、检查点和故障注入。这个阶段把前面所有原则重新组合：局部性变成数据本地调度，锁热点变成热点 key，背压变成限流，线性化点变成提交点，内存回收变成状态恢复。

心智模型

贯穿项目的每一次扩展，都必须同时回答三个问题：语义是否正确，性能瓶颈在哪里，失败时怎样恢复。只回答其中一个，都不是系统工程。

1.15 本册贯穿实验

第二册的实验项目叫 Compute Systems Labs。第一阶段实现顺序归约、并行归约、带锁计数器、原子计数器和有界队列。后续章节会围绕它继续扩展：加入线程池、任务图、工作窃取、无锁队列、并行扫描、I/O 背压和分布式模拟。

实验一：运行 Compute Systems Labs。记录 CPU 型号、核心数、线程数、缓存层级、系统版本、编译器版本和测试命令。不要只写测试通过，还要解释每个模块分别验证了计算系统的哪一类问题。

第 2 章

核心流水线、乱序执行与分支预测

现代 CPU 核心不是按源码顺序一条一条机械执行。为了提高吞吐，核心会把指令拆成微操作，放入队列，分析依赖，尽量让互不依赖的操作并行执行。理解这一层，是理解单线程性能、SIMD 前的标量优化和并发路径的基础。

2.1 流水线不是免费并行

流水线把取指、译码、执行、访存和提交分成阶段。理想情况下，每个周期都有新指令进入，每个阶段都在工作。但真实程序有数据依赖、控制依赖和资源冲突。后一条指令需要前一条指令结果时，执行端口即使空闲也不能使用；分支目标未知时，前端可能取错指令；访存 miss 时，后端会等待数据。

流水线的关键不是“越深越快”，而是吞吐和延迟的权衡。深流水线可以提高频率，却放大分支预测失败的代价。并发程序中的短临界区也会受流水线影响：一个看似只有几条指令的锁操作，背后可能包含原子读改写、cache line 独占获取和内存序约束。

2.2 前端：取指、译码和微操作

核心前端负责把指令流喂给后端。它要从 instruction cache 取指，预测下一条指令地址，译码复杂指令，把它们转成内部微操作，并放入队列。前端跟不上时，后端即使有空闲执行端口也没有工作可做。大函数、过多分支、间接跳转、指令 cache miss 和译码压力都可能让前端成为瓶颈。

x86-64 指令长度可变，译码比定长指令集复杂。现代核心通常会使用 micro-op cache 或类似结构缓存译码结果，让热循环不用每次重新译码。这个机制解释了为什么热循环体大小、函数内联和代码布局会影响性能。内联可以减少调用开销，但过度内联会膨胀代码，伤害 instruction cache 和前端供给。

第一册已经教读者读汇编。第二册读汇编时要多看一个维度：这段机器码对前端是否友好。短小热循环、稳定分支、连续代码布局和可预测调用目标，通常更容易让前端持续供给后端。反过来，大量模板实例化、过度内联、异常路径混在热路径、虚调用目标多变、解释器式大 switch，都可能让前端成本超过源码直觉。

深入理解

前端瓶颈的一个典型现象是：数据已经在缓存中，算术也不复杂，但 IPC 仍然低，并且改变数据布局帮助不大。这时应检查代码体积、分支目标、间接跳转和译码压力，而不是继续只改数组排列。

2.3 后端：执行端口和调度

后端接收微操作后，要等待操作数准备好，并把它们发射到合适执行端口。整数 ALU、浮点乘加、加载、存储、分支和地址生成都有各自资源。一个循环如果每轮需要太多 load，可能受加载端口限制；如果每轮地址计算复杂，可能受地址生成单元限制；如果依赖链很长，端口空着也不能发射。

因此分析单核性能时，要区分“端口吞吐不够”和“依赖导致发不出去”。多累加器能改善后者；SIMD 能提高每条指令处理的数据量；简化地址表达式和连续访问能减轻加载和地址生成压力。不同 CPU 微架构端口配置不同，严肃优化必须结合具体机器。

后端不是一个抽象黑箱。一个 load 通常需要地址生成、TLB 查询、缓存访问和可能的 miss 处理；一个 store 通常拆成地址和数据路径，并进入 store buffer；一个浮点 FMA 会占用特定向量执行资源；一个整数分支会占用分支执行资源。循环的性能取决于这些微操作怎样分布到端口上，也取决于它们之间是否互相等待。

这解释了为什么源码层面的“操作次数”常常误导。两个循环都执行同样数量的加法，一个可能因为单累加器依赖链而慢，另一个可能因为多累加器让后端并行发射而快。两个循环都做同样数

量的 load，一个可能连续访问被预取器支持，另一个可能指针追踪导致每次 load 等上一次 load 的地址。

2.4 乱序执行的边界

乱序执行允许核心在不改变单线程可观察结果的前提下，提前执行已经准备好的指令。寄存器重命名消除了很多假依赖，保留站和重排序缓冲区让多个指令同时等待资源。这样做可以隐藏一部分延迟，例如在一次乘法等待时执行另一个独立加法。

但是乱序执行不能越过所有边界。真实数据依赖不能消除，cache miss 太多时可并行等待的 miss 数量有限，内存屏障会限制重排，原子操作会带来更强的顺序约束。写高性能代码时，增加独立累加器、减少循环携带依赖、让数据访问可预测，都是在帮助乱序核心找到可调度空间。

2.5 重排序缓冲区和执行窗口

乱序执行不是无限的。核心只能观察一个有限窗口内的指令。重排序缓冲区保存尚未提交的指令，load buffer 保存未完成加载，store buffer 保存尚未对外可见的存储。如果窗口被长延迟 load 占住，后续可执行工作又不够，核心会停顿。

链表遍历就是典型例子。每次读取下一个指针之前，必须等当前节点加载完成。乱序核心很难提前知道下一步地址，所以内存级并行度很低。相比之下，遍历多个独立数组或同时处理多条链，可以让多个 load 同时飞行，更容易隐藏延迟。

执行窗口可以被理解为硬件能同时“看见”的未来。窗口越大，越有机会在一个长延迟操作等待时找到其他独立工作。但如果代码中充满循环携带依赖，窗口再大也找不到可提前执行的操作；如果 cache miss 太多，load buffer 被占满，后续加载也发不出去；如果分支预测失败，窗口里的错误路径工作会被丢弃。

软件能做的事情，是把独立工作显式放进窗口。多累加器、分块处理、批量处理多个请求、一次处理多个链表游标、把检查从热循环中外提，都属于给乱序核心提供更多可调度空间。相反，频繁调用不可内联函数、隐藏别名、每步都依赖共享原子结果，会压缩这个空间。

2.6 Load、Store 和内存消歧

内存操作比寄存器操作复杂。核心要判断一个 load 是否可能读取前面尚未完成的 store。如果地址关系不明确，硬件会做内存消歧预测。预测错误时，需要回滚并重新执行相关操作。高性能循环中，减少指针别名、使用清晰的数组边界、避免复杂交叉读写，可以帮助硬件和编译器更大胆地调度。

store buffer 也影响并发语义。普通 store 可以先进入 store buffer，稍后再对其他核心可见。内存屏障、原子操作和锁释放会约束这种行为。C++ 内存模型的 acquire/release 最终要落到这类硬件机制上。

Load/store queue 还是很多微妙性能问题的来源。若一个循环中既写数组又读另一个可能别名的数组，编译器和硬件都必须保守处理读写顺序。若 store 地址很晚才算出来，后面的 load 就难以确认是否可以提前。若 store buffer 堆积，后续 store 可能被阻塞，锁释放和原子操作也可能放大这个问题。

并发程序中，store buffer 让“本线程已经执行了写”不等于“其他线程已经能按你想象的顺序看到写”。这不是说硬件不可靠，而是说可见性需要同步语义。后面讲 release/acquire 时，会把这条硬件直觉变成 C++ 层面的 happens-before 关系。

2.7 分支预测和控制流

分支预测让核心在条件结果出来之前先猜路径。预测正确时，流水线保持饱满；预测失败时，错误路径上的工作要被丢弃。随机分支、数据相关分支和复杂间接跳转都会破坏预测。很多高性能循环会把条件变成掩码、查表或分段处理，就是为了减少不可预测控制流。

并发程序也受分支预测影响。锁的 fast path 通常期望“不竞争”，如果竞争比例上升，分支行为和 cache 行为都会改变。无锁结构的 CAS 循环在低竞争时很短，在高竞争时会反复失败，控制流和内存系统同时恶化。

深入理解

判断一段代码是不是受分支预测影响，可以比较不同输入分布下的 cycles、branches 和 branch-misses。如果随机数据显著慢于有序数据，而内存访问量没有明显变化，分支预测通常就是主要嫌疑。

2.8 SMT 和共享核心资源

SMT 让一个物理核心暴露多个逻辑 CPU。它可以在一个线程等待内存或前端供给不足时，让另一个线程使用空闲资源。但 SMT 不是免费翻倍。两个逻辑线程会共享前端、执行端口、缓存、TLB 和乱序资源的一部分。如果两个线程都在做同样高强度向量计算，它们可能互相抢执行资源；如果一个线程经常等内存，另一个线程可能填补空档。

因此 benchmark 中不能只写“8 个 CPU”。要区分物理核心和 SMT 线程。对于 CPU-bound 数值内核，先扩展到物理核心往往比立刻使用所有逻辑 CPU 更稳定；对于延迟隐藏明显的工作负载，SMT 可能提高吞吐。结论必须来自本机测量，而不是固定经验。

2.9 微架构和 ISA 的边界

第一册强调 ISA 和 ABI：同一条 `add` 在 x86-64 语义上做什么，函数参数怎样传递。第二册强调微架构：同一个 ISA 指令在不同 CPU 上可能拆成不同微操作，使用不同端口，拥有不同延迟和吞吐。ISA 给出程序可见语义，微架构决定实现成本。

这条边界非常重要。写教材时不能把某个 CPU 的端口配置说成 x86-64 的永久性质；写性能报告时也不能只说“x86 上这个指令很快”。更准确的说法是：在某个 CPU 型号、某个编译选项、某个输入分布下，观察到某个指令序列的瓶颈更像前端、端口、依赖、内存或分支。

2.10 典型性能症状

前端瓶颈常表现为 IPC 低、branch miss 或 instruction cache 相关事件高，代码布局和热循环大小值得检查。后端执行瓶颈常表现为数据在缓存中、分支也稳定，但 IPC 仍受端口或依赖限制，适合检查 SIMD、多累加器和指令混合。内存瓶颈常表现为 cache miss、TLB miss 或带宽接近平台上限。同步瓶颈则可能表现为 IPC 低、cache line 迁移、context switch、futex 等待或原子热点。

这些症状不会自动给答案，但能排除方向。不要在内存带宽瓶颈上执着减少几条整数指令，也不要锁竞争瓶颈上只调整循环展开。性能工程的价值在于把“慢”拆成可以证伪的假设。

2.11 从硬件回到代码

核心流水线给软件三个直接启发。第一，减少长依赖链，例如把单个累加器拆成多个局部累加器。第二，减少不可预测分支，让热路径尽量直。第三，减少会强制顺序的操作，例如不必要的原子和屏障。后续章节讨论 SIMD、锁、无锁队列和并行归约时，都会回到这些原则。

2.12 问题案例：为什么一个求和循环也会慢

本章不能只停留在“流水线很深、乱序很强”这种抽象描述上。我们从 Compute Systems Engine 的第一条主线开始：对一个很大的 `std::int32_t` 数组求和。这个问题看起来太简单，甚至像入门练习，但正因为它简单，才适合暴露单核执行的几个基本矛盾：每次循环都有加载、扩展、加法、循环控制和分支；总和变量形成循环携带依赖；数组访问通常可被预取器识别；输入足够大时又会被内存带宽限制。一个求和循环可以同时讲清“依赖、前端、后端、内存和测量”这几条线。

朴素版本通常写成一个累加器。

Listing 2.1 baseline: 单累加器顺序归约

```
1 std::int64_t sum_baseline(std::span<const std::int32_t> values) {
2     std::int64_t sum = 0;
3     for (std::size_t i = 0; i < values.size(); ++i) {
4         sum += values[i];
5     }
```

```

6     return sum;
7 }

```

这个版本的好处是清楚、正确、容易作为 reference。它的弱点也很明显：每一轮的 `sum` 都依赖上一轮的 `sum`。处理器可以提前加载后面的数组元素，也可以预测循环分支，但加法结果本身是一条长链。只要下一次加法必须等上一次加法结果，乱序执行就不能把这些加法任意并行起来。此时瓶颈不一定是加法器数量，而可能是依赖链延迟。

改进版本把一个长依赖链拆成多条短依赖链。

Listing 2.2 改进版本：四个独立累加器

```

1 std::int64_t sum_four_accumulators(std::span<const std::int32_t> values) {
2     std::int64_t sum0 = 0;
3     std::int64_t sum1 = 0;
4     std::int64_t sum2 = 0;
5     std::int64_t sum3 = 0;
6
7     std::size_t i = 0;
8     const std::size_t limit = values.size() / 4 * 4;
9     for (; i < limit; i += 4) {
10         sum0 += values[i];
11         sum1 += values[i + 1];
12         sum2 += values[i + 2];
13         sum3 += values[i + 3];
14     }
15     for (; i < values.size(); ++i) {
16         sum0 += values[i];
17     }
18     return (sum0 + sum1) + (sum2 + sum3);
19 }

```

这个版本不是为了“手动展开看起来高级”，而是给乱序核心更多可调度空间。四个累加器之间没有真实数据依赖，核心可以在一个累加器等待结果时推进另一个累加器。循环体变大，前端压力略增，但依赖链压力下降。若输入在缓存中，这种改法常能提高 IPC；若输入远大于 LLC 并且已经接近内存带宽上限，提升可能很小，因为瓶颈已经从依赖链迁移到数据移动。

还要注意边界处理。第二个循环处理不能被四整除的尾部元素。很多性能示例为了简短直接假设长度是四的倍数，这是教材不应采用的写法。真实系统里，输入规模经常来自文件、网络或上层分片，不能为了微优化丢掉边界正确性。Compute Systems Engine 的 reference 测试必须覆盖空输入、一个元素、非对齐长度、负数和大规模输入，不能只测漂亮输入。

2.13 优化不是越展开越好

既然四个累加器能拆依赖链，是否八个、十六个更好？答案取决于机器、编译器和工作负载。展开过少，依赖链仍然限制后端；展开过多，会增加寄存器压力、代码体积和前端供给压力。寄存器不够时，编译器可能把临时值溢出到栈上，反而引入额外 load/store。代码体积变大时，热循环可能更容易超过微操作缓存或 instruction cache 的舒适范围。

这就是本册反复强调的“瓶颈会迁移”。一个版本慢，是因为单累加器依赖；另一个版本慢，可能因为寄存器压力；再另一个版本慢，可能因为已经受内存带宽限制。优化不能只问“这个技巧有没有用”，要问“在当前证据下，它针对的是哪个瓶颈，改完后瓶颈会迁到哪里”。

教材里常见的坏写法是直接给“最佳版本”。这种写法让读者只会背答案，不会分析。正确写法应保留 baseline，因为 baseline 是 correctness oracle；给出有限改进版本，因为它暴露优化动机；再说明它的边界，因为工程中没有永久最优版本。对求和来说，baseline 用于验证结果，多累加器用于解释依赖链，SIMD 版本用于解释向量执行，线程本地 partial 用于解释共享写和 false sharing，多线程

程版本用于解释带宽和 NUMA。这些版本不是重复代码，而是同一个问题在不同系统层面的投影。

2.14 不要把共享原子当作并行归约

很多初学者把并行求和写成多个线程同时对一个原子变量做加法。这个版本结果可能是正确的，却几乎一定不是好的归约实现。

Listing 2.3 反例：所有 worker 争用同一个原子和同一条 cache line

```

1 std::int64_t sum_with_hot_atomic(std::span<const std::int32_t> values,
2                                 std::int32_t worker_count) {
3     std::atomic<std::int64_t> global_sum{0};
4     std::vector<std::thread> workers;
5     workers.reserve(static_cast<std::size_t>(worker_count));
6
7     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
8         const std::size_t begin =
9             values.size() * static_cast<std::size_t>(worker) /
10             static_cast<std::size_t>(worker_count);
11         const std::size_t end =
12             values.size() * static_cast<std::size_t>(worker + 1) /
13             static_cast<std::size_t>(worker_count);
14         workers.emplace_back([&values, &global_sum, begin, end]() {
15             for (std::size_t i = begin; i < end; ++i) {
16                 global_sum.fetch_add(values[i], std::memory_order_relaxed);
17             }
18         });
19     }
20
21     for (std::thread& worker : workers) {
22         worker.join();
23     }
24     return global_sum.load(std::memory_order_relaxed);
25 }
```

这里使用 `memory_order_relaxed` 并没有错，因为这个原子只用于统计自身数值，不发布其他对象。但 `relaxed` 只能减少顺序约束，不能消除一致性争用。所有线程仍在更新同一个 cache line。每一次 `fetch_add` 都要求对应 cache line 获得独占修改权，多个核心会反复争夺所有权。结果是用户以为自己写了“无锁高性能”，硬件看到的是一个极热的共享写点。

正确方向通常是线程本地 `partial`。

Listing 2.4 工程版本：线程本地 `partial`，再串行合并

```

1 struct alignas(64) PartialSum {
2     std::int64_t value = 0;
3 };
4
5 std::int64_t sum_with_partials(std::span<const std::int32_t> values,
6                                std::int32_t worker_count) {
7     std::vector<PartialSum> partials(static_cast<std::size_t>(worker_count));
8     std::vector<std::thread> workers;
9     workers.reserve(static_cast<std::size_t>(worker_count));
10
11     for (std::int32_t worker = 0; worker < worker_count; ++worker) {
12         const std::size_t begin =
```

```

13         values.size() * static_cast<std::size_t>(worker) /
14         static_cast<std::size_t>(worker_count);
15     const std::size_t end =
16         values.size() * static_cast<std::size_t>(worker + 1) /
17         static_cast<std::size_t>(worker_count);
18     workers.emplace_back([&values, &partials, worker, begin, end]() {
19         std::int64_t local = 0;
20         for (std::size_t i = begin; i < end; ++i) {
21             local += values[i];
22         }
23         partials[static_cast<std::size_t>(worker)].value = local;
24     });
25 }
26
27 for (std::thread& worker : workers) {
28     worker.join();
29 }
30
31 std::int64_t total = 0;
32 for (const PartialSum& partial : partials) {
33     total += partial.value;
34 }
35 return total;
36 }

```

这个版本把热路径写共享变成线程本地写。每个 worker 在自己的寄存器里累加，结束时只写一次 partial。对齐槽位用于避免多个 partial 落在同一条 cache line 上形成 false sharing。它的代价也要写清楚：需要额外 partial 数组，需要合并阶段，线程创建和 join 也有成本。如果输入很小，线程成本可能超过计算成本；如果输入很大，局部顺序扫描加一次合并通常比热原子好得多。

2.15 从源码到汇编应该看什么

第一册已经讲过如何读汇编。第二册读汇编不是为了炫耀指令名，而是为了验证源码是否产生了符合预期的执行形态。看求和循环时，应至少关注四件事。

第一，看编译器是否保留了循环。若结果没有被使用，优化器可能删除整个循环，benchmark 就变成空测。第二，看循环是否形成单累加器依赖，还是被编译器展开成多个累加器。现代编译器在优化级别较高时可能自动做循环展开和向量化，因此手写版本和编译器生成版本要放在同一编译选项下比较。第三，看是否有意外的内存访问。例如寄存器压力过大导致 spill，会在热循环中出现额外栈访问。第四，看是否有分支和调用留在热路径里。例如调试检查、虚调用、异常路径或不可内联函数都可能改变前端压力。

下面是一份实验报告可以采用的观察顺序。先用 **objdump** 或编译器的汇编输出确认热循环；再用 **perfstat** 比较 cycles、instructions 和 IPC；然后改变输入规模，让数据分别处在 L1、L2、LLC 和内存范围；最后再改变版本，从 baseline 到多累加器到线程本地 partial。若只看一个规模，很容易把依赖瓶颈和内存瓶颈混在一起。

深入理解

同一个源码版本，在不同编译器和不同优化级别下可能生成不同机器码。教材里的结论必须写成“分析方法”，不能写成“某段源码永远会生成某条指令”。严肃报告要记录编译器版本、目标架构、优化选项和关键汇编片段。


```

3     std::int64_t count = 0;
4     for (const std::int32_t value : values) {
5         count += value > threshold ? 1 : 0;
6     }
7     return count;
8 }

```

这段源码仍然写了条件表达式，编译器可能生成条件移动、集合指令或向量比较，也可能仍生成分支，具体取决于目标架构和优化器。教材重点不是保证某种机器码，而是教读者提出可验证假设：随机输入下 branch miss 是否更高；无分支版本 instructions 是否增加；总时间是否下降；当输入分布从随机变成有序后，两个版本差距是否消失。如果这些现象吻合，就说明分支预测是主要因素之一。

分支优化也有代价。把复杂 if 改成掩码运算，可能让所有路径都执行，增加算术或内存访问；把数据分组处理，可能增加预处理成本；把虚调用改成分发表，可能降低可读性。不要把“branchless”当作普遍更快。只有当分支不可预测且热路径短，减少分支才经常有收益。

2.18 前端瓶颈案例：热循环里的调用和大 switch

很多性能分析只盯着数据 cache，却忽略了前端供给。前端要取指、预测、译码和提供微操作。若热路径里有过多不可内联调用、异常边、模板膨胀后的大代码体、解释器式大 switch 或间接跳转，后端可能一直等不到足够微操作。

一个典型案例是记录处理器。每条记录包含类型字段，不同类型执行不同处理逻辑。朴素实现可能把所有逻辑放进一个巨大的 switch，每个 case 又调用复杂函数。若输入类型分布随机，间接分支或多路分支预测困难；若每个 case 代码很大，instruction cache 和微操作缓存压力上升。优化方向可能不是“少做几次整数加法”，而是重排数据：先按类型分桶，再批量处理同类记录。这样做改变了控制流形状，让分支更稳定，也让热代码更集中。

这类优化有一个重要边界：分桶会增加一次额外遍历和写入，还可能破坏原始顺序。若语义要求严格保持输入顺序，就需要额外索引或稳定输出；若记录数量很少，分桶成本超过收益；若每个记录处理非常重，分支成本可能不是主导。系统工程的结论永远要回到工作负载。

2.19 内存级并行度和指针追踪

乱序执行能隐藏一部分内存延迟，前提是窗口里有多个独立 load。顺序数组扫描虽然会 cache miss，但预取器能提前请求后续 cache line，多个 miss 可以同时飞行。链表遍历则不同：下一步地址存放在当前节点里，必须等当前节点加载完成后才知道下一次 load 地址。这种指针追踪几乎把内存访问串成依赖链。

下面的代码形态看起来只是遍历节点，硬件看到的是一条地址依赖链。

Listing 2.8 指针追踪：下一次加载地址依赖当前加载结果

```

1 struct Node {
2     std::int32_t value = 0;
3     Node* next = nullptr;
4 };
5
6 std::int64_t sum_list(const Node* head) {
7     std::int64_t sum = 0;
8     const Node* current = head;
9     while (current != nullptr) {
10         sum += current->value;
11         current = current->next;
12     }
13     return sum;
14 }

```

这段代码不适合作为热路径大规模数据扫描的默认结构。它的每个节点可能来自不同页面，cache miss 和 TLB miss 都可能很高；下一节点地址不可提前知道，内存级并行度低；分配器还可能把节点分散到不同 NUMA 节点。若业务语义允许，把节点压缩成连续数组或索引数组，通常更适合现代 CPU。若必须使用链式结构，可以考虑批量处理多条独立链，让核心同时等待多个地址链；也可以使用池化分配，让节点尽量连续；还可以把热字段和冷字段分离，减少每个 cache line 带入的无用数据。

2.20 和编译器优化的边界

有些读者会问：既然编译器能展开、向量化、调度指令，为什么还要学习微架构？答案是，编译器非常强，但它不是业务语义的读心器。编译器不知道某个输入分布是否随机，不知道两个 span 在业务上是否必不重叠，不知道一把锁竞争是否来自热点 key，不知道任务粒度是否会造成线程池空转，也不知道分布式 shuffle 是否会遇到倾斜。它只能在 C++ 语义、目标架构和优化启发式内工作。

软件工程师要做的是把真实语义表达给编译器和硬件。用连续容器表达顺序访问，用只读 span 表达输入不可修改，用线程本地 partial 表达写入独占，用稳定的数据分块表达可预测访问，用测试保证边界输入正确。高性能代码不是处处和编译器对抗，而是提供更清晰的结构，让编译器能安全优化，让硬件能持续执行。

2.21 Linux 与工具源码阅读入口

本章的 Linux 源码阅读不应从调度器全局策略开始，而应从性能证据的来源开始。若使用 `perfstat` 观察 cycles、instructions、branches 和 branch-misses，可以阅读 Linux 内核的 perf event 子系统入口，理解用户态工具如何请求硬件计数器、内核如何复用和调度计数器、权限如何限制某些事件。体系结构相关事件通常在 `arch/x86/events/` 一类目录下，通用框架在 `kernel/events/` 附近。不同内核版本路径会变化，读源码时必须记录版本。

编译器源码阅读可以从优化报告开始，而不是直接钻进后端。GCC 和 Clang 都能输出向量化或优化诊断。读者可以先写两个循环，一个容易向量化，一个因为别名或分支不容易向量化；观察诊断后，再选择性阅读对应编译器文档或源码。目标不是成为编译器开发者，而是理解“为什么这段循环没被优化”的证据链。

2.22 本章落到贯穿项目

Compute Systems Engine 在本章至少要得到三个能力。第一，顺序 reference: `sequential_sum` 保持最直接写法，用它验证所有优化版本结果。第二，单核优化版本：增加多累加器或向量化实验，但每个版本都要保留 correctness check。第三，测量 harness：能以相同输入、相同编译选项、相同计时边界比较 baseline、多累加器、热原子和线程本地 partial。这里的目标不是立刻写最强归约，而是建立“同一问题跨层演化”的实验骨架。

实验报告建议按下面结构写。先说明输入规模、数据类型和正确性 oracle；再说明 CPU 型号、编译器、优化选项、线程数和是否绑定；然后列出版本：baseline、四累加器、热原子、多线程 partial；最后解释每个版本的预期瓶颈和实际指标。若四累加器没有变快，不能直接说技巧无效，要检查编译器是否已经自动展开、输入是否被内存带宽限制、计时是否包含线程创建、以及结果是否被优化掉。

2.23 本章习题

习题与作业

习题一：为 `sum_baseline`、`sum_four_accumulators` 和当前项目里的 `parallel_sum` 写同一组正确性测试。输入必须包含空数组、单元元素数组、长度不是四的倍数的数组、包含负数的数组和足够大的随机数组。报告每个测试验证的不是“跑得快”，而是哪一种语义边界。

习题二：构造两个 `count_greater` 输入。第一个输入使分支非常可预测，第二个输入

使分支接近随机。用 `perfstat` 记录 `branches` 和 `branch-misses`。若当前设备权限不足，写出你本来要运行的命令、预期现象和无法运行的原因。

习题三：把一个链表求和改写成连续数组求和。要求说明两版在源码复杂度、内存布局、cache line、TLB、预取和对象生命周期上的差异。不要只写“数组更快”，要写清为什么更可能快，以及在哪些业务语义下不能直接替换。

习题四：阅读当前项目的 `books/cpu-volume-2/labs/compute_systems/src/parallel_reduce.cpp`。指出 `parallel_sum` 的 correctness oracle、任务切分方式、线程创建成本、partial 写入位置和可能的 false sharing 风险。给出一个改进计划，但不要直接删除 baseline。

实验：写两个求和版本。第一个使用单累加器，第二个使用四个独立累加器。用 `perfstat` 比较 `cycles`、`instructions` 和 `IPC`。再构造一个带随机分支的版本，观察 `branch-misses` 对时间的影响。

第 3 章

缓存、TLB 与页级性能

CPU 核心很快，主内存相对很慢。缓存层级的存在，是为了让核心大多数时候从更近、更小、更快的存储中取数据。虚拟内存的存在，是为了让进程拥有独立地址空间、权限保护和按需映射。第一册已经讲过进程地址空间、映射和缺页的基本概念；本章不重复这些基础，而是继续追问：地址翻译、缓存映射、写共享、page walk、首次触页和 NUMA 放置怎样限制多核程序的吞吐。

3.1 缓存层级与 cache line

缓存不是按单个字节移动数据，而是按 cache line 移动。常见 cache line 大小是 64 字节。顺序访问数组时，一次 miss 可以带来后续多个元素；随机访问链表时，每个节点可能引入一次新的 miss。这个差异解释了为什么连续数组常常比指针结构更快。

缓存有容量、相联度和替换策略。容量不足会产生容量 miss，相联度不足会产生冲突 miss，多核心共享写会产生一致性流量。一个算法理论复杂度更低，不代表真实更快；如果它把连续访问变成随机访问，可能输给复杂度略高但局部性更好的算法。

从第二册的角度看，cache line 还是多核协作的最小一致性单元。两个线程哪怕写的是两个不同字段，只要字段落在同一条 cache line 上，硬件就必须在核心之间移动同一个一致性单元。很多并发程序的扩展性问题，不是算法复杂度问题，而是写入布局问题。后面讲锁、原子和无锁队列时，都要回到这个事实。

```
one cache line, 64 bytes in a common x86 machine

| counter[0] | counter[1] | counter[2] | counter[3] | ...
    T0         T1         T2         T3

looks independent in source code
shares one coherence unit in hardware
```

这类图要让读者形成一个直觉：源码里的变量边界，不等于硬件里的传输边界。硬件搬的是 cache line，TLB 记的是页，操作系统调度的是线程，分布式系统发送的是消息。边界错位，是系统性能问题最常见的来源之一。

3.2 缓存映射和冲突

缓存通常按 set associative 组织。一个地址会映射到某个 set，同一个 set 里只有有限个 way。如果多个热点地址恰好映射到同一个 set，而数量超过相联度，就会频繁互相驱逐，即使总工作集小于缓存容量也会慢。这类冲突 miss 很隐蔽，因为从容量角度看“不应该放不下”。

矩阵步长、数组对齐和结构体大小都可能触发冲突。若多个数组按相同大步长访问，并且基地址关系不佳，它们可能持续争用同一组 cache set。改变 padding、改变块大小或调整访问顺序，有时能显著改善性能。

3.3 Inclusive、Exclusive 和共享缓存

不同处理器的缓存层级策略不同。有的层级倾向 inclusive，也就是上层 cache 的内容也在下层保留；有的倾向 exclusive，尽量让不同层级存放不同内容；还有很多混合策略。软件通常不需要依赖某个具体策略，但要知道共享 LLC 不是无限资源。多个线程在同一 socket 上跑不同任务，可能互相驱逐 LLC。

这对并行任务调度有直接影响。把共享数据的任务放在共享缓存附近，可能复用 LLC；把互相干扰的大流式任务放在同一 socket，可能争抢带宽和 LLC。操作系统调度器会做一般性决策，但高

性能程序常需要自己记录拓扑并做亲和性实验。

3.4 写路径和 false sharing

读缓存可以共享，写缓存必须取得 cache line 的独占所有权。两个线程写不同变量，如果这些变量落在同一条 cache line 上，硬件仍然会把它当成同一块一致性单元反复迁移，这就是 false sharing。它不是锁竞争，却会表现出类似的扩展性下降。

解决 false sharing 的方法包括按线程分配独立槽位、使用对齐和填充、批量合并结果、减少共享写频率。并行归约通常先让每个线程写自己的 partial sum，最后再合并，正是为了避免每次加法都写同一个共享变量。

Listing 3.1 用对齐槽位减少 false sharing

```
1 struct alignas(64) LocalCounter {
2     std::int64_t value = 0;
3 };
4
5 std::vector<LocalCounter> counters(worker_count);
6 for (std::int32_t worker = 0; worker < worker_count; ++worker) {
7     counters[static_cast<std::size_t>(worker)].value += 1;
8 }
```

这段代码不是说所有结构体都要强行 64 字节对齐，而是表达一个设计原则：如果多个线程会高频写不同槽位，这些槽位不要共享同一条 cache line。否则程序表面没有共享变量，硬件层面仍然在共享一致性单元。

false sharing 的危险在于它不像数据竞争那样直接破坏结果。程序可能完全正确，测试也全部通过，只是在核心数增加时吞吐突然停止增长。排查时要看写入频率、字段布局、线程到槽位的映射和 cache line 迁移事件。若没有 `perfctr` 或类似工具，也可以用对齐填充做对照实验：只改变布局，不改变算法。如果对齐版本显著更快，就说明共享写路径值得重点检查。

3.5 读共享和写共享

共享数据不总是坏事。只读共享表、常量配置、不可变索引和代码段可以被多个核心同时缓存，通常扩展性很好。真正昂贵的是写共享，尤其是多个核心高频写同一条 cache line。读多写少的数据可以使用读写锁、RCU、版本化快照或 copy-on-write，目的都是让读路径尽量不进入写共享热路径。

写共享也分层次。最差情况是每次操作都写同一个全局变量，例如所有线程对一个原子计数器做 `fetch_add`。较好的情况是每个线程写本地槽位，周期性合并。更好的情况是让写入天然按数据分片分散，例如按 key 分片的哈希表或按 NUMA 节点分组的内存池。布局、同步和算法在这里是同一个问题的三个面。

3.6 TLB 和页表

虚拟地址访问物理内存前需要地址翻译。TLB 缓存虚拟页到物理页的映射。工作集页数太多或访问模式太随机时，TLB miss 会增加，核心需要走页表。大页可以减少页数，提高 TLB 覆盖范围，但也会影响内存分配、NUMA 放置和碎片。

理解 TLB 对数据结构设计很重要。一个跨很多小对象的图遍历，不只会造成 cache miss，还可能造成 TLB miss。紧凑存储、池化分配和按块遍历，可以同时改善 cache 和 TLB 行为。

TLB 的特殊之处在于，它限制的是“能高效覆盖多少虚拟页”。一个程序的工作集如果只看字节数不大，但分散在大量页面上，仍然可能受到 TLB miss 影响。链表、树、哈希表桶、对象池和图邻接结构都可能出现这个问题。把对象压缩到连续数组里，不只是减少 cache miss，也是在减少活跃页数。

3.7 Page walk 的成本

TLB miss 后，硬件需要按页表层级查找映射。x86-64 常见四级或五级页表，每一级页表项本身也在内存里，虽然页表项可能被缓存，但最坏情况下一个地址翻译会触发多次内存访问。若程序随机访问大量页面，真正拖慢它的不只是数据 load，还有地址翻译。

大页把页大小从 4 KiB 扩大到 2 MiB 或更大，显著增加 TLB 覆盖范围。它适合大数组、数据库缓存和数值计算工作集，但不适合所有场景。大页可能增加内存碎片，NUMA 放置也要更谨慎。使用大页前应先使用计数器确认 TLB 确实是瓶颈。

Page walk 还会和缓存层级交织。页表项本身也要从内存层级读取，也可能命中或错过缓存。一个随机访问大数组的程序，表面上每次只读一个元素，实际上可能同时触发数据 cache miss 和页表访问。若工作集跨越大量页面，TLB miss 的成本会叠加在数据访问成本上。优化时不能只看“每个元素读几个字节”，还要看“每个时间窗口内活跃多少页”。

3.8 缺页和内存分配

分配内存不等于物理页已经准备好。操作系统通常按需分配页面，第一次触碰页面时可能发生 page fault。benchmark 如果把第一次触碰计入核心循环，会把页错误成本混入算法时间。严谨实验通常要预热、固定输入规模，并记录 page-faults。

缺页也分 minor fault 和 major fault。minor fault 不需要从磁盘读数据，可能只是建立页表映射；major fault 需要从存储设备读取，成本高得多。高性能计算通常要避免在热路径出现 major fault，也要避免把大量 minor fault 误判为算法本身慢。

第一册已经要求 benchmark 避免把初始化混进热路径。第二册要进一步要求初始化方式和计算方式一致。NUMA 机器上，如果主线程单独触碰全部页面，再让多个 socket 上的 worker 计算，页面可能集中在主线程所在节点，后续计算变成远端内存访问。正确实验通常让每个 worker 初始化自己之后要处理的分块，这样 page fault、物理页分配和计算拓扑保持一致。

3.9 从虚拟内存到 mmap

mmap 可以把文件或匿名内存映射到进程地址空间。它让文件 I/O 看起来像内存访问，但不意味着没有 I/O 成本。第一次访问未驻留页面时仍然会缺页，顺序扫描大文件时内核预读可能有效，随机访问时可能产生大量 page fault。

数据库、搜索引擎和日志系统常利用 mmap 或 page cache，但必须理解内核何时回收页面、何时写回脏页、何时产生 I/O 抖动。系统工程里，虚拟内存既是便利抽象，也是性能变量。

3.10 Linux 源码阅读入口

第一册的 Linux 源码阅读只要求读者能从用户态现象找到入口。第二册要读出数据结构和性能路径。围绕本章，可以固定一个 Linux 版本，沿下面几条路径做窄范围阅读：页错误入口可从体系结构相关的 fault handler 进入，再走到通用内存管理逻辑；VMA 管理可关注 `mm/mmap.c` 和相关 maple tree 结构；页表和 fault 处理可关注 `mm/memory.c`；NUMA 策略可关注 `mm/mempolicy.c`；page cache 和文件映射可从 `mm/filemap.c` 开始。

源码阅读报告不要写成“Linux 使用某某机制”这么宽泛。合格报告要写清内核版本、阅读路径、关键结构、用户态实验和结论边界。例如研究首次触页，可以写一个大数组实验，记录 page-faults，再阅读 fault 路径中如何建立匿名页映射。若当前设备权限不足，仍然可以保留源码阅读，但必须承认它不是当前运行内核的直接证据。

3.11 本章的设计原则

缓存和 TLB 优化可以压缩成四条原则。第一，让热路径连续访问，减少随机指针跳转。第二，让共享写分散，避免多个线程争用同一条 cache line。第三，让工作集按块复用，避免超出缓存后仍反复访问。第四，让页面放置和线程执行位置一致，避免首次触页造成远端访问。这四条原则会贯穿后面的数据布局、并行归约、线程池、工作窃取和分布式 shuffle。

常见误区

不要把 `std::vector` 的连续性理解成所有成本都消失了。连续布局改善 cache line 和预取，但如果容量超过缓存，仍然会受内存带宽限制；如果首次访问大量新页，仍然会看到缺页成本。

3.12 从变量边界到硬件边界

本章最重要的模型，是把软件边界和硬件边界分开。C++ 源码里，`counter[0]` 和 `counter[1]` 是两个不同元素；硬件里，如果它们落在同一条 cache line 上，写入时就是同一个一致性单元。C++ 里，两个对象可能由两个 `new` 表达式产生；分配器里，它们可能挨在同一个页上，也可能散落到不同页。进程里，虚拟地址看起来连续；物理页可能分布在不同 NUMA 节点。性能问题经常不是因为某一层“错了”，而是因为层与层的边界没有对齐。

因此分析内存性能时，不要只画类图和对对象关系，还要画四种边界。第一是对象边界，说明字段和容器怎样组织。第二是 cache line 边界，说明哪些字段会一起被搬运和一致性维护。第三是页边界，说明工作集跨越多少页，TLB 是否能覆盖。第四是 NUMA 边界，说明页面在哪个节点，线程在哪个节点执行。只有对象图没有硬件边界，分析会停留在源码层；只有硬件术语没有对象图，优化又会脱离业务语义。

Compute Systems Engine 的 partial 数组就是一个很好的例子。源码里每个 worker 写一个 partial，逻辑上互不共享；如果每个 partial 只是一个 `std::int64_t`，八个 partial 可能正好落在同一条 64 字节 cache line 上。八个 worker 写不同元素，却让同一条 cache line 在核心之间迁移。若给每个 partial 做 `alignas(64)`，源码上的数组变胖了，内存占用增加了，但写共享路径被拆开。这里没有绝对答案，只有工作负载下的权衡：若 worker 很少或写入很少，padding 可能不值得；若每个 worker 高频写 partial，对齐常常值得。

3.13 局部性不是一个词

局部性至少有三种。时间局部性表示刚访问过的数据很可能很快再次访问；空间局部性表示访问一个地址后，附近地址很可能被访问；结构局部性表示程序的数据结构让相关数据天然挨在一起。很多教材只讲时间和空间局部性，但工程中结构局部性更容易被设计改变。

数组顺序扫描空间局部性很好，时间局部性可能不强，因为每个元素只用一次。矩阵分块同时制造时间局部性和空间局部性，因为一个块中的数据会被多次复用。哈希表查询如果 key 随机，空间局部性差；若桶数组连续但节点链表分散，第一跳和后续跳的局部性还不同。图算法更复杂，邻接表可能连续，但顶点状态数组访问可能随机。说“这个算法局部性不好”太粗，应说明是哪一类局部性不好，坏在数据结构的哪个位置。

局部性还要和读写性质一起看。只读随机访问通常只是慢，写随机共享访问可能同时慢且影响其他核心。只读大表可以被多个核心缓存；写热点计数器会触发一致性迁移；写不同字段如果落在同一 cache line 上，会形成 false sharing。后续讲无锁队列时，head 和 tail 的布局就是典型局部性问题：把它们放近可以减少对象体积，但若生产者写 tail、消费者写 head，放太近反而让两端互相干扰。

3.14 案例：三种计数器的内存路径

为了把缓存一致性讲清楚，可以比较三个计数器版本。第一个版本用 mutex 保护全局计数器，第二个版本用 atomic 全局计数器，第三个版本用分片计数器。三者都能给出正确总数，但内存路径完全不同。

Listing 3.2 baseline: mutex counter 表达最清楚的不变量

```
1 class MutexCounter {
2 public:
3     void add(std::int64_t delta) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->value_ += delta;
```



```

6     }
7
8     std::int64_t value() const {
9         std::lock_guard<std::mutex> lock(this->mutex_);
10        return this->value_;
11    }
12
13 private:
14     mutable std::mutex mutex_;
15     std::int64_t value_ = 0;
16 };

```

mutex 版本的语义最直接：锁保护 `value_`。低竞争时，它可能只走用户态 fast path；竞争上升后，线程可能进入 futex 等待，调度器和唤醒成本会进入系统。它的瓶颈通常是临界区串行化和锁等待，不只是那一次整数加法。

Listing 3.3 改进但不一定扩展：atomic counter

```

1 class AtomicCounter {
2 public:
3     void add(std::int64_t delta) {
4         this->value_.fetch_add(delta, std::memory_order_relaxed);
5     }
6
7     std::int64_t value() const {
8         return this->value_.load(std::memory_order_relaxed);
9     }
10
11 private:
12     std::atomic<std::int64_t> value_{0};
13 };

```

atomic 版本消除了锁等待和内核睡眠路径，但所有线程仍写同一 cache line。低竞争时它可能比 mutex 快；高竞争时它可能因为 cache line 所有权迁移而扩展性很差。这里的 relaxed 只说明不需要用计数器同步其他数据，不说明这次写没有硬件成本。

Listing 3.4 扩展方向：按 worker 分片计数

```

1 struct alignas(64) CounterShard {
2     std::int64_t value = 0;
3 };
4
5 class ShardedCounter {
6 public:
7     explicit ShardedCounter(std::int32_t shard_count)
8         : shards_(static_cast<std::size_t>(shard_count)) {}
9
10    void add(std::int32_t shard, std::int64_t delta) {
11        this->shards_[static_cast<std::size_t>(shard)].value += delta;
12    }
13
14    std::int64_t value() const {
15        std::int64_t total = 0;
16        for (const CounterShard& shard : this->shards_) {

```



```

17         total += shard.value;
18     }
19     return total;
20 }
21
22 private:
23     std::vector<CounterShard> shards_;
24 };

```

分片版本把高频写入分散到多个 cache line。它牺牲了读取总数的成本和实时性：每次读取总数要扫描所有分片；如果读取和写入并发，还需要额外同步或定义弱一致语义。它适合写多读少的统计路径，不适合每次操作都需要立即得到全局精确值的协议。这个案例说明高性能设计常常不是“换一个更快原语”，而是改变读写比例和共享形状。

3.15 false sharing 的对照实验

false sharing 很适合用对照实验学习，因为它不改变算法结果，只改变布局。实验可以构造两个版本。版本一使用紧凑数组，每个线程反复写自己的槽位；版本二使用 `alignas(64)` 的槽位。两个版本的循环次数、线程数和写入次数完全相同，只改变槽位是否共享 cache line。如果对齐版本在多线程下显著更快，而单线程差异不大，就说明写共享边界是关键变量。

实验报告要避免两个误区。第一，不要只跑一个线程数。false sharing 的危害随线程数和核心拓扑变化，至少应从 1 线程跑到物理核心数。第二，不要只看耗时。若工具允许，应观察 cache line 迁移或相关事件；若没有权限，也应写出对照设计为什么能隔离变量。第三，不要把 padding 当成默认答案。对齐会增加内存占用，可能降低缓存容量利用率。只有高频并发写的槽位才优先考虑 padding。

3.16 TLB 覆盖范围和数据结构

很多人把 TLB miss 当成“虚拟内存细节”，但它会直接改变数据结构选择。假设页面大小是 4 KiB，一个数据结构每个节点只有几十字节，但节点随机分布在几百万个页面上。每次访问节点不仅可能 cache miss，还可能 TLB miss。TLB 覆盖范围是有限的，活跃页数超过覆盖范围后，地址翻译本身就会变成瓶颈。

紧凑数组同时改善两件事：一条 cache line 带来多个相邻元素，一个 TLB 项覆盖同一页内多个元素。对象池也有类似作用：即使保留指针结构，至少让节点来自较少的连续页面。图算法中的 CSR 格式把邻接边压成连续数组，不只是为了少分配，也是为了提高 cache 和 TLB 友好性。哈希表开放寻址常常比链地址法更 cache 友好，也部分因为它减少指针跳转和页面分散。

大页是另一种提高 TLB 覆盖范围的方法。把 4 KiB 页换成 2 MiB 页，同样数量的 TLB 项能覆盖更大地址范围。它适合大数组和长时间驻留的大工作集，但要考虑碎片、权限、部署和 NUMA 放置。不要一看到 TLB miss 就无脑开大页。正确顺序是先证明活跃页数和 TLB miss 是瓶颈，再评估大页对分配、首次触页和系统配置的影响。

3.17 首次触页和 NUMA 的隐藏变量

NUMA 机器上，页面通常在首次写入时分配到触碰它的线程所在节点。若主线程分配并初始化整个数组，然后把计算分给其他 socket 上的线程，其他线程可能大量访问远端内存。程序源码里看不到这个问题，因为数组地址连续、线程切分也正确；硬件拓扑里却出现远端访问。

这就是为什么 benchmark 的初始化也属于实验设计。若计算阶段按 worker 分块，那么初始化阶段也应按同样分块让 worker 触碰自己的页面。这样 page fault、物理页分配和计算线程位置更一致。若为了方便只让主线程初始化，测到的可能不是算法性能，而是错误页面放置后的远端带宽。

Compute Systems Engine 后续要把输入生成拆成两种模式。第一种是 single-touch，由主线程初始化，用于展示错误实验设计可能带来的远端访问。第二种是 parallel first-touch，由每个 worker 初始化自己负责的块，用于展示正确放置。两者结果如果在单 socket 机器上接近，在多 socket 机器上可能差异明显。教材不能假设每个读者都有 NUMA 设备，但要让读者知道该怎样设计实验。

3.18 矩阵访问：从行列顺序到分块

矩阵是讲缓存最经典的案例。行优先存储的二维数组按行扫描，连续访问；按列扫描，步长等于一行长度，若矩阵很大，每次访问都可能落到新的 cache line 甚至新的页。这个例子不能只停在“行优先快”上，还要继续推进到分块。

矩阵乘法中，朴素三重循环会反复访问矩阵块。如果循环顺序不合适，某些数据刚被加载就被驱逐，下次又要从更慢层级取回。分块的思想是让一小块数据在缓存中被多次复用，提高算术强度。块太小，循环开销和复用不足；块太大，超过缓存后又失效。块大小还与 cache 容量、相联度、TLB、SIMD 宽度和数据类型有关。

这个案例的意义不是要求第二册变成矩阵计算专著，而是让读者学会一类推理：先估算工作集大小，再估算每个块的数据复用，再根据缓存层级选择实验矩阵。后面讲分布式 shuffle 时，思想相同：先本地聚合减少网络数据，再按分区批量发送，避免每条记录都独立跨网络移动。cache block 和 network batch 在不同尺度上解决同一个问题：让昂贵移动服务更多有用计算。

3.19 分配器和对象布局

C++ 程序常把性能问题归咎于容器，却忽略分配器。频繁小对象分配会让对象分散、增加元数据开销、破坏 cache 和 TLB 局部性，还可能引入锁竞争。多线程分配器通常有线程本地缓存、arena 或 tcache 机制，用于减少全局锁，但也会影响对象在哪个线程、哪个 NUMA 节点上实际分配。

对象池不是万能答案。它可以改善局部性和分配成本，但也会引入生命周期管理、碎片复用、调试困难和峰值内存占用问题。如果对象跨线程释放，池的所有权和回收路径又会变复杂。无锁数据结构尤其要小心：节点从结构中摘除，不等于可以立即释放；分配器复用同一地址还可能放大 ABA 问题。缓存、TLB、分配器和内存回收最终会在无锁章节汇合。

3.20 写回、脏页和 I/O 误判

并不是所有内存成本都来自 DRAM。文件映射、page cache 和脏页写回会把内存访问和 I/O 混在一起。一个程序用 `mmap` 扫描文件，第一次访问可能触发缺页和磁盘读取；写入映射文件后，脏页可能稍后由内核写回，导致延迟在看似无关的时刻出现。若 benchmark 只测用户态循环，不记录 page faults 和 I/O，就可能把系统行为误判为算法波动。

日志系统和数据库特别容易遇到这个问题。用户写入内存缓冲并不等于数据已经安全落盘；调用 `fsync` 又会把持久化成本集中暴露出来。第二册虽然重点是计算系统，但分布式计算章节会讨论检查点和输出提交，那里必须区分“写入内存”“写入 page cache”“持久化到存储”和“对外宣布提交”。单机虚拟内存知识会直接影响分布式正确性。

3.21 本章落到贯穿项目

Compute Systems Engine 在本章至少要增加四类实验。第一，partial 对齐实验：比较紧凑 partial 和 `alignas(64)` partial，观察多线程写入扩展曲线。第二，访问模式实验：比较顺序数组、固定步长、随机索引和链表遍历。第三，页级实验：改变工作集跨越页数，记录 page faults 和可能的 dTLB 事件。第四，first-touch 实验：在支持 NUMA 的机器上比较主线程初始化和 worker 分块初始化。

这些实验都必须有 reference。访问模式版本必须返回同样的逻辑结果；partial 对齐版本必须只改变布局，不改变算法；first-touch 版本必须分离初始化时间和计算时间，同时也可以单独报告初始化成本。若实验一次改变多个变量，结论就不可靠。

3.22 本章习题

习题与作业

习题一：在当前项目中为 partial 数组设计两个布局，一个紧凑，一个按 cache line 对齐。说明每个字段的大小、对齐、每个 cache line 可能容纳几个 partial，以及为什么这个设计能隔离 false sharing。

习题二：写三个遍历版本：连续数组遍历、随机索引数组遍历、链表遍历。要求三者处理相同数量的逻辑元素。报告中分别分析 cache line、TLB、预取器和内存级并行度，不

允许只写“随机慢”。

习题三：设计一个首次触页实验。说明如何分离分配、初始化、预热和计算；如何记录 page-faults；在没有 NUMA 机器时，哪些结论不能验证，哪些结论仍然可以通过 page fault 观察。

习题四：阅读 `mm/memory.c`、`mm/mmap.c` 或对应内核版本中的 fault 路径入口。写一份不超过两页的源码阅读报告，要求把用户态实验现象和内核路径联系起来，不要罗列无关函数。

实验：比较行优先访问矩阵和列优先访问矩阵。记录 cache-misses、dTLB 相关事件和时间。再把矩阵规模从 L1、L2、L3 可容纳范围逐步扩大，观察拐点。

第 4 章

NUMA、互连与缓存一致性

多核心 CPU 不是把多个核心简单放在一起。核心之间通过片上互连通信，共享某些缓存层级，连接内存控制器，并用缓存一致性协议维护共享内存语义。多 socket 机器还会引入 NUMA：不同核心访问不同内存节点的成本不同。

4.1 共享内存的硬件代价

在 C++ 程序里，多个线程似乎可以直接读写同一地址。但硬件上，写一个共享 cache line 需要让其他核心的副本失效或更新。一个原子自增不是普通加法，它需要获得 cache line 独占权，并按内存序约束发布结果。争用越激烈，cache line 在核心之间来回迁移越频繁。

这解释了为什么全局计数器扩展性差。四个线程分别做本地计数，最后合并，通常比四个线程同时 `fetch_add` 同一个原子变量快。前者减少共享写，后者让每次更新都进入一致性协议热路径。

4.2 MESI 的直觉

很多一致性协议都可以用 MESI 直觉理解：Modified 表示本核心有被修改且尚未写回的独占副本，Exclusive 表示本核心有未修改的独占副本，Shared 表示多个核心可以读，Invalid 表示副本无效。当一个核心要写 Shared 行时，需要让其他核心副本失效；当其他核心随后要读，又要重新获取数据。

软件看不到这些状态名，但能看到它们的后果。一个只读共享表可以被多个核心同时缓存，扩展性很好；一个频繁更新的共享 head 指针会让 cache line 在核心之间跳来跳去；一个包含多个线程本地字段的结构体如果没有 padding，也会因为 false sharing 触发同样的迁移。

真实处理器可能使用 MESIF、MOESI 或其他变体，但教学时先抓住一个核心事实：读共享容易扩展，写共享需要取得所有权。Modified 或 Owned 这类状态的具体名称不重要，重要的是写入会触发状态迁移，状态迁移会占用互连和缓存层级资源。高性能并发设计的第一反应，不应该是“换一条更快的原子指令”，而应该是“能不能减少共享写”。

```
read mostly table:
  core0 Shared
  core1 Shared
  core2 Shared

hot atomic counter:
  core0 Modified -> core1 Modified -> core2 Modified -> core0 Modified
```

上面的文本图表达两种完全不同的共享。前者多个核心一起读，硬件可以复制；后者多个核心轮流写，所有权不断迁移。很多扩展性问题就藏在这条差异里。

4.3 一致性和内存模型的分工

缓存一致性解决的是“同一内存位置的多个副本如何保持一致”，内存模型解决的是“不同内存操作之间能被哪些线程以什么顺序观察”。硬件一致性不等于程序自动无数据竞争。没有同步的普通读写仍然可能在 C++ 层面形成未定义行为。

一致性协议通常围绕状态机工作，例如 Modified、Exclusive、Shared、Invalid 这类状态。软件不需要手写这些状态，但必须理解状态迁移的成本：共享读便宜，共享写贵，频繁写同一行最贵。

C++ 内存模型和硬件一致性的边界也要分清。硬件一致性通常保证同一 cache line 或同一地址的副本最终按协议协调，但 C++ 程序如果对普通变量并发读写而没有同步，语言层面就是数据竞争。语言规则先决定程序是否有定义，硬件规则再决定有定义的同步怎样落地。不要用“我的 CPU

有缓存一致性”来为未同步普通变量辩护。

4.4 Store buffer 和可见性

核心执行 store 后，写入可能先进入 store buffer。对本线程来说，后续 load 可能通过 store forwarding 看到自己的写；对其他核心来说，这个写何时可见还受缓存一致性和内存序约束影响。锁释放、release store 和内存屏障会把这种可见性变成可依赖的同步关系。

这就是为什么“我在写 flag 前已经写了 data”在并发中不够。编译器和硬件都可能在允许范围内重排或延迟可见性。正确发布对象需要用 release/acquire、mutex 或其他同步机制建立 happens-before。

Store buffer 还会影响锁的性能。释放锁时，持锁线程在临界区内的写入必须以满足同步语义的方式对后续拿锁线程可见。获取锁时，线程可能需要等待锁所在 cache line 的最新状态。低竞争时这条路径很短；高竞争时，锁变量所在 cache line 会成为所有核心争夺的热点。锁的成本不是一个固定常数，而是竞争、拓扑和写入路径共同决定的结果。

4.5 NUMA 放置

NUMA 机器上，内存离某些核心更近。线程在节点 A 上运行，却频繁访问节点 B 分配的内存，会产生远端访问。Linux 的 first-touch 策略让首次触碰页面的线程影响页面放置，因此并行初始化和线程绑定会影响后续计算性能。

NUMA 优化的基本原则是让计算靠近数据。大数组按块分给线程时，初始化也应按相同块并行完成。跨节点共享队列和全局分配器要谨慎，因为它们可能把远端访问和同步争用叠加在一起。

NUMA 不只出现在多 socket 服务器上。有些桌面和移动平台也有非均匀的缓存、内存或芯片 let 拓扑，只是操作系统暴露方式不同。教材里使用 NUMA 这个词，是为了让读者形成拓扑意识：核心不是一个无差别集合，内存也不是一个无差别池。线程、数据和 I/O 设备之间的位置关系会影响成本。

4.6 First-touch 和并行初始化

Linux 常见 first-touch 策略意味着页面通常分配到第一次写入它的线程所在 NUMA 节点。如果主线程单线程初始化一个大数组，然后工作线程分布在多个节点上处理这个数组，很多工作线程可能访问远端内存。更好的方式是让每个工作线程初始化自己后续要处理的分块。

这条规则在 benchmark 中尤其重要。若初始化方式和计算方式不一致，测到的可能是内存放置错误，而不是算法本身。实验报告必须记录初始化策略、线程绑定和 NUMA 策略。

4.7 互连不是无限带宽

核心、缓存、内存控制器和 I/O 设备之间的互连带宽有限。一个程序即使单线程局部性很好，多线程扩展后也可能撞上内存带宽或互连带宽上限。此时继续增加线程，只会让每个线程分到更少带宽，吞吐不再提高。

互连瓶颈常常表现为“每个线程单独跑都快，一起跑就都慢”。原因可能是多个核心争用同一内存控制器，多个 socket 之间远端访问过多，或者共享写导致 cache line 在互连上来回移动。排查时要把线程数、绑定位置、数据放置和访问模式作为实验矩阵，而不是只改算法代码。

4.8 拓扑感知设计

拓扑感知不是过早优化，而是避免把硬件当作平面资源池。线程池可以按 NUMA 节点分组，内存池可以按节点分配，队列可以按生产者或消费者分片，归约可以先在 socket 内合并再跨 socket 合并。这样的层次化设计和分布式系统中的本地聚合很像：先减少近处流量，再把少量结果发到远处。

4.9 Linux 观察入口

在 Linux 上，拓扑可以从 `lscpu`、`/sys/devices/system/cpu/`、`/sys/devices/system/node/` 和 `numactl--hardware` 开始观察。线程绑定可以用 `taskset`、`numactl--physcpubind` 或程序内亲和性 API。内存策略可以用 `numactl--membind`、`numactl--interleave` 和 `first-`

touch 实验观察。若权限允许，`perf` 可以帮助识别 cache-to-cache 流量，`perfstat` 的 cache、LLC、context-switch 和迁核指标也能提供间接证据。

源码阅读可以从调度拓扑、NUMA 策略和内存分配路径切入。不要试图一次看完整调度器或完整内存管理。更好的问题是：线程迁移时哪些状态会变化，first-touch 影响页面放置的路径在哪里，NUMA 策略怎样被记录和查询，某个 benchmark 的现象能不能从这些路径得到合理解释。

4.10 层次化归约案例

并行归约是理解拓扑的好例子。最差设计是所有线程对同一个原子变量累加。改进设计是每个线程写自己的 partial，最后一个线程合并。进一步的拓扑感知设计是每个核心或每个 worker 先本地累加，每个 NUMA 节点内再合并，最后跨节点合并。这样做减少共享写，也减少远端通信。

这个结构和分布式计算中的树形聚合完全一致。不同只是成本单位变化：单机里远处是另一个 NUMA 节点，分布式里远处是另一台机器。理解这条相似性，可以帮助读者把第二册前后内容串起来。

4.11 一致性流量的三个等级

共享内存程序里的数据共享可以分成三个等级。第一个等级是只读共享，例如配置表、只读索引、常量权重、不可变输入数组。多个核心可以同时缓存这些数据，扩展性通常很好。第二个等级是低频写共享，例如阶段结束时写一次完成标志、每秒汇总一次指标、偶尔替换一份配置快照。它需要同步，但不一定成为瓶颈。第三个等级是高频写共享，例如所有请求更新同一个原子计数器、所有 worker 争用同一个队列 tail、多个线程反复写同一 cache line 上的不同字段。这个等级最危险，往往表现为核心数越多越慢。

优化时要先判断共享等级，而不是先决定使用 mutex、atomic 还是无锁结构。mutex 保护低频复杂状态可能完全合理；atomic 保护高频热点计数器仍然可能很慢；无锁队列如果所有生产者都争用同一个 tail，也逃不开一致性流量。把共享写变成分片写、批量写、线程本地写或按 NUMA 节点写，通常比替换同步原语更根本。

这也是第二册区分“正确性同步”和“扩展性设计”的原因。正确性同步回答的是数据竞争、可见性和不变量；扩展性设计回答的是共享写频率、cache line 迁移和拓扑距离。一个程序可以同步正确但扩展性很差，也可以局部很快但同步语义错误。高质量系统必须同时满足二者。

4.12 目录协议和窥探协议的直觉

一致性协议在小规模系统中可以用广播窥探思想理解：一个核心想写某条 cache line，就向其他核心广播或通过互连通知，让其他副本失效。核心数量增加后，纯广播会变贵，因此很多系统使用目录式信息，记录某条 cache line 可能在哪些核心或节点有副本，再有针对性地发送消息。具体实现非常复杂，但软件层面的直觉很简单：共享范围越大，维护一致性的通信越难便宜。

这条直觉能解释为什么“全局共享结构”在小机器上看不出问题，在大机器上突然变差。四核心机器上，一个全局队列的 cache line 迁移还勉强能承受；多 socket 服务器上，迁移可能跨互连和 NUMA 节点，延迟和带宽成本都更高。工程设计不能只在开发机上看正确性测试通过，还要在目标拓扑上看扩展曲线。

目录和窥探的差别不需要读者背协议细节，但要形成层次化思维。让数据只在一个核心私有，成本最低；让数据在一个 socket 内共享，成本较低；让数据跨 socket 高频写共享，成本高；让数据跨机器共享，成本更高。线程本地 partial、socket 内合并、跨 socket 合并、跨机器 reduce，就是把把这个成本阶梯显式写进算法。

4.13 内存控制器和带宽饱和

NUMA 节点通常连接自己的内存控制器。一个节点上的线程大量访问本地内存，会消耗本地内存控制器带宽；远端访问则需要经过互连到另一个节点的内存控制器。若所有线程都访问一个节点上的内存，即使 CPU 核心分布在多个节点上，带宽也可能集中压在一个控制器上。此时线程数增加，吞吐未必提高，因为瓶颈不是核心数量，而是某个内存节点的带宽。

first-touch 实验的价值就在这里。主线程初始化全部页面，可能让所有页面落在一个 NUMA 节点；worker 分布到多个节点计算时，远端访问集中出现。按 worker 分块初始化，可以让页面分散到

各自节点，使多个内存控制器共同工作。若实验显示 parallel first-touch 版本带宽更高，就说明优化来自页面放置和控制器并行，而不是加法本身改变。

带宽饱和也有反直觉现象。使用更多线程可能让总吞吐略增，但每线程吞吐下降；使用 SMT 可能让内存延迟隐藏更好，也可能只增加争用；把线程平均分到两个节点可能提高总带宽，也可能因为跨节点合并阶段变慢。结论必须来自实验矩阵，不应写成固定经验。

4.14 拓扑感知队列

队列是互连成本的集中点。一个全局 MPMC 队列很容易成为共享写热点：所有生产者争用 tail，所有消费者争用 head 或槽位状态。低线程数下它简单好用，高线程数下它可能把所有 worker 绑在一组 cache line 上。拓扑感知设计通常会拆成多级队列：每个 worker 或每个 NUMA 节点有本地队列，只有负载不均时才访问其他队列。

线程池的工作窃取就是这个思想。worker 优先使用本地 deque，空闲时才偷别人的任务。NUMA 感知线程池还可以先在同节点 worker 中窃取，再跨节点窃取。这样做不是为了复杂而复杂，而是为了让高频路径局部化、低频路径承担远端成本。

有界队列的背压也要考虑拓扑。若所有生产者把任务推到单个队列，队列满时所有生产者都等待同一个条件变量，唤醒和竞争集中；若每个分片有自己的队列，热点分片仍会背压，但无关分片可以继续执行。分片设计会增加任务路由和负载均衡复杂度，因此要先测出全局队列确实是瓶颈，再拆分。

4.15 Linux NUMA 策略和迁移

Linux 提供多种 NUMA 相关策略：默认 first-touch、按节点绑定、交错分配、自动 NUMA balancing 等。自动机制可以在一般场景改善局部性，但它不是性能实验的替代品。自动迁移页面本身有成本，统计窗口和迁移决策也可能与 benchmark 生命周期不匹配。短时间 benchmark 尤其容易受到初始化策略影响。

用户态实验应记录三类信息。第一，线程位置：线程是否绑定，绑定到哪些 CPU，是否发生迁移。第二，页面位置：是否通过 first-touch、mbind 或 interleaved 控制，是否能观测各节点内存使用。第三，访问形状：线程是否主要访问本地分块，是否有跨节点共享写，合并阶段是否集中到一个节点。只记录 `numactl--hardware` 不够，必须把拓扑和程序行为联系起来。

源码阅读可以围绕 `mm/mempolicy.c`、NUMA balancing 相关路径和调度拓扑展开。不要试图一次读懂所有内存管理。合格阅读报告应选择一个问题，例如“为什么主线程初始化会影响页面节点”，然后追踪 VMA 策略、缺页处理和页面分配之间的关系。

4.16 C++ 接口如何表达拓扑

C++ 标准库不直接暴露 NUMA 拓扑，但工程代码可以在接口层表达位置意识。例如线程池可以接收 worker 拓扑描述；内存池可以按节点创建 arena；任务可以带有 preferred node；归约可以返回分层 partial；队列可以按 shard id 提交。接口不需要一开始绑定某个 Linux API，但要给后续拓扑策略留下位置。

下面是一个概念性的任务描述。

Listing 4.1 任务元数据中保留位置偏好

```
1 struct TaskPlacement {
2     std::int32_t preferred_worker = -1;
3     std::int32_t preferred_numa_node = -1;
4 };
5
6 struct ComputeTask {
7     std::size_t begin = 0;
8     std::size_t end = 0;
9     TaskPlacement placement;
10 };
```

这段代码不是要读者立刻实现完整 NUMA runtime，而是提醒接口设计不要把任务看成无位置的函数。对数组块任务，`begin/end` 暗含数据位置；对分布式 shuffle，partition id 暗含目标节点；对线程池，本地队列暗含 worker 位置。位置语义一旦在接口中完全丢失，后面很难再补回来。

4.17 本章落到贯穿项目

Compute Systems Engine 在本章应加入三个设计点。第一，拓扑记录：benchmark 输出可见 CPU 数、用户指定 worker 数、是否启用绑定、是否能读取 NUMA 信息。第二，分层 partial：即使当前只实现每 worker partial，也在设计文档中说明未来可以扩展成每 NUMA 节点 partial。第三，队列分片路线：线程池初版可以用全局队列，但要记录全局队列可能成为共享热点，后续工作窃取章节再拆成本地队列。

本章的验收不是一定在手机或单 socket 机器上测出 NUMA 差异，而是实验设计正确。没有 NUMA 设备时，报告应写明限制：可以验证 false sharing、线程绑定和内存带宽曲线，但不能验证远端内存放置。承认证据边界，比编造一个漂亮结论更专业。

4.18 本章习题

习题与作业

习题一：把一个全局原子计数器、每线程计数器、每 NUMA 节点计数器画成三张数据共享图。分别标出哪些 cache line 被谁写，哪些阶段需要合并，读取全局值的成本在哪里。

习题二：设计一个 NUMA first-touch 实验。要求写清初始化线程、计算线程、绑定策略、输入规模、测量指标和预期现象。若当前设备没有 NUMA，写出哪些部分只能作为计划保留。

习题三：为线程池设计一个两级队列方案：worker 本地队列加全局溢出队列。说明高频路径、低频路径、锁保护对象、可能的窃取顺序和关闭语义。

习题四：阅读 Linux NUMA 策略或调度拓扑的一个窄入口。报告必须从一个用户态实验问题出发，不允许只复制函数名。

实验：在支持 NUMA 的机器上，用 `numactl--hardware` 查看节点。写一个大数组初始化和求和程序，比较单线程初始化、多线程初始化、线程绑定和未绑定情况下的带宽。如果没有 NUMA 机器，也要用 `lscpu` 记录拓扑并解释限制。

第零部分

单机高性能计算：从测量到数据流

第 5 章

可信测量、Roofline 与性能计数器

性能工程必须先建立测量纪律。第一册已经讲过输入规模、环境、编译选项、计时方式、热路径隔离、正确性校验和基础 `perfstat`。本章不重复这些基本规则，而是讨论第二册真正需要的进阶测量：如何把多核程序分解成计算、内存、分支、TLB、同步、调度和 I/O 假设；如何用 Roofline 判断上限；如何用性能计数器和对照实验排除错误解释。

5.1 测量对象

测量前先定义对象。是测单次操作延迟，还是测批量吞吐；是包含内存分配，还是只测热循环；是冷缓存，还是热缓存；是单线程，还是多线程；是平均值，还是尾延迟；是关心总吞吐，还是关心每个 worker 是否均衡。不同对象需要不同方法，不能混用结论。

常见错误包括：把初始化时间算进内核，把编译器优化掉的循环当作结果，把首次缺页当作算法成本，把输出打印放进计时区，把不同 CPU 频率状态下的结果直接比较。

第二册还要特别避免一种错误：只测总时间，不看扩展曲线。多线程程序的核心问题不是“某一次运行快不快”，而是从 1 个线程到多个线程时，吞吐如何变化。一个程序在 2 个线程时加速，在 8 个线程时停滞，在 16 个线程时变慢，这条曲线本身就是证据。它提示瓶颈可能从单核执行转移到了内存带宽、锁竞争、调度、NUMA 或 I/O。

5.2 计时器和统计方法

计时器应使用单调时钟，避免系统时间调整影响。短任务要重复执行足够多次，并防止编译器消除结果。报告结果时不要只给一次运行时间，应至少给出多次运行的最小值、中位数和波动范围。最小值常代表干扰较少的情况，中位数更接近日常情况，尾部异常则提示系统噪声或资源竞争。

预热同样重要。第一次运行可能包含页错误、动态链接、分支预测冷启动、缓存冷启动和 CPU 频率爬升。若目标是冷启动延迟，就应该保留这些成本；若目标是热循环吞吐，就应该把它们排除。测量对象不同，预热策略不同。

多核 benchmark 还要记录线程绑定和运行顺序。若每轮测试先跑单线程再跑多线程，CPU 温度、频率和缓存状态可能系统性变化。更稳妥的做法是随机化版本顺序，保留原始样本，并记录每轮线程数、绑定策略和输入位置。若差异小于系统波动，就不要写强结论。

5.3 Roofline 模型

Roofline 用算术强度连接计算峰值和内存带宽。算术强度是操作数与数据移动字节数的比值。低算术强度程序通常受内存带宽限制，高算术强度程序才可能接近计算峰值。这个模型不是精确预测器，而是帮助判断优化方向。

数组求和每读取一个整数只做一次加法，算术强度低；矩阵乘如果分块得当，每个元素可以被多次复用，算术强度高。若一个内核已经接近内存带宽上限，继续手写更复杂的算术指令可能没有意义；若内核远低于计算上限且数据复用充分，就要检查 SIMD、指令级并行和执行端口。

Roofline 的价值在于先给出“最多可能多快”的粗上限。假设一个内核每处理一个元素读取 8 字节、写入 8 字节，只做少量整数运算，那么它的算术强度很低。若平台可持续内存带宽是 50 GiB/s，那么不管循环写得漂亮，只要每个元素必须从内存读写 16 字节，吞吐上限就被数据移动限制。反过来，若内核通过分块让同一批数据被重复使用，算术强度提高，才有资格讨论 FMA、SIMD 宽度和执行端口。

心智模型

Roofline 不是为了画漂亮图，而是为了阻止错误优化：内存带宽瓶颈不要只改算术指令，计算端口瓶颈不要只改数据结构，同步瓶颈不要只改循环展开。

5.4 从字节数推导瓶颈

Roofline 的关键是先估算字节数。以 `std::int32_t` 数组求和为例，每个元素读取 4 字节，做一次整数加法。如果数组远大于缓存，主要流量来自内存读取。若机器可持续内存带宽是 40 GiB/s，那么理想情况下每秒最多读取约 100 亿个整数。实际还要扣除循环开销、预取效率、TLB 和其他系统干扰。

这个估算能提前判断优化方向。如果实测已经接近带宽上限，就不该期待换一种加法写法获得数量级提升。若实测只有带宽上限的一小部分，就要检查访问是否连续、是否触发 TLB miss、是否有分支、是否被编译器向量化、是否被线程调度干扰。

字节数估算要区分“算法逻辑字节”和“实际流量”。逻辑上，一个原子计数器每次只更新 8 字节；硬件上，它可能让一整条 cache line 在核心之间来回迁移。逻辑上，一个对象只读取一个字段；硬件上，cache line 可能带入多个冷字段。逻辑上，`mmap` 文件像数组；系统上，第一次访问可能触发缺页和 page cache 读取。第二册的实验报告必须说明这些差异，否则字节数估算会过于乐观。

5.5 性能计数器

Linux 上常用 `perfstat` 查看 cycles、instructions、branches、branch-misses、cache-misses、page-faults、context-switches 等指标。更深入时可以用 `perfrecord` 和 `perfreport` 定位热点。不同 CPU 的事件名称不同，实验报告要记录命令和平台。

计数器要组合解释。IPC 低可能因为 cache miss，也可能因为分支失败或同步等待。cache-misses 高不一定说明程序差，流式扫描大数组本来就会不断 miss，但可能充分利用带宽。context-switches 多可能说明线程过多、阻塞频繁或系统有干扰。

进阶测量更重要的是“假设驱动”。如果怀疑前端瓶颈，就比较指令缓存、分支、uops 供给和代码布局；如果怀疑内存带宽，就比较工作集大小、线程数、带宽工具和 LLC miss；如果怀疑 TLB，就改变页大小或访问页数；如果怀疑锁竞争，就看吞吐曲线、等待时间、futex 路径和临界区长度；如果怀疑 NUMA，就改变 first-touch 和绑定策略。不要一次性收集一大堆事件再事后找故事。

5.6 Top-Down 思维

现代性能分析常把 CPU 周期粗分为前端受限、错误预测受限、后端受限和有效退休。不同平台工具名称不同，但思路相同：先判断核心为什么没有持续退休有用指令，再进入更细分类。前端受限时检查取指、译码、代码布局和间接跳转；错误预测受限时检查分支输入分布；后端受限时继续区分执行端口、内存延迟、内存带宽和同步等待；有效退休比例高时，说明核心大部分时间确实在做有用工作，优化要回到算法或工作量本身。

这套思维可以避免把 IPC 低简单等同于“CPU 不行”。同样是 IPC 低，链表随机访问、锁等待、page fault、系统调用阻塞和分支乱跳的解决方向完全不同。报告里应该写“当前证据更支持哪一类受限”，而不是只列一个 IPC 数字。

5.7 同步和调度的测量

多线程程序经常慢在不退休用户态指令的地方。线程可能睡在 futex 上，可能被调度器切走，可能自旋等待 cache line，可能阻塞在 I/O，也可能因为 cgroup 配额用完而被节流。只看 cycles 和 instructions，容易忽略这些等待。

测同步要记录至少三类事实。第一，吞吐随线程数怎样变化；第二，等待在哪里发生，是用户态自旋、内核 futex、条件变量、I/O 还是队列为空；第三，竞争对象是什么，是一把锁、一个原子变量、一条 cache line、一个队列还是一个分片。若能使用 `perfsched`、`perflock`、`perfc2c` 或 eBPF 工具，应记录工具版本和权限；若不能使用，也要用对照实验替代，例如拆分计数器、增加 padding、改变线程绑定和改变队列容量。

5.8 多核扩展曲线

一个高质量性能报告不只给“优化前后快了多少”，还要给扩展曲线。横轴是线程数或 worker 数，纵轴可以是吞吐、耗时、加速比、效率或尾延迟。理想线性加速很少出现，重要的是解释拐点。若 1 到 4 线程接近线性，8 线程开始变慢，可能是共享缓存、内存带宽、NUMA 或同步热点；若从

2 线程开始就没有提升，可能是任务粒度太小、串行部分太大或全局锁太重。

扩展曲线还要配合正确性校验。并行程序在高线程数下更快但偶尔错误，不是优化成功，而是暴露了竞争。每轮性能样本都应关联校验结果，校验失败的样本不能混入性能统计后继续分析速度。

5.9 实验记录模板

一份合格性能记录至少包含：机器型号、核心和 NUMA 拓扑、内存容量、编译器和编译选项、输入规模、数据类型、线程数、绑定策略、运行次数、计时范围、正确性校验、perf 命令和原始输出摘要。缺少这些信息，别人无法复现实验，你自己几周后也很难判断数字是否可靠。

性能结论要写成果因果假设，而不是口号。例如“并行版本在线程数超过 8 后不再加速，perf 显示 LLC miss 和内存带宽接近平台上限，因此瓶颈更像内存带宽，而不是线程创建成本”。这样的结论可以继续被验证或推翻。

本册建议把报告分为五段。第一段写被测问题和正确性 oracle。第二段写机器、系统、编译和绑定环境。第三段写实验矩阵，包括输入规模、线程数、版本、运行轮次和顺序。第四段写原始结果摘要，不只给平均值，也给中位数、最小值和异常。第五段写归因假设和下一步验证。若证据不足，要明确写“当前只能排除某些解释，不能确认最终瓶颈”。

5.10 从问题到实验矩阵

真正的性能实验不是随手跑一个命令，而是把问题拆成可证伪的假设。以 Compute Systems Engine 的归约为例，用户观察到“并行版本没有线性加速”。这句话本身不能直接指导修改，因为可能原因太多：线程创建成本太高，任务粒度太小，单线程 baseline 已经被编译器向量化，输入超过内存带宽上限，partial 发生 false sharing，worker 被调度到同一个物理核心的 SMT sibling，主线程首次触页导致远端内存，或者 benchmark 把输入生成混入了计时区。

实验矩阵要把这些解释逐步分开。第一组只测单线程：baseline、四累加器、可能的 SIMD 版本，输入规模从 L1 可容纳、L2 可容纳、LLC 可容纳一直扩到内存。若小规模下四累加器有效、大规模下无效，说明瓶颈可能从依赖链迁移到内存。第二组固定算法，改变线程数，从 1 到物理核心数，再到逻辑 CPU 数。若到物理核心数后增长变慢，再使用 SMT 变差，说明资源共享或带宽上限值得检查。第三组固定线程数，改变 partial 布局，比较紧凑和对齐。若对齐版本明显改善，说明 false sharing 是关键证据。第四组改变初始化方式，比较主线程初始化和 worker first-touch。若 NUMA 机器上差异显著，说明页面放置参与了结果。

这样的矩阵看起来比“优化前后跑一下”繁琐，但它能防止错误归因。性能工程最怕的是改了十个变量后得到一个数字，然后把提升归功于自己最喜欢的解释。严肃实验宁愿一次只改变一个变量，哪怕多跑几组，也不要同时把线程数、布局、输入、编译选项和绑定策略同时改变。

5.11 正确性 oracle 和防优化

性能实验必须有正确性 oracle。归约的 oracle 可以是顺序 reference；过滤计数的 oracle 可以是简单版本；并发队列的 oracle 不能只看最终数量，还要验证 FIFO 或线性化语义；分布式 shuffle 的 oracle 需要验证每个 key 的最终聚合值和重复请求处理。没有 oracle 的 benchmark 很危险，因为最快的程序可能只是算错了，或者被编译器优化掉了。

防止优化器删除计算也要讲清。常见错误是循环计算了结果但从未使用，优化器发现结果对程序可观察行为没有影响，于是删除整个循环。为了保留计算，可以把结果参与校验、打印摘要、写入 `volatile` sink，或者通过 benchmark 框架的接口告知编译器。更好的方式是把性能样本和 correctness check 绑定：每轮运行都返回结果，结果不对则丢弃样本并报告错误，而不是继续统计耗时。

但防优化不能破坏测量对象。如果把每轮结果都打印到终端，测到的就不是计算时间，而是 I/O 时间。如果把结果写入同一个全局原子，可能引入新的共享写瓶颈。如果把校验放进热循环，校验成本会污染内核。正确做法通常是：热循环只做被测工作，循环外做轻量校验，输出只写汇总，不在计时区打印每个元素。

5.12 预热、冷启动和稳态

预热不是形式主义。第一次运行可能包含动态链接、页面分配、minor fault、缓存冷启动、分支预测器冷启动和 CPU 频率变化。若目标是服务冷启动延迟，这些成本应保留；若目标是稳定吞吐，就应先预热再测。实验报告必须说明测的是冷启动、热循环还是混合流程。

稳态也不是永远稳定。CPU 温度升高可能降频，后台任务可能抢占，容器 CPU 配额可能周期性限流，内核写回线程可能突然工作，透明大页可能在后台整理内存。一个严肃报告应保留多次样本，而不是只给最漂亮的一次。若最小值和中位数差距很大，说明系统噪声或尾部事件值得解释。若样本呈周期性波动，可能要检查 cgroup、频率、热限制或 I/O 写回。

本册建议在报告里至少列出三类值：最小值、中位数和最大值或尾部百分位。最小值常代表干扰较少的理想近似；中位数代表常规情况；尾部代表稳定性风险。高性能系统不只追求平均吞吐，还要理解波动来源。一个吞吐高但尾延迟不可控的队列，可能不适合作为服务请求路径。

5.13 计数器不是事实本身

性能计数器非常有用，但它们不是无条件真理。硬件事件可能受平台支持、内核权限、计数器复用、采样误差和事件定义影响。同一个事件名在不同微架构上含义可能不同；某些高层事件由工具组合推导，不是硬件直接计数；同时请求太多事件时，内核可能 multiplex，导致估计值精度下降。

因此报告中应写出命令、事件列表、是否出现 multiplex、运行时间是否足够长、是否固定 CPU、是否只测当前进程。若工具输出里有权限限制或不支持事件，不能忽略。特别是在手机、容器或受限 Linux 环境中，很多硬件事件无法读取，这时可以退而使用时间、上下文切换、page faults、输入规模对照和源码结构分析，但要明确证据等级降低。

计数器解释也要组合。IPC 高不一定代表程序好，可能只是做了很多无用指令但都能退休；IPC 低不一定代表前端差，可能是内存等待、锁等待或分支错误。cache-misses 高也不一定坏，流式扫描大数组天然会 miss，但如果带宽接近上限，miss 可能是有效数据移动。branch-misses 高才需要结合输入分布、分支数量和耗时变化解释。单个数字不能代替因果链。

5.14 Roofline 的实际计算步骤

Roofline 很容易被画成漂亮图，却没有指导实验。实际使用时先做三步。第一步，估算每个元素的逻辑数据移动和算术操作。例如 `std::int32_t` 求和每个元素读取 4 字节，做一次整数加法，最终写一个总和。第二步，估算实际数据移动的额外成本：cache line 带入未使用数据，写分配带来额外读，原子写导致 cache line 迁移，TLB miss 导致页表访问。第三步，用平台可持续带宽和计算吞吐给出上限，再和实测比较。

例如一个简单拷贝内核读取 4 字节并写入 4 字节，逻辑上每元素 8 字节。但如果写入使用普通 write-allocate 路径，写 miss 可能先把目标 cache line 读入，再修改，实际内存流量高于 8 字节每元素。若使用非临时写入，可能减少写分配，但也改变缓存污染和后续复用。教材不能把字节数估算写成固定公式，要让读者知道实际流量取决于写策略、缓存命中和硬件行为。

Roofline 还要区分单线程和多线程。单线程可能受单核加载端口、预取能力或乱序窗口限制；多线程可能逐步接近内存控制器带宽。若单线程远低于平台总带宽，增加线程可能提升；若多线程已经接近可持续带宽，继续加线程只会增加争用。性能报告应把单线程曲线和多线程曲线放在同一个解释里。

5.15 Amdahl、Gustafson 和扩展曲线

多线程加速经常用 Amdahl 定律解释：如果程序中有不可并行的串行部分，那么线程数再多也会受串行部分限制。这个模型有用，但容易被误用。真实程序的串行部分不是固定不变的：任务切分、合并、内存分配、队列调度、锁竞争和 NUMA 远端访问都可能随线程数变化。换句话说，线程数增加后，程序结构本身也在变。

Gustafson 的视角强调问题规模随资源增加而增长。在很多计算任务中，固定输入规模时加速有限，但输入规模扩大后，并行资源可以处理更多工作。教材要同时讲这两种视角。面试刷题常把复杂度看成固定输入下的函数；系统工程还要看“给更多核心后，能否处理更大批量、更高吞吐或更高并发”。固定规模加速和弱扩展是两个不同实验。

扩展曲线至少有三种画法。强扩展固定总工作量，观察线程数增加后时间下降；弱扩展让每个

线程工作量固定，观察线程数增加后总吞吐是否保持；尾延迟扩展关注并发请求增加后延迟分布怎样变化。Compute Systems Engine 的归约适合强扩展和弱扩展；有界队列和线程池更适合吞吐和尾延迟；分布式 shuffle 则要同时看吞吐、长尾任务和重试成本。

5.16 同步路径的测量陷阱

锁和原子很容易被微基准误导。一个锁在空循环里竞争，测到的是最坏的锁争用形态；在真实业务里，临界区可能保护较大的工作，锁成本占比反而不高。反过来，一个队列在单生产者单消费者测试里表现很好，不代表多生产者多消费者下仍能扩展。同步结构必须按目标竞争模式测。

测 mutex 时应区分无竞争、短竞争和阻塞路径。无竞争 fast path 通常很快；短竞争可能自旋；阻塞路径会进入 futex，涉及内核调度和唤醒。测 atomic 时应区分单线程原子成本和多线程 cache line 争用。测无锁队列时应记录 CAS 失败率或至少记录重试行为，否则只看吞吐无法解释尾延迟。测条件变量时应记录等待谓词、队列长度和唤醒策略，否则不知道线程是在等数据、等空间还是被错误唤醒。

5.17 案例：归约实验报告示范

下面给出一段报告式叙述，展示本册希望读者达到的分析密度。

深入理解

问题：比较顺序归约、四累加器归约、热原子并行归约和线程本地 partial 归约。输入为 256 MiB 的 `std::int32_t` 数组，结果用顺序 reference 校验。机器为某型号 8 物理核心 16 逻辑 CPU，编译器使用固定版本和 -O3。每个版本预热一次，正式运行 10 次，报告中位数和最小值。线程数取 1、2、4、8、16。

预期：顺序 baseline 可能受依赖链和内存流量共同影响；四累加器在小输入下可能改善 IPC，但在大输入下可能接近带宽上限；热原子版本在多线程下会因为共享写扩展性差；线程本地 partial 应显著减少共享写，但 16 逻辑 CPU 可能因 SMT 和带宽争用收益下降。若观察结果与预期不同，下一步分别检查编译器是否已自动向量化、partial 是否 false sharing、初始化是否 first-touch、线程是否迁移。

这段报告没有给固定数字，因为数字属于具体机器；它给的是实验结构和解释路径。一本教材如果只给“某机器快了 3 倍”，读者换设备后很难迁移。更重要的是教读者怎样提出假设、怎样排除错误、怎样承认结论边界。

5.18 观测性进入项目设计

很多学生把观测性当成上线后再加的日志，这在系统工程里太晚。Compute Systems Engine 从第一天就应该记录版本、输入规模、线程数、运行时间、校验结果、队列长度、任务数和错误计数。后续加入线程池时，要记录每个 worker 执行任务数、空闲时间和偷取次数；加入有界队列时，要记录 push 等待次数、pop 空队列次数和最大队列长度；加入分布式模拟时，要记录消息数、重试数、重复请求数、提交延迟和检查点耗时。

观测性也要控制成本。热路径中每个元素都写日志会摧毁性能；每个任务都拿全局锁记录指标会引入新的瓶颈。常见做法是线程本地计数、批量汇总、采样、分级日志和只在实验模式下打开详细指标。观测系统本身也是计算系统，仍然受本册所有原则约束。

5.19 本章落到贯穿项目

本章要求项目拥有一个最小但严肃的 benchmark harness。它至少要支持：选择输入规模，选择版本，选择线程数，重复运行多轮，校验输出，输出机器和编译信息，记录最小值和中位数。后续可以逐步加入 CPU 绑定、NUMA 初始化、perf 命令模板、JSON 输出和图表脚本。不要一开始就做复杂框架，但也不能继续依赖手工改代码和肉眼读终端。

项目里的每个优化提交都应附带实验说明：优化目标是什么，影响哪条路径，正确性如何验证，基线版本是什么，在哪些输入和线程数下有效，在哪些情况下无效。这样做看似增加工作量，但能避免“改完之后不知道为什么快，也不知道什么时候会慢”的维护风险。

5.20 本章习题

习题与作业

习题一：为顺序归约、多累加器归约和线程本地 partial 归约设计一份实验矩阵。要求列出输入规模、线程数、运行次数、预热策略、计时范围、正确性校验和需要记录的计数器。每个变量都要说明用于排除哪一种错误解释。

习题二：写一段报告，解释为什么热原子归约即使使用 relaxed 内存序也可能扩展性很差。报告必须同时使用 C++ 语义、cache line 所有权和测量指标三个层面的语言。

习题三：设计一个队列 benchmark。要求区分 SPSC、MPSC 和 MPMC 三种竞争模式，记录吞吐、队列长度和等待次数。说明为什么只测单生产者单消费者不能证明一个 MPMC 队列适合线程池全局队列。

习题四：阅读当前系统的 `perfstat` 可用事件。记录哪些事件可用，哪些事件因权限或平台不可用。对不可用事件，写出替代实验方案，而不是直接放弃分析。

核心思想

测量不是为了得到一个数字，而是为了排除错误解释，缩小瓶颈范围，并验证优化是否真的改变了瓶颈。

实验：对 Compute Systems Labs 的顺序归约和并行归约运行 `perfstat`。记录时间、cycles、instructions、IPC、cache-misses 和 context-switches。解释并行版本是否真的更快，如果没有，瓶颈可能在哪里。

第 6 章

数据布局、局部性与预取

数据布局决定程序怎样使用缓存、TLB 和内存带宽。很多高性能优化并不是改变算法复杂度，而是把数据摆成硬件更容易消费的形状。理解布局，是理解数组、结构体、队列、图和矩阵内核的共同基础。

6.1 AoS 与 SoA

Array of Structures 把一个对象的字段放在一起，适合经常同时访问完整对象的场景。Structure of Arrays 把同类字段连续存放，适合批量处理某个字段的场景。例如粒子模拟中，如果每轮只更新所有粒子的 x 坐标，SoA 更容易形成连续访问和 SIMD；如果每次处理一个粒子的全部属性，AoS 更直观。

选择布局不能只看代码好不好写，要看热路径读写哪些字段、是否需要向量化、是否有 false sharing、是否跨页跳转。工程中常见折中是 AoSoA，把数据按小块组织，既保留局部对象关系，又适合 SIMD 和缓存块。

Listing 6.1 AoS 与 SoA 的访问差异

```
1 struct Particle {
2     float x;
3     float y;
4     float z;
5 };
6
7 struct ParticleSoA {
8     std::vector<float> x;
9     std::vector<float> y;
10    std::vector<float> z;
11 };
```

如果热路径只更新所有粒子的 x ，SoA 版本会连续读取和写入 x 数组；AoS 版本每次还把 y 和 z 附近的数据带入 cache line。若热路径总是同时使用 x 、 y 、 z ，AoS 的空间局部性反而更自然。

布局选择要从访问矩阵出发，而不是从结构体是否“面向对象”出发。可以把字段列成列，把热路径列成行，在每个格子里标出读、写、只读配置、冷路径调试。若一个热路径只读少数字段，SoA 或冷热分离更有优势；若多个字段总是一起使用，AoS 或 AoSoA 更清晰；若需要 SIMD，字段连续性和对齐会更重要；若需要并发写，线程到字段和槽位的映射必须避免 false sharing。

```
hot path: update position
  reads:  x, y, z, vx, vy, vz
  writes: x, y, z

hot path: compute bounding box
  reads:  x, y, z
  writes: thread local min and max

cold path: debug print
  reads:  name, id, material, history
```

这张访问矩阵会自然推出布局：调试字段不应混在高频数值字段里；只在统计阶段使用的字段

不应污染每帧更新；线程本地输出应分开存放，最后合并。

6.2 预取

硬件预取器擅长识别顺序访问和固定步长访问。连续数组扫描通常能被预取器提前加载，随机指针追踪则很难。软件预取可以在少数场景有效，但如果预取太早会挤出有用缓存，太晚则来不及隐藏延迟。

最好的预取优化通常不是手写预取指令，而是改变访问模式：把随机访问排序，按块处理，压缩节点，减少指针跳转，让硬件预取器能看懂程序。

软件预取只有在地址能提前算出、未来确实会访问、当前有足够独立工作、预取距离合适时才可能有效。指针追踪中，如果下一步地址必须等当前节点加载完成，预取很难发挥作用；批量处理多个游标时，才有机会提前为另一个游标发起预取。预取错误还可能污染缓存，把真正热的数据挤出去。

因此本书把预取看成最后一步，而不是第一步。先让数据连续，先分块，先减少对象分散，先测出内存延迟或带宽瓶颈，再考虑预取指令。很多工程代码的性能来自“让硬件预取器自然工作”，而不是手写大量 prefetch。

6.3 冷热数据分离

对象里不是所有字段都同样热。把热字段和冷字段混在一起，会让缓存带入很少使用的数据。冷热分离把热路径频繁访问的字段放在紧凑结构中，把调试信息、统计字段、名字、配置等冷数据放到旁边结构或间接存储。数据库执行器、游戏引擎和网络服务都常使用这种思路。

冷热分离也有代价：代码复杂度增加，对象关系不如原来直观。是否值得做，要看热路径字段访问比例和缓存压力。设计数据结构时，先写清哪个路径是热路径，哪个字段在热路径必读或必写。

6.4 内存分配和对象布局

小对象频繁分配会带来分配器开销、碎片、TLB 压力和缓存局部性问题。高性能系统常用 arena、对象池、批量分配或结构化生命周期来减少热路径分配。并发分配器还要避免全局锁和跨线程释放导致的远端缓存流量。

常见误区

不要把“使用堆”简单等同于慢。真正的问题是生命周期是否清晰、分配是否在热路径、对象是否分散、是否跨线程释放、是否破坏局部性。

对象池和 arena 的核心收益不是“分配函数少调用几次”这么简单。它们把生命周期相近的对象放在相近内存中，减少元数据开销和碎片，提高遍历局部性，也让释放可以批量发生。并发环境下，还可以按线程或 NUMA 节点维护本地池，减少全局分配器锁和远端释放。

代价同样要写清楚。Arena 适合批量释放，不适合单个对象精确析构；对象池可能保留大量空闲内存；跨线程归还对象可能重新引入同步；自定义分配器如果没有测试，可能比通用分配器更难维护。高质量教材不能只说“用内存池更快”，必须说明适用的生命周期形状。

6.5 布局与并发

多线程程序的数据布局还要考虑写共享。线程本地数据应尽量分开，跨线程只读数据可以共享，跨线程写数据要减少同一 cache line 上的冲突。并行归约、线程池队列和日志缓冲区都常常需要为每个线程准备独立槽位。

6.6 结构体大小和 ABI

结构体字段顺序影响 padding 和总大小。较大的结构体会降低缓存密度，也会增加拷贝成本。公共 ABI 中随意调整字段顺序可能破坏二进制兼容，因此高性能库常把内部热结构和外部稳定接口分开。内部结构按性能布局，外部接口保持稳定语义。

6.7 布局重构流程

数据布局优化应按流程进行。第一步，确认热路径和访问矩阵。第二步，估算每轮实际需要读写的字节数。第三步，检查对象是否连续、是否跨页、是否存在共享写。第四步，设计一个只改变布局、不改变算法语义的对照版本。第五步，用相同输入和相同绑定策略测量时间、cache、TLB 和扩展曲线。第六步，评估代码复杂度是否值得。

很多布局优化失败，是因为同时改了太多东西：换容器、改算法、改线程数、改内存分配、改输入规模。这样即使变快，也不知道原因。教材实验必须强调单变量对照，让读者学会建立因果证据。

6.8 AoSoA：折中不是妥协

AoS 和 SoA 常被讲成二选一，但真实系统经常使用 AoSoA。它把数据按小块分组，每个块内部采用 SoA，块与块之间保持对象批次关系。这样可以让一个块适配 SIMD 宽度或缓存块，同时又不会把整个系统改造成完全分离的字段数组。粒子系统、物理模拟和向量数据库里都能看到类似结构。

Listing 6.2 AoSoA 的基本形态

```
1 constexpr std::int32_t PARTICLE_BLOCK_WIDTH = 8;
2
3 struct ParticleBlock {
4     std::array<float, PARTICLE_BLOCK_WIDTH> x;
5     std::array<float, PARTICLE_BLOCK_WIDTH> y;
6     std::array<float, PARTICLE_BLOCK_WIDTH> z;
7     std::array<float, PARTICLE_BLOCK_WIDTH> vx;
8     std::array<float, PARTICLE_BLOCK_WIDTH> vy;
9     std::array<float, PARTICLE_BLOCK_WIDTH> vz;
10 };
11
12 struct ParticleBlocks {
13     std::vector<ParticleBlock> blocks;
14     std::int32_t size = 0;
15 };
```

这里的块宽度不是魔法数字。它可能来自 SIMD lane 数、cache line 大小、对齐要求、尾部处理成本和代码复杂度。块太小，循环层次变多但复用不足；块太大，尾部浪费和缓存压力上升。AoSoA 的价值是提供一个调节旋钮，让布局能同时服务向量化和对象组织。

AoSoA 也有成本。随机访问第 *i* 个对象需要计算块号和块内偏移；插入删除比 `std::vector<Particle>` 更复杂；调试打印不如 AoS 直观；接口要隐藏布局细节，否则业务代码会到处写块索引。工程中通常把 AoSoA 封装在专用容器或视图后面，让热路径拿到高效访问，普通路径仍保持清晰 API。

6.9 访问矩阵如何落到代码

访问矩阵不是文档装饰，它应该直接影响类型设计。假设日志统计系统的每条记录有时间戳、用户 ID、事件类型、原始文本、解析状态和调试信息。热路径只需要用户 ID 和事件类型做聚合，原始文本只在错误报告时使用。如果把所有字段放在一个大结构里，聚合时每次都把冷字段附近的数据带入缓存。更好的设计是把热字段压成连续数组，把冷字段放在旁边，通过索引关联。

Listing 6.3 冷热分离的记录批次

```
1 struct HotLogBatch {
2     std::vector<std::uint64_t> user_ids;
3     std::vector<std::int32_t> event_types;
4 };
5
```



```

6 struct ColdLogBatch {
7     std::vector<std::string> raw_lines;
8     std::vector<std::int32_t> parse_status;
9 };
10
11 struct LogBatch {
12     HotLogBatch hot;
13     ColdLogBatch cold;
14 };

```

这个设计牺牲了“一个对象包含所有字段”的直观性，换来热路径更少的数据移动。它还让并发更容易：聚合 worker 可以只读 **hot**，错误处理线程再查看 **cold**。若业务经常需要完整记录，冷热分离可能不值得；若绝大多数请求只走热路径，它就很自然。布局设计必须从访问频率和访问组合出发。

6.10 索引优于指针的场景

指针直接表达对象关系，但在大规模数据结构中，索引常常更适合性能和序列化。索引可以是 `std::uint32_t` 或 `std::uint64_t`，指向连续数组中的位置。它比指针更紧凑，容易持久化和跨进程传递，也让对象搬迁和压缩更容易。图的邻接表、编译器 IR、游戏 ECS 和列式存储都大量使用索引。

索引也有代价。每次访问需要多一步数组寻址；删除对象后要处理空洞、代际版本或稳定 ID；越界和悬空索引需要调试检查。若系统需要稳定引用，可以使用“索引加 generation”的句柄，避免旧索引误指向新对象。这个思想和无锁结构里的 ABA 类似：同一个位置再次被复用时，需要版本信息区分“看起来一样”和“语义上还是同一个对象”。

Listing 6.4 索引句柄表达稳定引用

```

1 struct EntityId {
2     std::uint32_t index = 0;
3     std::uint32_t generation = 0;
4 };
5
6 struct EntitySlot {
7     std::uint32_t generation = 0;
8     bool alive = false;
9 };

```

这里使用明确位宽类型，是为了让内存布局和序列化边界清楚。系统教材中不能随手使用 `longlong` 这类平台相关宽度。布局设计和数据类型选择是一件事：类型宽度会影响结构体大小、cache 密度、文件格式和网络协议。

6.11 压缩、编码和计算成本

有时减少内存流量比减少计算更重要。列式数据库和搜索系统常把整数压缩成变长编码、差分编码或位图。这样每个元素读取的字节数减少，但解码需要额外 CPU。是否划算取决于瓶颈：若系统受内存带宽限制，压缩可能提高吞吐；若系统受计算限制，压缩可能变慢。

这个权衡也出现在分布式 shuffle 中。压缩中间数据可以减少网络 and 磁盘 I/O，但会增加 CPU 时间和延迟。单机 cache 层面的“少搬字节，多做一点计算”和分布式层面的“少发网络，多做一点压缩”是同一个思想。教材应让读者从底层内存开始建立这种成本交换模型。

6.12 并发写布局

并发数据布局要先标出写者。字段是否共享，不看它是否在同一个结构体里，而看哪些线程写它、写入频率如何、是否落在同一 cache line 上。一个常见错误是把多个线程的统计字段放进同一个 `Stats` 数组，觉得每个线程写自己的下标就安全。结果每个字段很小，多个线程的字段落在同一条 cache line，形成 false sharing。

更好的设计是把写热字段按写者分开，并且让每个写者独占 cache line 或至少独占写热点。读者汇总时再扫描这些槽位。若读取频率很高，可以维护分层汇总；若读取只是指标展示，可以接受延迟。并布局本质上是在写入吞吐、读取成本和内存占用之间取舍。

Listing 6.5 线程本地统计槽位

```
1 struct alignas(64) WorkerStats {
2     std::uint64_t tasks_executed = 0;
3     std::uint64_t steal_attempts = 0;
4     std::uint64_t empty_polls = 0;
5 };
6
7 std::vector<WorkerStats> stats(static_cast<std::size_t>(worker_count));
```

这类结构适合 worker 高频写自己的统计。它不适合每个请求都读取全局精确值。如果业务需要实时限流，统计槽位还要配合周期性汇总或原子发布。布局优化总是要回到语义。

6.13 迁移旧代码的策略

把成熟代码从 AoS 改成 SoA 或冷热分离，风险很高。直接大改会破坏接口、测试和调试习惯。稳妥策略是分阶段迁移。第一阶段写访问矩阵和性能证据，证明布局确实是瓶颈。第二阶段增加新布局的内部表示，但保留旧接口，通过 adapter 暴露兼容访问。第三阶段把热路径迁到新布局，保留 reference 对照。第四阶段逐步清理旧路径。

迁移时要防止“双写不一致”。如果旧结构和新结构同时存在，更新必须保证二者一致，或者明确一个是权威数据，另一个是缓存。缓存需要失效策略；双写需要事务边界；延迟同步需要版本号。布局重构不是单纯换容器，而是数据所有权迁移。

6.14 本章落到贯穿项目

Compute Systems Engine 可以在本章加入两个实验模块。第一个是记录批处理模块：用 AoS、SoA 和冷热分离三种布局统计 key，比较热字段扫描、错误记录访问和内存占用。第二个是 worker stats 模块：紧凑统计槽位与 `alignas(64)` 槽位对照，观察多线程写入扩展性。这两个模块分别训练“读局部性”和“写共享”两种布局能力。

项目代码不必一口气实现复杂 AoSoA 容器，但要把布局实验和 benchmark harness 接起来。每个布局版本必须使用同一份输入、同一个 reference 结果和同一组线程策略。否则布局差异会和算法差异混在一起。

6.15 本章习题

习题与作业

习题一：为一个日志统计系统写访问矩阵。字段至少包括时间戳、用户 ID、事件类型、原始文本、解析错误、调试标签。热路径至少包括解析、按用户聚合、错误输出和调试查询。根据矩阵给出布局方案。

习题二：把一个指针链表设计改成索引数组设计。说明索引宽度、generation、删除策略、遍历局部性和序列化影响。不要只写“索引更快”，要写清生命周期如何保证。

习题三：设计一个冷热分离迁移计划。要求保留旧接口、建立 reference 测试、说明双写或缓存一致性策略，并给出何时删除旧布局的条件。

习题四：实现或设计 worker stats 的紧凑版和对齐版。报告每个版本的结构体大小、对

齐、cache line 分布、写入线程和汇总成本。

实验：实现 AoS 和 SoA 两个版本的字段求和。比较顺序访问和随机访问。再构造两个线程分别写相邻字段和分离字段的场景，观察 false sharing。

第 7 章

SIMD、指令级并行与向量化

SIMD 让一条指令处理多个数据 lane，是现代 CPU 单核吞吐的重要来源。指令级并行则让多个互不依赖的操作同时在不同执行资源上推进。二者共同决定很多数值循环的上限。

7.1 从标量语义到向量语义

向量化不是把代码表面改成更复杂的形式，而是确认多个元素之间没有依赖，可以被同一条向量指令处理。数组逐元素加法容易向量化，因为每个元素独立；前缀和不容易直接向量化，因为后一个结果依赖前一个结果。

编译器自动向量化需要知道别名关系、对齐、循环边界和数据依赖。使用 `std::span` 表达连续区间有助于接口清晰，但编译器仍可能因为别名保守而放弃向量化。查看优化报告和汇编，比猜测更可靠。

判断循环能否向量化，要先写出每次迭代之间的关系。如果第 `i` 次迭代只读写第 `i` 个位置，通常容易向量化；如果第 `i` 次迭代读取第 `i-1` 次写出的结果，就存在循环携带依赖；如果每次迭代根据数据写入不同位置，就可能有冲突写；如果循环内部调用不可内联函数，编译器可能无法证明副作用边界。向量化首先是语义问题，其次才是指令问题。

7.2 别名和 restrict 思维

两个指针可能指向同一片内存时，编译器必须保守。例如 `a[i]=b[i]+c[i]` 中，如果 `a`、`b`、`c` 可能重叠，编译器要保证重叠情况下结果仍正确，这可能阻碍向量化。C++ 没有标准 `restrict` 关键字，但接口设计可以减少别名不确定性，例如使用不重叠容器、清晰文档和局部拷贝。

手写 SIMD 前，应该先让标量代码容易被编译器理解。清晰的循环边界、连续数据、少分支、少别名，是自动向量化的基本条件。

7.3 多累加器和延迟隐藏

即使不用 SIMD，多个独立累加器也能提高吞吐。单累加器形成长依赖链，每次加法依赖上一次结果；多个累加器把依赖链拆开，让乱序执行有更多可调度操作。SIMD 内核中也常见多个向量累加器，目的相同。

这就是并行归约在单线程内部也有优化空间的原因。先分成多个局部和，再合并，不仅适用于多线程，也适用于单核流水线。

Listing 7.1 多个累加器拆开依赖链

```
1 std::int64_t s0 = 0;
2 std::int64_t s1 = 0;
3 for (std::size_t i = 0; i + 1 < values.size(); i += 2) {
4     s0 += values[i];
5     s1 += values[i + 1];
6 }
7 const std::int64_t sum = s0 + s1;
```

这段代码的意义不是只用两个累加器，而是展示依赖链拆分。真实内核会根据执行端口、寄存器数量和向量宽度选择更多累加器。累加器太少隐藏不了延迟，太多会增加寄存器压力。

7.4 尾部处理和对齐

向量宽度通常不能整除数组长度，因此要处理尾部元素。常见方法是标量尾循环、掩码加载和补齐填充。对齐可以减少某些加载成本，但现代 CPU 对非对齐连续访问已经比较友好；真正要避免的是跨 cache line、跨页和随机访问。

尾部处理是性能代码里常见的正确性漏洞。主循环只处理完整向量宽度，剩余元素必须保证不丢、不重复、不越界。手写 intrinsics 时，尾部可以用标量循环、mask load/store 或提前 padding。选择哪种方式要看 ISA 支持、数据类型、越界读是否合法、输出是否允许覆盖 padding。教材示例宁愿保守清晰，也不能为了展示技巧隐藏边界条件。

常见误区

不要为了让 SIMD 主循环漂亮而忽略尾部。真实库里的很多错误，不发生在主循环，而发生在长度为 0、1、向量宽度减 1、刚好跨页或输入输出重叠的边界。

7.5 SIMD 与线程的关系

SIMD 和多线程不是替代关系。SIMD 提高单核吞吐，多线程使用多个核心。高性能程序通常先让单线程内核足够高效，再扩展到多线程。否则多线程只会复制低效内核，并更快撞上内存带宽。

7.6 数值语义

向量化可能改变浮点运算顺序，从而改变舍入误差。整数加法在不溢出的数学模型下更容易重排，浮点加法不严格满足结合律。编译器的 fast-math 选项会放宽数值语义，可能提高性能，也可能改变边界行为。工程中必须明确是否允许这种变化。

7.7 自动向量化到手写 SIMD

优化顺序通常是先写清晰标量代码，再查看编译器是否自动向量化，再决定是否手写 intrinsics。自动向量化失败时，先看原因：别名不明确、循环边界复杂、函数调用副作用、数据依赖、类型转换、未对齐访问还是成本模型认为不值得。只有当标量语义已经清楚、数据布局稳定、热点足够重要、自动向量化不足时，手写 SIMD 才值得。

手写 SIMD 的维护成本很高。它引入 ISA 分发、不同宽度实现、尾部处理、测试矩阵和可移植性问题。生产库通常会把通用标量 reference、自动向量化版本和手写 ISA 版本分开，并用同一组 correctness tests 验证。第二册讲 SIMD 的目标不是让读者到处写 intrinsics，而是让读者知道什么时候应该追求向量吞吐，什么时候瓶颈根本不在这里。

7.8 向量化和内存带宽

SIMD 提高的是每条指令处理的数据量，但它不能凭空增加内存带宽。对于流式数组求和，如果单线程已经被内存带宽限制，更宽的 SIMD 可能只减少指令数，不显著提高总时间。对于算术强度高、数据能复用的内核，SIMD 才可能接近计算峰值。判断前必须回到 Roofline：这个内核到底缺算力，还是缺数据。

实验：用编译器优化报告观察一个数组加法循环是否向量化。再写单累加器和四累加器求和，比较性能。记录编译选项和 CPU 指令集。

第 8 章

典型计算内核模式

高性能计算里反复出现一些内核模式：map、reduce、scan、stencil、矩阵块计算、直方图、排序、图遍历和队列驱动任务。掌握这些模式，比记住某个库函数更重要。

8.1 Map 与 Reduce

Map 对每个元素独立变换，通常容易并行和向量化。Reduce 把多个元素合成一个结果，存在合并顺序问题。整数求和通常结合律明确，浮点求和则会因为舍入误差导致不同合并顺序产生不同结果。

并行 reduce 的基本形态是每个线程处理一段输入，写入线程本地 partial，然后合并。这个结构减少共享写，也让每个线程顺序扫描自己的数据块。Compute Systems Labs 的 `parallel_sum` 就是这个最小模型。

Reduce 的关键不是“把加法拆给多个线程”这么简单，而是确定合并操作是否满足结合律、是否允许重排、partial 放在哪里、合并树长什么样。整数求和如果可能溢出，重排也可能改变 C++ 语义；浮点求和即使不溢出，也会改变舍入；字符串拼接满足语义结合时，内存分配可能成为主要成本。因此每个 reduce 都要同时写正确性模型和成本模型。

8.2 Map 的融合

多个 map 连在一起时，如果每一步都写出中间数组，会增加内存流量。融合把多个逐元素变换合在一次循环里，减少中间读写。融合不是总是好事：融合后循环体变大，寄存器压力可能增加，分支也可能更复杂。是否融合，要看减少的内存流量是否超过新增的执行压力。

8.3 Stencil 与邻域计算

Stencil 计算每个输出元素时读取邻域元素，例如图像卷积、有限差分 and 网格模拟。它的关键是复用邻域数据，避免重复从内存加载。分块、滑动窗口和边界处理是主要问题。

Stencil 的边界处理会影响代码形状。可以把内部区域和边界区域分开，让内部热循环没有复杂分支；也可以用 padding 或 ghost cell 让边界访问统一。前者代码路径多但热循环干净，后者内存多一点但逻辑规律。并行 stencil 还要处理块之间的 halo 数据，单机上表现为相邻块共享边界，分布式上表现为节点之间交换 halo 消息。

8.4 矩阵和块算法

矩阵计算展示了数据复用的重要性。朴素矩阵乘的数学公式很简单，但访问顺序会决定是否复用缓存。分块算法把矩阵切成能放入缓存的小块，让一个块内的数据被多次使用。这个思想不仅用于矩阵，也用于排序、join、图处理和文件扫描。

块大小不是越大越好。太小会增加循环和调度开销，太大又放不进缓存。实际工程中常用 benchmark 或自动调优选择块大小，并针对不同 CPU 做参数化。

8.5 直方图和冲突更新

直方图看似简单：读一个元素，更新一个桶。但多个线程可能更新同一个桶，导致锁竞争或原子争用。常见优化是线程本地直方图，最后合并。这个模式体现了一个原则：把高频共享写变成低频合并。

直方图还会遇到数据倾斜。如果 key 分布均匀，线程本地桶很好；如果大量输入集中到少数桶，全局原子会退化，分片后合并也可能在少数桶上集中。工程实现常按线程、NUMA 节点或缓存块建立局部桶，并在合并阶段使用连续扫描。若桶数量很大，本地直方图会占用大量内存，这时要在内存占用和冲突减少之间折中。

8.6 图遍历

图遍历通常局部性差、分支多、负载不均。压缩邻接表、按层处理、前沿队列和方向优化，都是为了改善访问模式和任务分配。图算法提醒我们，不是所有问题都能靠 SIMD 和连续数组轻松解决。

图遍历的性能问题往往同时包含三件事：邻接表访问随机，顶点度数不均，前沿大小变化剧烈。BFS 在某些层只有少量顶点，线程不够忙；在某些层前沿巨大，访问又非常分散。方向优化 BFS 会在 top-down 和 bottom-up 之间切换，目的就是根据前沿大小改变访问模式。这个例子说明，高性能算法不是固定套一个并行模板，而是随数据形状选择执行策略。

8.7 排序和分区

排序是数据系统的核心内核。快速排序、归并排序、基数排序和样本排序在不同场景下表现不同。并行排序的难点不只是比较次数，还包括分区均衡、临时空间、cache 行为和跨线程合并。分布式排序还会引入 shuffle 和热点分区。

8.8 内核选择清单

遇到一个新计算内核，可以按清单分析。第一，输入输出是否连续，是否能按块切分。第二，每个元素工作量是否接近，是否存在数据倾斜。第三，是否有跨元素依赖，依赖能不能分层处理。第四，写入位置是否唯一，是否会冲突。第五，是否能把全局共享写改成本地写加合并。第六，算术强度大概是多少，瓶颈更像计算还是内存。第七，单机算法扩展到分布式时，哪一步会变成 shuffle 或远程通信。

这份清单会在后面的并行算法和分布式计算里反复出现。它让读者从“我知道几个算法名字”转向“我能根据数据和依赖形状选择执行方式”。

核心思想

分析计算内核时先问四个问题：每个元素读写多少字节，是否存在跨元素依赖，写入是否冲突，工作量是否均衡。

实验：在 Compute Systems Labs 中扩展一个线程本地直方图。先写全局 mutex 版本，再写线程本地版本，比较扩展性，并解释差异来自锁竞争还是 cache line 迁移。

第零部分

多线程、同步与内存模型

第 9 章

线程、调度与亲和性

线程是操作系统调度的执行实体。多线程程序能利用多核心，但也引入调度、同步、内存可见性和资源竞争。理解线程，必须同时看用户态代码和内核调度。

9.1 线程不是核心

创建八个线程不等于拥有八个物理核心。硬件可能有 SMT，一个物理核心暴露两个逻辑 CPU；系统里还运行着其他进程；线程可能被迁移到不同核心。线程迁移会破坏缓存热度，过多线程会增加上下文切换。

CPU-bound 程序通常不应远多于核心数，I/O-bound 程序可以有更多并发等待。判断依据不是经验数字，而是运行时是否在计算、等待锁、等待 I/O 或被调度器抢占。

9.2 用户线程和内核线程

C++ 的 `std::thread` 通常映射到操作系统线程。操作系统调度的是内核可见的执行实体，它拥有寄存器上下文、栈、调度状态和优先级。用户态协程则不同，协程切换通常不进入内核，它把等待点显式暴露给运行时。协程适合组织大量 I/O 等待，不会自动让 CPU 密集代码并行。

这一区分很重要。把 CPU 密集任务放进单线程协程事件循环，不会使用多个核心；把大量 I/O 等待都变成内核线程，可能造成过多栈内存、调度成本和上下文切换。运行时设计要把计算并行和等待并发分开。

9.3 调度和上下文切换

上下文切换需要保存和恢复执行状态，可能破坏 cache 和 TLB 局部性。频繁阻塞、线程过多、锁竞争和系统调用都可能增加切换。性能计数器里的 context-switches 和 migrations 可以帮助定位这类问题。

Linux CFS 调度器大体追求公平，它会根据虚拟运行时间选择任务。实时调度策略和普通调度策略不同，容器和 cgroup 也会改变 CPU 配额。benchmark 时若机器上有其他负载，或者进程受 cgroup 限制，结果会明显波动。严谨实验应记录负载、CPU governor、容器限制和是否独占机器。

调度器不是只在“线程很多”时才重要。即使线程数等于核心数，线程也可能因为缺页、锁等待、I/O、信号、中断、抢占和 cgroup 配额离开 CPU。对于低延迟系统，一次迁核可能让热缓存失效；对于吞吐系统，频繁上下文切换会减少有效计算时间。性能报告如果只写线程数，不写线程是否真正持续运行在 CPU 上，就缺少关键证据。

Linux 调度源码阅读不建议一开始追完整策略。更好的入口是从一个用户态现象出发：为什么 `perfstat` 里 context-switches 增加，为什么线程会 migration，为什么绑定 CPU 后波动变小，为什么 cgroup 限额下吞吐呈周期性抖动。围绕这些问题，再去看调度实体、runqueue、唤醒路径和上下文切换路径，读出的内容才有边界。

9.4 阻塞、睡眠和自旋

线程等待锁时，可以自旋，也可以睡眠。自旋避免进入内核，适合等待时间极短的场景；睡眠释放 CPU，适合等待时间较长的场景。很多锁实现会先短暂自旋，再进入 `futex` 等内核等待路径。这样低竞争时快，高竞争时不至于烧满 CPU。

选择自旋还是阻塞，本质是在 CPU 时间和唤醒延迟之间取舍。如果临界区很短且核心充足，自旋可能好；如果锁持有者被抢占或等待 I/O，自旋会浪费大量周期。不要在没有测量的情况下把自旋当成高性能。

`futex` 是理解 Linux 用户态锁的重要入口。常见 `mutex` 在无竞争时尽量只走用户态原子操作；竞争严重时，线程通过 `futex` 进入内核睡眠，锁释放时再唤醒等待者。这样做把低竞争 fast path 和

高竞争 blocking path 分开。读锁性能时，应区分用户态自旋成本、futex 睡眠成本、唤醒成本和被唤醒后重新竞争锁的成本。

9.5 线程亲和性

线程亲和性让线程倾向或固定在某些 CPU 上运行。它可以改善缓存局部性和 NUMA 放置，也可能降低调度器自由度。使用亲和性前要理解拓扑：物理核心、SMT 线程、NUMA 节点和共享缓存层级。

亲和性实验要有对照。只绑定线程不绑定内存，可能仍然访问远端页面；只绑定内存不绑定线程，线程迁移后也可能变成远端访问。合理实验至少比较未绑定、只绑线程、只绑内存、线程和内存都按节点绑定几种情况。若平台没有 NUMA，也应记录物理核心和 SMT 拓扑，避免把两个重负载线程绑定到同一个物理核心的两个逻辑 CPU 上后误判算法慢。

9.6 线程生命周期

频繁创建销毁线程成本高，也让任务调度不可控。线程池把线程生命周期和任务生命周期分离，适合大量短任务。但线程池不是万能的，如果任务太小，调度成本会超过计算成本；如果任务阻塞，线程池可能被耗尽。

9.7 线程局部存储

线程局部存储让每个线程拥有独立对象，适合缓存临时缓冲、统计计数和避免共享写。它可以减少锁竞争，但也可能增加内存占用，并让跨线程汇总更复杂。线程池里使用线程局部对象时，还要注意任务迁移：同一个逻辑请求的不同阶段可能被不同 worker 执行，不能把请求状态误放到线程局部。

9.8 线程池里的调度语义

线程池把固定 worker 暴露成任务执行资源，但它不会自动解决调度语义。CPU 密集任务应该限制并发度，避免超过核心数太多；阻塞 I/O 任务如果占住 worker，可能让计算任务饥饿；任务内部再提交子任务并等待，可能造成线程池死锁。设计线程池时，要明确任务是否允许阻塞、是否允许嵌套提交、是否支持 work helping、是否需要独立 I/O 池。

线程池还会改变线程局部数据的含义。线程局部缓冲属于 worker，不属于请求。请求如果跨多个任务执行，就不能假设后续阶段还能访问同一个线程局部对象。运行时应把“worker 本地缓存”和“请求上下文”分成两个概念。

9.9 从任务到线程的映射

并发程序里最容易混淆的是任务和线程。任务是逻辑工作单元，例如处理一段数组、解析一个文件块、执行一个 reduce 分区、发送一个网络请求。线程是操作系统调度实体，它拥有栈和寄存器上下文，会被放入 runqueue，由调度器分配 CPU 时间。一个任务可以在线程上执行，一个线程可以执行很多任务；二者不是同一概念。

Compute Systems Engine 最初的 `parallel_sum` 每次调用都创建一组线程，任务和线程几乎一一对应。这种写法适合教学，因为边界清楚；但它不适合大量短任务，因为线程创建、栈分配、调度注册和 join 都有成本。线程池把线程生命周期延长，让任务通过队列进入 worker。这样任务变轻了，但又引入队列、等待、关闭和异常传播问题。每前进一步，性能和语义都会同时变化。

设计运行时，要先回答三个问题。第一，任务是否 CPU 密集。如果是，worker 数通常不应远超物理核心，除非需要隐藏内存或 I/O 等待。第二，任务是否可能阻塞。如果任务会等待磁盘、网络、锁或 future，把它放进 CPU 池可能耗尽 worker。第三，任务之间是否有依赖。如果一个任务提交子任务后等待，而线程池没有 work helping 或足够 worker，可能死锁。线程池不是“并发万能容器”，它只是执行资源的组织方式。

9.10 线程创建成本和任务粒度

任务粒度是并行运行时最基本的参数。粒度太大，负载不均衡，一个慢任务拖住整体；粒度太小，队列操作、调度、同步和计数器开销超过有效计算。归约任务如果每个 worker 处理上千万元素，线程创建成本相对很小；如果每个任务只处理几十个元素，线程池也可能被调度开销吞掉。

选择粒度不能只靠固定常数。不同机器核心数不同，缓存大小不同，内存带宽不同，任务内核成本也不同。一个合理策略是先用 coarse chunk 保证每个任务有足够计算量，再根据负载不均衡逐步拆小。对均匀数组求和，按连续大块切分足够；对图遍历或 key 倾斜聚合，任务成本不均匀，可能需要动态调度、工作窃取或分片负载估计。

实验上要画任务粒度曲线。固定总输入和线程数，改变 chunk size，记录吞吐、任务数、队列等待、worker 空闲时间和尾部完成时间。曲线通常会出现一个中间区域：粒度太小，调度成本高；粒度太大，负载不均衡；中间区域才是合理范围。这个实验比背“任务数等于线程数几倍”更可靠。

9.11 阻塞任务和线程池耗尽

考虑一个线程池有四个 worker。每个任务启动后提交一个子任务，然后等待子任务完成。如果四个 worker 都在执行父任务并等待子任务，而子任务还在队列里没有 worker 执行，线程池就死锁了。这个问题不是 mutex 死锁，却同样来自等待关系形成环。

解决方向有几种。第一，禁止 worker 内同步等待同一线程池的子任务，把依赖交给任务图调度。第二，支持 work helping：worker 等待时不睡眠，而是继续执行队列里的其他任务。第三，分离 CPU 池和阻塞池，让阻塞 I/O 不占满计算 worker。第四，增加结构化并发语义，让任务的生命周期和父子关系明确。选择哪种方式取决于运行时目标，但必须在接口文档中说明，不能让调用者猜。

本章先建立问题，后续任务运行时章节会实现更完整的任务图。这里要记住：线程池接口不只是 submit，还包括等待、取消、关闭、异常和嵌套提交语义。一个只会把函数塞进队列的线程池，很容易在真实系统中出现停不下来、等不到、吞异常或死锁的问题。

9.12 亲和性和拓扑记录

亲和性优化的前提是拓扑记录。报告里至少要写清逻辑 CPU 数、物理核心数、SMT 关系、NUMA 节点、共享 L1/L2/L3 的范围和内存节点。Linux 上可以用 `lscpu`、`numactl--hardware`、`/sys/devices/system/cpu/` 下的拓扑文件和工具输出记录。没有这些信息，绑定 CPU 的结论很难解释。

绑定策略要和工作负载匹配。CPU 密集向量计算通常先使用每个物理核心一个线程，再测试 SMT；内存延迟敏感但单线程经常等待的任务可能从 SMT 获益；共享数据强的任务可能受益于放在共享 LLC 内；NUMA 分片任务应让线程和内存位于同一节点；跨分片通信强的任务要考虑互连成本。亲和性不是为了把线程“钉死”就结束，而是为了把执行位置、数据位置和共享层级对齐。

亲和性也可能伤害性能。过度绑定会减少调度器处理系统噪声的自由度；把两个重负载线程绑到同一物理核心的 SMT sibling 会互抢执行资源；把线程绑到一个 NUMA 节点但数据在另一个节点，会稳定地产生远端访问；在手机或容器环境里，系统可能限制可用 CPU，强行绑定到不可用 CPU 会失败。实验报告必须记录绑定是否成功。

9.13 上下文切换的真实代价

上下文切换不是一个固定纳秒数。直接成本包括保存和恢复寄存器状态、切换内核调度结构和更新运行队列；间接成本包括 cache 热度下降、TLB 上下文影响、分支预测状态变化和被迁移到不同核心后的数据重新加载。若切换发生在锁持有者身上，其他线程可能自旋或睡眠等待；若切换发生在低延迟请求路径上，尾延迟可能被放大。

测上下文切换要区分自愿和非自愿。自愿切换通常来自线程主动阻塞，例如等待条件变量、I/O 或 futex；非自愿切换通常来自时间片耗尽、抢占或更高优先级任务。两者解决方向不同。自愿切换多，可能要减少阻塞、批处理或改变等待策略；非自愿切换多，可能线程数过多、CPU 竞争严重或系统负载干扰。

在 `perfstat` 或系统工具中看到 context-switches 增加时，不要立刻下结论。一个 I/O 服务有上下文切换可能是正常的；一个 CPU-bound 热循环频繁切换就值得怀疑。要结合 CPU 利用率、线程数、锁等待、runqueue 长度和任务类型分析。

9.14 Linux 调度源码阅读路线

Linux 调度器很复杂，不适合一开始从所有策略全局阅读。本册建议按现象驱动。若你想理解普通线程为什么被调度，可以从核心调度入口、调度类、运行队列和上下文切换路径读起；若想理解唤醒延迟，可以沿 `futex` 或条件变量唤醒路径看到任务如何重新进入 `runqueue`；若想理解迁移，可以阅读负载均衡和 CPU affinity 相关路径；若想理解 `cgroup` 限额，可以阅读 CPU controller 对可运行时间的限制。

报告要写具体问题。例如“在 16 个线程测试中，migrations 明显增加，绑定 CPU 后波动下降”，然后阅读调度迁移和亲和性相关代码。不要写“Linux 使用 CFS 调度器，所以性能如何”这种大而空的句子。源码阅读的价值在于把用户态现象和内核数据结构连接起来：`task`、`runqueue`、调度类、唤醒、抢占、迁移和上下文切换。

9.15 线程局部缓存和内存生命周期

线程局部缓存可以减少分配和共享写，但它改变了对对象生命周期。线程池中的 `worker` 会长期存在，线程局部对象也会长期存在。如果线程局部缓冲按最大请求扩容，处理一次大请求后可能长期占用大量内存。若任务之间复用线程局部状态，还要保证不会泄漏上一个请求的数据。高性能系统里，缓存和生命周期总是绑在一起。

线程局部统计同样有读取语义问题。每个 `worker` 写自己的计数很快，但读取总数需要遍历所有 `worker` 状态。若 `worker` 正在写，读者是否需要精确值？如果只用于指标展示，可以接受近似或延迟汇总；如果用于协议决策，例如限流额度，就需要更强同步。把所有统计都改成线程局部之前，必须说明读取方的语义需求。

9.16 案例：从每次创建线程到固定线程池

下面用归约说明线程生命周期的演化。`baseline` 是每次调用 `parallel_sum` 都创建 `worker_count` 个线程。它适合少量大任务，但不适合频繁短任务。改进方向是固定线程池：程序启动时创建 `worker`，归约请求被拆成多个 `chunk`，提交到队列，`worker` 执行后写 `partial`，主线程等待所有 `chunk` 完成。

这个改进引入了新问题。队列需要同步，提交任务需要分配或复用任务对象，等待所有 `chunk` 完成需要计数器或 `latch`，异常需要传回调用者，线程池关闭时不能丢任务。如果允许多个归约请求同时进入线程池，`partial` 的所有权和生命周期要隔离。一个高质量线程池章节不能只给一段 `while(true)` `worker` 循环，还要讲清这些语义。

本章只要求读者理解演化方向。后续运行时章节会把线程池拆成接口、队列、`worker`、任务状态、关闭协议和指标。这里先建立判断：当任务足够大且调用次数少，每次创建线程可以接受；当任务短且频繁，线程池值得引入；当任务之间有依赖，普通 FIFO 线程池不够，需要任务图或 `work stealing`。

9.17 本章落到贯穿项目

Compute Systems Engine 在本章要增加运行环境记录和线程策略实验。第一，程序启动时记录硬件拓扑和可用线程数，至少输出逻辑 CPU 数和用户指定 `worker` 数。第二，归约 `benchmark` 要支持从 1 到多倍逻辑 CPU 的线程扫描。第三，实验报告要记录 `context-switches` 和 `migrations`。第四，设计线程池前先保留每次创建线程版本作为对照，避免引入线程池后不知道收益来自哪里。

后续项目可以逐步加入亲和性选项，例如不绑定、按物理核心绑定、按 NUMA 节点绑定。每种绑定都必须能失败并报告原因，不能在不支持的平台上默默假装成功。手机或 Termux 环境尤其要谨慎，因为权限、CPU 拓扑和系统调度策略可能和服务器不同。

9.18 本章习题

习题与作业

习题一：给当前 `parallel_sum` 写一份线程生命周期分析。指出每次调用创建多少线程，线程做多少实际工作，`join` 等待在哪里发生，哪些成本会在输入很小时占主导。

习题二：设计一个任务粒度实验。固定总输入大小和 worker 数，改变 chunk size。报告要包含任务数、总时间、每个 worker 任务数和尾部完成时间。解释粒度太小和太大分别会出现什么现象。

习题三：构造线程池死锁案例：worker 任务提交子任务并等待。画出等待图，说明为什么增加队列容量不能解决这个死锁。再提出两种运行时语义上的修复方案。

习题四：记录本机或手机上的 CPU 拓扑。写清逻辑 CPU、物理核心或可见核心、是否能看到 SMT、是否有 NUMA 信息、是否允许设置 affinity。不要把无法获取的信息编造成确定结论。

核心思想

线程设计的核心不是“开几个线程”，而是决定哪些状态线程本地，哪些状态共享，哪些共享状态需要同步，哪些等待应该阻塞或异步化。

实验：把并行归约的 worker 数从 1 增加到逻辑 CPU 数的两倍。记录时间、context-switches 和 migrations。解释从哪个点开始扩展性变差。

第 10 章

锁、条件变量与信号量

锁的作用是保护共享状态，让临界区在同一时间只被一个线程执行。锁不是性能敌人，也不是正确性的全部。正确使用锁，需要明确保护对象、锁顺序、临界区长度和等待条件。

10.1 互斥锁保护什么

每把锁都应该有清晰的保护范围。一个队列锁保护 head、tail、size 和 buffer 状态；一个计数器锁保护 value；一个对象锁保护对象不变量。锁如果没有对应的不变量，就容易出现“看起来加锁了但仍然错”的情况。

临界区应尽量短，但不能为了短而破坏不变量。把状态检查放在锁外，可能引入竞态；把耗时 I/O 放在锁内，会扩大阻塞范围。工程上要在正确性和粒度之间找到边界。

Listing 10.1 锁保护队列不变量

```
1 class QueueState {
2 public:
3     void push(std::int32_t value) {
4         std::lock_guard<std::mutex> lock(this->mutex_);
5         this->values_.push_back(value);
6     }
7
8 private:
9     std::mutex mutex_;
10    std::vector<std::int32_t> values_;
11 };
```

这段代码的关键不是 `push_back` 这一行，而是锁和不变量的关系：`values_` 的内部结构不能被多个线程同时修改。如果未来再加入 `size_`、`head_` 或统计字段，它们也必须在同一把锁保护下保持一致。

写锁代码时，应先写“锁保护的不变量”，再写代码。比如有界队列的不变量可以是：`0<=size<=capacity`，`head` 和 `tail` 总是落在环形数组范围内，`size==0` 表示不能 pop，`size==capacity` 表示不能 push。互斥锁保护的是这些关系，不只是保护某一个变量。若 `size` 在锁内改，`head` 在锁外改，这个不变量就被拆碎了。

10.2 锁粒度

粗粒度锁容易证明正确，但并发度低。细粒度锁提高并发度，却增加锁顺序、死锁和状态拆分复杂度。分片锁是一种常见折中：按 key、队列分段或线程组拆成多把锁，让无关操作并行，热点操作仍受保护。

锁粒度应从访问模式出发。如果所有请求都访问同一个热点 key，分片没有帮助；如果请求天然分散，分片锁可以显著减少竞争。锁优化前要先测量等待时间和竞争比例。

10.3 条件变量

条件变量用于等待某个状态变真。它必须和互斥锁配合，并在循环里检查谓词。原因是线程可能被虚假唤醒，也可能在被唤醒后发现条件已经被其他线程消费。正确模式是“持锁检查条件，不满足则等待，醒来继续检查”。

Listing 10.2 条件变量等待谓词

```

1 std::unique_lock<std::mutex> lock(this->mutex_);
2 this->not_empty_.wait(lock, [this]() {
3     return !this->values_.empty();
4 });
5 const std::int32_t value = this->values_.front();

```

这里谓词属于受锁保护的状态。通知本身不携带数据，数据在共享状态里。若不用循环检查谓词，就可能在虚假唤醒或竞争消费后读取空队列。

条件变量最重要的句子是：等待的是谓词，不是通知。通知可能在等待前发生，也可能唤醒多个线程，也可能出现虚假唤醒。只要谓词在锁保护下检查，程序就正确；只依赖“我收到过一次 notify”，程序就脆弱。生产者消费者队列里，`not_empty` 的谓词是队列非空，`not_full` 的谓词是队列未滿。两个谓词对应同一组受锁保护状态。

10.4 信号量和资源计数

信号量表示可用资源数量。它适合限制并发度、实现 bounded buffer 或控制连接池。信号量和互斥锁不同：互斥锁保护状态的一致性，信号量表达资源额度。混用时要小心锁顺序，避免死锁。

10.5 死锁和锁顺序

死锁通常需要互斥、持有并等待、不可抢占和循环等待四个条件。破坏循环等待的常见方法是规定全局锁顺序。复杂系统里，锁顺序文档和锁级别检查很重要，因为死锁往往在低概率路径中出现。

10.6 优先级反转和 convoy

优先级反转发生在低优先级线程持有锁，高优先级线程等待它，而中等优先级线程持续占用 CPU，导致高优先级线程无法前进。锁 convoy 则是多个线程排队等待同一锁，锁释放后唤醒和抢占成本让系统吞吐下降。这些问题说明锁不仅是用户态对象，也和调度器行为相互作用。

工程系统要避免在锁内做不可控工作，例如磁盘 I/O、网络调用、复杂回调和用户插件。锁内执行时间越不可预测，越容易出现尾延迟问题。

10.7 锁的性能路径

mutex 通常有三条路径。第一条是无竞争 fast path，用户态原子操作成功，成本较低。第二条是短暂竞争路径，线程可能自旋几轮，等待锁持有者很快释放。第三条是阻塞路径，线程进入内核等待，涉及 futex、调度和唤醒。性能分析要判断当前主要走哪条路径。一个低竞争 mutex 没必要改成复杂无锁结构；一个长期进入阻塞路径的热点锁，则需要拆分状态、缩短临界区或改变算法。

临界区长度不只由代码行数决定。锁内分配内存、写日志、调用用户回调、访问远端 NUMA 内存、发生 page fault、触发异常路径，都会让持锁时间变长。锁优化报告应列出锁内可能的慢操作，而不只是说“这个锁很粗”。

10.8 读写锁和 RCU 直觉

读多写少场景中，读写锁允许多个读者并发，但写者需要独占。它适合读临界区不太长、写入不太频繁的结构。若读者很多且读路径必须极轻，RCU 的思想更进一步：读者在不阻塞写者的情况下读取旧版本，写者发布新版本后，等所有旧读者离开再回收旧对象。

RCU 的难点不在“读很快”这个口号，而在对象生命周期。旧对象什么时候没人再读，什么时候可以释放，如何处理写者之间的同步，都是必须证明的问题。后面讲无锁内存回收时，会看到相同主题。

10.9 有界队列从不变量开始

Compute Systems Engine 的第一个同步结构应是有界队列，因为它把互斥、条件变量、背压和关闭语义放在同一个小系统里。有界队列不是一个简单容器，它表达了生产者和消费者之间的容量契约：队列未满时生产者可以放入，队列非空时消费者可以取出，队列满时生产者必须等待或失败，队列关闭后等待者必须能醒来并退出。

先写不变量，再写代码。一个环形有界队列至少有这些不变量：容量大于零；`head` 和 `tail` 总在 $[0, \text{capacity})$ 范围内；`size` 总在 $[0, \text{capacity}]$ 范围内；`size==0` 时没有元素可取；`size==capacity` 时没有空槽可写；`head` 指向下一个可读槽；`tail` 指向下一个可写槽。互斥锁保护的是这组关系，而不是某一个整数。

只要写代码时先写不变量，很多错误会提前暴露。例如 `push` 修改 `tail` 但忘了增加 `size`，`try_pop` 读取元素后忘了通知 `not_full`，`size` 用无符号类型导致下溢，析构时仍有线程阻塞，关闭后生产者继续写入，这些都不是语法问题，而是不变量和状态转换没有定义清楚。

10.10 从 try 到 blocking 的接口分层

队列接口不应只有一种操作。常见接口至少分成三类：非阻塞尝试、阻塞等待和关闭。非阻塞 `try_push` 或 `try_pop` 立即返回，适合事件循环或自定义调度；阻塞 `push` 或 `pop` 等待谓词变真，适合普通生产者消费者；关闭 `close` 唤醒等待者，告诉它们系统不再接受新任务或不再产生新元素。

如果队列没有关闭语义，线程池很难优雅退出。worker 可能永远阻塞在空队列上，主线程析构线程池时只能强行终止进程或依赖外部哨兵值。哨兵值在泛型队列中也不可靠，因为每种元素类型未必都有安全哨兵。正确做法是把关闭作为队列状态的一部分，受同一把锁保护，并在关闭时通知所有等待者。

Listing 10.3 有关闭语义的有界队列接口草图

```
1 class BoundedQueue {
2 public:
3     explicit BoundedQueue(std::int32_t capacity);
4
5     bool try_push(std::int32_t value);
6     bool push(std::int32_t value);
7     std::optional<std::int32_t> pop();
8     void close();
9
10 private:
11     std::vector<std::int32_t> buffer_;
12     std::int32_t head_ = 0;
13     std::int32_t tail_ = 0;
14     std::int32_t size_ = 0;
15     bool closed_ = false;
16     std::mutex mutex_;
17     std::condition_variable not_empty_;
18     std::condition_variable not_full_;
19 };
```

这里 `push` 返回 `bool`，是为了表达关闭后不能再写入；`pop` 返回 `optional`，是为了表达关闭且队列为空时消费者应退出。这个接口比单纯抛异常更适合作为线程池内部队列，因为关闭不是异常，而是正常生命周期事件。当然，业务层接口也可以选择抛异常，但内部状态机仍要明确。

10.11 等待谓词的完整写法

生产者等待的谓词不是“收到 not full 通知”，而是“队列未满或已经关闭”。消费者等待的谓词不是“收到 not empty 通知”，而是“队列非空或已经关闭”。关闭必须出现在谓词里，否则关闭时即使 `notify_all`，线程醒来后发现队列仍满或仍空，又会继续睡回去，线程池无法退出。

Listing 10.4 push 的状态转换

```

1 bool BoundedQueue::push(std::int32_t value) {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_full_.wait(lock, [this]() {
4         return this->closed_ ||
5             this->size_ < static_cast<std::int32_t>(this->buffer_.size());
6     });
7     if (this->closed_) {
8         return false;
9     }
10
11     this->buffer_[static_cast<std::size_t>(this->tail_)] = value;
12     this->tail_ = (this->tail_ + 1) %
13         static_cast<std::int32_t>(this->buffer_.size());
14     ++this->size_;
15     this->not_empty_.notify_one();
16     return true;
17 }

```

Listing 10.5 pop 的状态转换

```

1 std::optional<std::int32_t> BoundedQueue::pop() {
2     std::unique_lock<std::mutex> lock(this->mutex_);
3     this->not_empty_.wait(lock, [this]() {
4         return this->closed_ || this->size_ > 0;
5     });
6     if (this->size_ == 0) {
7         return std::nullopt;
8     }
9
10    const std::int32_t value =
11        this->buffer_[static_cast<std::size_t>(this->head_)];
12    this->head_ = (this->head_ + 1) %
13        static_cast<std::int32_t>(this->buffer_.size());
14    --this->size_;
15    this->not_full_.notify_one();
16    return value;
17 }

```

这两个片段体现了条件变量的关键原则：状态改变和谓词检查都在同一把锁保护下；通知只用于让等待者重新检查谓词；关闭是状态机的一部分。注意通知可以在持锁时调用，也可以在释放锁后调用，不同写法有不同唤醒和竞争细节。教材阶段更重要的是先保证谓词正确，再讨论微小性能差异。

10.12 锁内代码的边界

有界队列的锁内代码应只做状态转换，不应执行用户回调、复杂分配、日志格式化或长时间 I/O。若 **push** 在锁内调用用户提供的处理函数，队列锁就保护了不可控代码，任何慢操作都会阻塞所有生产者和消费者。若 **pop** 在锁内执行任务，线程池全局队列会退化串行执行器。

这条原则可以推广到所有锁。锁内只维护不变量，锁外做耗时工作。若必须在锁内移动复杂对象，应考虑预分配、移动语义、异常安全和对象析构成本。C++ 中对象析构也可能执行代码，如果最后一个引用在锁内释放，析构函数可能做不可控工作。高质量并发代码会尽量让重析构发生在锁

外。

10.13 异常安全和状态回滚

C++ 队列如果存储复杂对象，`push` 可能在移动或拷贝元素时抛异常。若异常发生在 `tail` 和 `size` 已经修改之后，不变量就会损坏。教学示例用 `std::int32_t` 可以避开这个问题，但正式教材必须说明边界：泛型队列需要把可能抛异常的操作和状态提交点分开，或者要求元素移动不抛异常，或者使用预构造槽位和 `placement new` 小心管理生命周期。

状态机里要有提交点。对于 `push`，元素成功放入槽位并且 `tail/size` 更新后，操作才对消费者可见；对于 `pop`，元素被取出并且 `head/size` 更新后，槽位才重新可用。若中间步骤可能失败，就要保证失败前队列仍保持旧状态，或失败后能回滚。锁只能提供互斥，不能自动提供异常安全。

10.14 信号量实现 bounded buffer 的视角

有界队列也可以用两个信号量表达：一个信号量记录空槽数量，一个信号量记录已有元素数量，再用互斥保护环形索引。生产者先获取空槽，再写入；消费者先获取元素，再读取。这个模型把“资源数量”和“状态互斥”分开，适合帮助读者理解信号量。

但信号量版本同样需要关闭语义。若消费者阻塞在元素信号量上，关闭时如何唤醒？若生产者阻塞在空槽信号量上，关闭时如何退出？标准信号量本身不携带“关闭”状态，需要额外状态和唤醒策略。条件变量版本把谓词写在代码里，更适合教学；信号量版本更接近资源额度控制。两者没有绝对优劣，关键是状态机是否完整。

10.15 锁顺序和组合操作

单个队列容易证明，多个锁组合就会出现锁顺序问题。假设线程池有全局队列锁、worker 本地队列锁和指标锁。如果 worker 持有本地队列锁时尝试拿全局队列锁，而另一个线程持有全局队列锁时尝试拿本地队列锁，就可能死锁。工程系统应定义锁级别：例如先全局配置锁，再队列锁，再指标锁；或者避免同时持有多把锁，用消息和局部复制拆开组合操作。

组合操作还会诱导“为了省事加一把大锁”。大锁让正确性简单，却可能把整个运行时串行化。合理做法是先用粗锁实现 reference，写清不变量和测试；再根据测量拆分锁，而不是一开始就写复杂细锁。拆锁时，每拆一把都要重新写不变量：哪些状态仍由旧锁保护，哪些状态由新锁保护，跨锁关系如何保持。

10.16 futex 慢路径和用户态 fast path

Linux 上常见 mutex 和 condition variable 都会尽量把无竞争路径留在用户态。无竞争加锁可能只是一次原子状态改变；竞争时才进入 futex 让线程睡眠。条件变量等待也会释放 mutex 并进入等待，唤醒后重新竞争 mutex。这个设计说明锁不是一上来就系统调用，系统调用通常是竞争或等待的慢路径。

阅读 futex 相关内容时，要抓住三个点。第一，用户态变量保存同步状态，内核只在需要睡眠和唤醒时参与。第二，等待必须和用户态状态检查配合，避免丢失唤醒。第三，唤醒不等于立即执行，被唤醒线程还要进入调度器、被安排到 CPU，并重新竞争锁。因此高竞争条件变量的成本不只是 `notify_one`，还包括调度和再次竞争。

10.17 本章落到贯穿项目

Compute Systems Engine 当前的 `BoundedMpmcQueue` 已经有容量、head、tail、size、mutex 和 `not_full`，但它还不是完整线程池队列，因为它只有阻塞 `push` 和非阻塞 `try pop`，没有 `not_empty` 等待，也没有 `close`。作为教学阶段的最小模块可以接受，但后续运行时章节必须补全关闭语义和消费者等待，否则 worker 无法优雅阻塞和退出。

本章要求项目至少增加设计文档或测试计划：列出队列不变量，列出 `push`、`try pop`、未来 `blocking pop` 和 `close` 的状态转换，列出需要的测试。测试不能只 `push` 两个值再 `pop` 两个值，还要覆盖容量为一、队列满时生产者等待、队列空时消费者等待、`close` 唤醒等待者、多生产者多消费者压力、异常容量和析构时没有等待线程。

10.18 本章习题

习题与作业

习题一：为当前项目的 `BoundedMpmcQueue` 写不变量清单。逐行解释 `push` 和 `try_pop` 怎样保持这些不变量。指出当前接口缺少哪些线程池需要的语义。

习题二：把 `bounded queue` 设计成支持 `close`。不要求立刻写完整代码，但必须画出状态转换：`open nonempty`、`open empty`、`open full`、`closed nonempty`、`closed empty`。说明每个状态下 `push` 和 `pop` 应返回什么。

习题三：设计一个队列等待实验。多个生产者写入，消费者故意变慢，让队列经常满。记录生产者等待次数、最大队列长度和总吞吐。解释这是背压，不是单纯性能退化。

习题四：阅读 `glibc`、`musl` 或 `Linux futex` 文档和源码入口，写清 `mutex` 低竞争 `fast path` 和竞争 `slow path` 的差异。报告中必须说明哪些部分发生在用户态，哪些部分进入内核。

常见误区

不要把锁竞争问题简单理解成“换成无锁”。如果临界区保护的是复杂不变量，无锁实现可能更难证明正确，也可能在高竞争下 `CAS` 失败更多。

实验：实现一个 `bounded queue`。先用 `mutex` 和 `condition variable` 写阻塞 `push/pop`，再记录高生产者数量下的等待时间。解释锁保护的状态和等待谓词。

第 11 章

原子操作与内存序

原子操作提供不可分割的读写或读改写，并参与线程间同步。内存序定义操作之间的可见性和重排约束。理解原子操作，必须区分原子性、顺序性和生命周期。

11.1 原子不是普通变量

`std::atomic` 的 `load`、`store`、`fetch_add`、`compare_exchange` 等操作会生成特殊指令或特殊约束。它们可以避免数据竞争，但不自动保证算法正确。一个原子计数器容易写，一个无锁队列需要处理节点发布、竞争失败、ABA 和回收。

低竞争下，原子操作可能比 `mutex` 轻；高竞争下，多个线程反复修改同一 `cache line`，原子也会扩展性很差。判断依据仍然是共享写频率和一致性流量。

11.2 常见内存序

`memory_order_relaxed` 只保证原子性，不建立跨变量同步。计数器统计常用 `relaxed`，因为只关心最终数值。`memory_order_release` 发布写入，`memory_order_acquire` 获取发布的结果，二者配合可建立 `happens-before`。`memory_order_seq_cst` 提供更强的全局顺序，但成本和约束也更强。

选择内存序的原则是从语义出发，而不是从性能愿望出发。若一个线程写好对象后通过原子标志发布，写标志应 `release`，读标志应 `acquire`。若只是统计事件数量，`relaxed` 足够。

Listing 11.1 `release/acquire` 发布数据

```
1 struct Shared {
2     std::int32_t data = 0;
3     std::atomic<bool> ready = false;
4 };
5
6 void producer(Shared& shared) {
7     shared.data = 42;
8     shared.ready.store(true, std::memory_order_release);
9 }
10
11 std::int32_t consumer(Shared& shared) {
12     while (!shared.ready.load(std::memory_order_acquire)) {
13     }
14     return shared.data;
15 }
```

`release store` 之前的普通写，在 `acquire load` 看到该 `store` 后，对 `consumer` 可见。这里不能把 `ready` 简单改成普通 `bool`，也不能把 `acquire/release` 无脑改成 `relaxed`；否则程序缺少跨线程发布关系。

理解 `acquire/release` 时，不要把它想成“刷新所有缓存”这样粗糙的动作。更准确的教材模型是：`release` 把之前的写放在发布点之前，`acquire` 在看到这个发布后，让之后的读写不能越过获取点，并建立 `happens-before`。具体硬件怎样实现，取决于架构强弱和编译器约束。C++ 程序只应依赖语言层面的同步关系。

11.3 Relaxed 的正确用途

Relaxed 常用于统计计数、采样标志、引用计数的某些阶段和不参与对象发布的指标。它保证原子变量自身不会数据竞争，但不保证其他变量的可见性。用 relaxed 写高性能代码时，必须能说清“我只需要这个原子变量自己的数值，不需要借它同步别的数据”。

例如请求计数器可以用 relaxed，因为最终统计只关心总次数。对象指针发布不能只用 relaxed，因为读到指针的线程还需要看到对象内部字段已经初始化。

11.4 CAS 循环

Compare-and-swap 读取旧值，若仍等于期望值就写入新值。无锁算法常用 CAS 循环处理竞争。CAS 失败不表示错误，而是说明其他线程已经修改状态，需要重新读取并尝试。

CAS 循环必须考虑进度。低竞争时很快，高竞争时可能大量失败。复杂结构还要考虑 ABA：一个位置从 A 变 B 又变回 A，CAS 看到 A 可能误以为没有变化。解决方法包括版本号、hazard pointer、epoch 回收等。

CAS 循环还必须考虑循环体里是否做了不可重复副作用。失败重试意味着循环体可能执行多次，如果每次尝试都分配内存、写日志、修改外部状态或增加不可回滚计数，就会产生额外语义问题。常见写法是先准备候选新状态，在 CAS 成功后才执行对外可见副作用；失败时只更新局部期望值并重试。

11.5 CAS 的 weak 和 strong 版本

`compare_exchange_weak` 允许伪失败，即使当前值等于期望值也可能失败，因此通常放在循环里。`compare_exchange_strong` 不允许伪失败，更适合不在循环里的单次尝试。很多平台上二者生成代码可能接近，但语义差异必须理解。

CAS 还有一个容易忽视的行为：失败时会把当前实际值写回 `expected` 参数。这让循环可以基于新值继续尝试，但也容易让初学者误用 `expected`。写无锁代码时，要把成功内存序和失败内存序都写清楚。

11.6 内存屏障和硬件

不同架构的内存模型不同。x86-64 相对强，ARM 和 POWER 更弱。C++ 内存序提供跨平台抽象，编译器和硬件会根据目标架构生成必要约束。不要因为某段代码在 x86 上“看起来没问题”，就认为它是正确的并发程序。

11.7 原子与缓存一致性

原子读写通常会目标 cache line 进入独占修改路径。多个核心同时对一个原子变量做 `fetch_add`，实际上是在争夺同一条 cache line 的所有权。内存序越强，编译器和硬件重排空间越小；竞争越强，一致性流量越大。

这就是为什么高性能计数常用分片计数器。每个线程或每个核心写自己的本地计数，读取总数时再合并。它牺牲实时精确读取成本，换取写入热路径扩展性。

11.8 内存序选择流程

选择内存序可以按问题反推。第一，是否只需要原子变量自身不撕裂、不数据竞争。如果是统计计数，通常 relaxed 足够。第二，是否通过某个原子变量发布了其他普通数据。如果是，需要 `release/acquire` 或更强同步。第三，是否需要多个线程观察到一个全局一致顺序。如果确实需要，才考虑 `memory_order_seq_cst`。第四，是否可以用 mutex 表达更清晰的不变量。如果共享状态复杂，mutex 往往比一组难懂的原子更可靠。

内存序不是性能调参旋钮。把 `memory_order_seq_cst` 改成 relaxed 之前，必须写出被保留的同步关系和被放弃的顺序关系。若写不出来，说明程序设计还没清楚到可以安全降级。

11.9 x86 和 ARM 的差异

x86-64 的内存模型相对强，很多 acquire/release 在硬件指令上看起来很轻；ARM 等架构更弱，可能需要更显式的屏障。可移植 C++ 代码不能因为在 x86 上测试通过，就省略必要内存序。教材示例必须按 C++ 语义写正确，而不是按某台机器的偶然行为写正确。

11.10 三件事必须分开

学习原子操作时，最重要的是把三件事分开：原子性、可见性和顺序性。原子性回答“这个变量本身的读写会不会撕裂，会不会产生数据竞争”。可见性回答“一个线程写入的其他数据，另一个线程什么时候能看到”。顺序性回答“多个操作之间能否被编译器或硬件重排，其他线程能以什么顺序观察到”。很多错误来自把原子性误当成全部同步。

例如 `std::atomic<bool>ready` 可以原子地表示标志，但如果生产者用 relaxed 存储 `ready=true`，消费者用 relaxed 读取到 true 后再读普通字段，普通字段的初始化不一定通过这个标志建立可见性关系。标志本身没有数据竞争，不代表对象发布正确。反过来，一个 relaxed 计数器用于统计请求次数完全可以，因为读取计数不需要借它观察其他普通数据。

教材里的判断标准应是：这个原子变量是否只是自己的值有意义，还是它承担了发布其他状态的责任？只看自己的值，通常 relaxed 可以讨论；发布其他状态，就需要 release/acquire、mutex 或更强同步。

11.11 发布对象的完整故事

对象发布是内存序最常见的真实案例。生产者分配或准备一个对象，写入多个普通字段，然后把指针或 ready 标志交给消费者。消费者看到标志后读取对象字段。如果没有同步，消费者可能看到未初始化或部分初始化状态；即使在某台 x86 机器上看不出来，C++ 语义也不允许依赖这种偶然。

Listing 11.2 用原子指针发布只读对象

```

1 struct Config {
2     std::int32_t shard_count = 0;
3     std::int32_t queue_capacity = 0;
4 };
5
6 std::atomic<Config*> published_config{nullptr};
7
8 void publish(Config* config) {
9     config->shard_count = 16;
10    config->queue_capacity = 4096;
11    published_config.store(config, std::memory_order_release);
12 }
13
14 Config* acquire_config() {
15     Config* config = nullptr;
16     while (config == nullptr) {
17         config = published_config.load(std::memory_order_acquire);
18     }
19     return config;
20 }

```

release store 之前对 `config` 字段的写入，在 acquire load 读到同一个指针后，对消费者可见。这个例子还隐藏了生命周期问题：谁拥有 `Config`，何时释放，是否允许热更新？release/acquire 只解决发布时的可见性，不自动解决对象回收。无锁章节会继续处理这个问题。

11.12 消息传递和 happens-before

内存序最终要落到 happens-before 关系。若线程 A 的某些写入 happens-before 线程 B 的某些读取，B 就能按 C++ 规则观察到 A 发布的结果。mutex 的 unlock 和后续 lock 可以建立这种关系；release store 和读到它的 acquire load 也可以建立这种关系；线程创建和 join 也有同步语义。

理解 happens-before 时，不要把它想成“时间上先发生”。两个线程在物理时间上先后执行，不一定建立 C++ 同步关系。生产者可能确实先写了 data，再写 ready；消费者也可能确实后读 ready，再读 data。但如果 ready 使用 relaxed，语言层面没有建立 data 的发布关系，程序就缺少可移植保证。并发正确性看的是同步关系，不是肉眼看到的执行顺序。

11.13 Release sequence 和读改写

C++ 内存模型里，release sequence 处理了一类常见场景：一个 release 操作后面接着对同一个原子变量的一系列读改写操作，acquire 读到这个序列中的值时，仍可能与最初 release 建立同步。这个规则让一些引用计数、队列状态和发布协议能正确表达，但也容易让初学者迷糊。

本书不要求在入门阶段手写复杂 release sequence 算法，但要知道两个原则。第一，同步关系通常围绕同一个原子对象上的读写建立，不能随便把一个原子变量的 release 借给另一个原子变量。第二，读改写操作既读旧值又写新值，它的成功和失败内存序都需要理解。CAS 成功时可能发布新状态，失败时只是观察当前状态；失败内存序不能比成功内存序更强。

11.14 CAS 循环的副作用边界

CAS 循环最容易漏讲的是副作用边界。因为 CAS 可能失败，循环体可能执行很多次。若循环体里做了不可回滚副作用，例如写日志、发送网络请求、增加外部计数、释放对象或修改另一个结构，那么失败重试会让语义混乱。正确写法通常把“准备候选值”和“提交副作用”分开，只有 CAS 成功后才执行对外可见副作用。

Listing 11.3 CAS 循环只在成功后提交外部副作用

```

1 bool try_set_max(std::atomic<std::int64_t>& target,
2                 std::int64_t candidate) {
3     std::int64_t observed = target.load(std::memory_order_relaxed);
4     while (observed < candidate) {
5         if (target.compare_exchange_weak(
6             observed,
7             candidate,
8             std::memory_order_release,
9             std::memory_order_relaxed)) {
10             return true;
11         }
12     }
13     return false;
14 }
```

这个函数在失败时，observed 会被更新成当前实际值，下一轮基于新值继续判断。函数没有在循环内部写外部日志或修改其他共享结构，因此失败重试不会产生额外语义。若调用者需要记录“最大值被更新”，应在返回 true 后记录。

11.15 原子引用计数的双重语义

引用计数常用原子实现，但它不是简单计数器。增加引用通常只需要保证计数本身原子，减少引用并发现自己是最后一个引用时，需要在销毁对象前看到其他线程对对象的修改。也就是说，引用计数同时涉及原子性和对象生命周期。很多成熟库在 increment、decrement 和最后释放之间使用不同内存序，就是因为不同阶段承担不同语义。

教材不应让读者以为“引用计数用 relaxed 就行”或“引用计数都用 memory_order_seq_cst

最安全”。更好的说法是：引用计数需要一套完整协议。获取引用时必须确保对象仍然活着；释放引用时如果不是最后一个，只是减少计数；如果是最后一个，必须和其他线程对对象的访问建立足够同步，然后才能析构。协议比单条原子指令重要。

11.16 Seq-cst 的价值和成本

`memory_order_seq_cst` 提供一个所有 seq-cst 操作参与的单一全局顺序。它让推理更接近直觉，适合初始实现、教学示例和确实需要全局顺序的算法。但它不是“让并发程序自动正确”的开关。若对象生命周期没处理，seq-cst 也不能防止 use-after-free；若不变量跨多个变量没有协议，seq-cst 原子也不能自动保持不变量。

性能上，seq-cst 可能限制编译器和硬件重排，在某些架构上需要更重屏障。x86 上某些 seq-cst load/store 也许看起来不重，但读改写和跨平台成本仍要考虑。工程建议是：先用清晰同步写正确，再在热点路径中根据证明降低到 acquire/release 或 relaxed。不要为了性能从一开始就写最弱内存序，也不要为了省脑把所有原子永远 seq-cst 后忽略结构设计。

11.17 原子和锁的组合

原子和 mutex 可以组合使用，但必须明确各自保护什么。常见模式是用原子做 fast path 标志，用 mutex 保护复杂慢路径状态。例如一个队列可以用原子 size 做近似指标，但真实 head、tail、buffer 仍由 mutex 保护。错误模式是同一份状态有时用锁保护，有时绕过锁用原子修改，导致不变量被拆碎。

如果一个变量参与锁保护的不变量，就不要随意在锁外用 relaxed 读它来做决策，除非这个决策允许过期和近似。例如队列长度指标可以近似展示；判断是否可以安全 pop 不能靠锁外近似 size。区分“指标”和“协议状态”非常重要。

11.18 内存序测试为什么困难

内存序 bug 很难通过普通测试证明不存在。它可能只在特定架构、编译器优化、核心数和时序下出现。压力测试能增加信心，但不能替代语义证明。写并发代码时，应先用 happens-before 关系证明，再用 ThreadSanitizer、压力测试和架构测试辅助验证。

手机或 x86 设备上跑过不代表 ARM 服务器、POWER 或未来编译器也正确。可移植 C++ 并发必须按语言模型写。教材示例因此宁愿稍保守，也不能依赖某个硬件“实际上不会乱序”的经验。

11.19 本章落到贯穿项目

Compute Systems Engine 在本章应增加三类原子实验。第一，统计计数器：mutex counter、atomic relaxed counter、sharded counter，用来区分原子性和扩展性。第二，发布实验：一个 producer 初始化配置后 release 发布，consumer acquire 获取，用来说明对象可见性。第三，CAS 实验：实现 `try_set_max` 或无锁最大值更新，记录 CAS 失败次数，说明竞争下重试成本。

这些实验都要写清语义。计数器 relaxed 的理由是“不发布其他数据”；配置发布 acquire/release 的理由是“读取者需要看到普通字段初始化”；CAS 的理由是“成功那一刻是线性化点”。如果写不出理由，就不应该选择内存序。

11.20 本章习题

习题与作业

习题一：给三个场景选择内存序并解释理由：请求计数器、配置对象发布、无锁最大值更新。每个场景都要说明是否同步其他普通数据，是否需要全局顺序，失败时会发生什么。

习题二：把一个错误的 relaxed 发布例子改成 release/acquire。要求画出 happens-before 关系，并说明对象生命周期仍然需要额外处理。

习题三：写一个 CAS 循环，要求循环体内没有不可回滚副作用。然后列出三种不应该放进 CAS 重试循环的副作用，并解释原因。

习题四: 阅读项目中的 `AtomicCounter`。解释为什么当前 `fetch_add` 可以使用 `relaxed`, 也说明它为什么不适合作为对象发布机制或队列同步机制。

实验: 比较 `mutex counter` 和 `atomic counter`。在低线程数和高线程数下记录耗时。解释 `relaxed` 为什么适合这个计数器, 也说明它为什么不能替代对象发布同步。

第 12 章

无锁数据结构

无锁数据结构的目标不是“没有同步”，而是在不使用互斥锁阻塞线程的情况下维护共享状态。它通常依赖原子操作、CAS、内存序和严格的对象生命周期管理。学习无锁结构，要先学习进度保证和失败模式。

12.1 进度保证

Blocking 算法可能因为一个线程持锁停顿而阻塞其他线程。Lock-free 算法保证系统整体持续前进，但单个线程可能反复失败。Wait-free 算法保证每个线程在有限步内完成，通常更难实现。Obstruction-free 只在无竞争时保证进展。

这些术语不是装饰。一个队列在压力下 CAS 反复失败，可能仍然 lock-free，但尾延迟很差。工程上需要结合竞争程度、临界区复杂度和可维护性选择方案。

进度保证要和系统目标匹配。低延迟交易路径可能愿意为 wait-free 付出巨大复杂度；普通后台队列通常 mutex 就足够；高吞吐日志管线可能只需要 SPSC ring buffer；线程池全局队列可能更适合分片队列加工作窃取。不要把 lock-free 当成荣誉标签，它只是进度性质的一种选择。

12.2 无锁栈和 ABA

Treiber stack 是经典无锁栈。push 读取 head，把新节点 next 指向旧 head，再 CAS head。pop 读取 head 和 next，再 CAS head 到 next。问题在于 ABA：head 可能从 A 变成 B 又变回 A，CAS 看见 A 成功，但 A 的生命周期可能已经变化。

解决 ABA 需要额外机制。版本标签把指针和计数器一起比较；hazard pointer 让线程声明自己正在访问的节点；epoch reclamation 延迟释放节点直到所有线程离开旧 epoch。无锁算法难点往往不在 CAS 本身，而在内存回收。

```
Treiber push shape:
load old head
new node next = old head
CAS head from old head to new node

Treiber pop shape:
load old head
load next from old head
CAS head from old head to next
```

这张形态图比直接贴完整代码更重要。读者必须先看出线性化点在哪里：push 和 pop 都在 CAS 成功那一刻生效。CAS 之前读到的状态只是一次尝试，失败后不能假装操作已经发生。

12.3 内存回收为什么困难

带锁结构中，线程持有锁时可以知道没有其他线程正在修改结构。无锁结构没有这个简单边界。一个线程从 head 读到节点指针后，可能还没来得及读取 next，另一个线程已经 pop 并释放了该节点。第一个线程随后访问这个指针，就会出现 use-after-free。

垃圾回收语言可以把节点释放推迟到不可达之后，C++ 程序必须自己设计回收协议。Hazard pointer 的思路是线程公开声明“我可能访问这个节点”，释放者扫描这些声明，确认没有线程持有后再释放。Epoch 的思路是把释放延迟到所有线程都离开旧时代之后。二者都有开销，也都有使用约束。

Hazard pointer 更精确，但每个线程需要发布自己正在保护的指针，释放者需要扫描 hazard 集

合。Epoch 回收批量化更强，读路径常常更轻，但如果某个线程长时间停留在旧 epoch，回收会延迟，内存占用可能增长。RCU 也是类似思想：读者进入读侧临界区，写者等待 grace period 后回收旧版本。不同机制的共同点是：释放不能只看“节点已经从结构中摘掉”，还要看“是否还有线程可能持有旧指针”。

12.4 ABA 的具体例子

假设栈顶是 A，线程 T1 读取 head=A，准备把 head 改成 A->next。此时 T2 pop A，又 pop B，然后把 A push 回栈顶。T1 再 CAS head，从 A 改成旧的 A->next。CAS 看到 head 仍然是 A，于是成功，但这个 A 已经不是 T1 当初理解的结构关系。版本标签通过让 head 变成“指针加版本号”，使 A 回来时版本不同，从而让 CAS 失败。

12.5 环形队列

有界环形队列适合固定容量、低分配、缓存友好的场景。单生产者单消费者队列可以用较简单的 head/tail 原子实现；多生产者多消费者队列需要为槽位维护序号或状态，避免多个线程同时写同一槽位。

环形队列的性能来自固定内存、连续布局和减少分配。但如果所有生产者争用同一个 tail，仍然会有原子热点。分片队列、每线程队列和工作窃取可以缓解热点。

12.6 SPSC、MPSC 和 MPMC

队列难度取决于生产者和消费者数量。SPSC 只有一个生产者和一个消费者，head 只被消费者写，tail 只被生产者写，可以用较轻的同步。MPSC 有多个生产者争用 tail，消费者独占 head。MPMC 两端都有竞争，通常需要更复杂的槽位序号或链表算法。

设计队列时先问清使用场景，不要直接上最复杂的 MPMC。日志线程、音频缓冲和管线阶段之间常常只需要 SPSC；线程池全局队列可能需要 MPMC；工作窃取运行时则常用每 worker deque 加窃取操作。

SPSC ring buffer 是学习无锁队列的最佳起点。生产者独占 tail，消费者独占 head，因此每个索引只有一个写者；双方通过 acquire/release 观察对方进度。MPSC 和 MPMC 复杂得多，因为多个线程会争用同一个端点，需要 CAS、槽位状态或序号。教材顺序应从 SPSC 到 MPSC 再到 MPMC，而不是一开始贴一个难以证明的万能队列。

12.7 线性化点

并发数据结构的线性化点，是操作在逻辑上生效的那一瞬间。带锁结构的线性化点通常在锁内某次状态修改；无锁结构的线性化点通常是成功 CAS。若无法指出线性化点，就很难证明结构正确。

例如无锁栈 push 的线性化点是 CAS head 成功。CAS 失败的尝试没有生效，必须重试。测试无锁结构时，不能只看最终数量，还要考虑每个操作是否能排列成某个合法串行顺序。

线性化点还帮助写测试。对于栈，可以记录每个 push 的值和每个 pop 的返回，检查是否存在某个合法串行顺序；对于队列，要检查 FIFO 语义是否在并发交错下仍成立；对于计数器，要检查每次成功自增是否唯一。压力测试只能增加信心，不能替代线性化论证。无锁结构没有明确线性化点，通常就不该进入生产代码。

12.8 何时不用无锁

如果共享状态复杂、竞争不高、临界区很短，mutex 可能更清晰也足够快。如果需要条件等待，锁加条件变量往往比忙等 CAS 更节能。无锁结构适合明确的高频热路径，且必须有严格测试、压力测试和代码审查。

12.9 工程验收标准

无锁结构的验收标准应比普通容器更高。第一，要有清晰的不变量、线性化点和进度保证说明。第二，要说明内存回收策略，不能留下 use-after-free 风险。第三，要有单线程 reference 行为测试、并发压力测试、边界容量测试和关闭语义测试。第四，要有性能对照，证明它在目标竞争模式下确

实优于 mutex 或分片锁。第五，要说明不适用场景，例如高竞争 CAS 风暴、需要阻塞等待或对象生命周期复杂。

如果这些条件做不到，就应该优先使用更简单的同步结构。复杂无锁代码的维护成本非常高，错误通常低概率且难复现。严肃工程里，简单正确经常比炫技更有价值。

12.10 SPSC ring buffer 完整推导

学习无锁队列应从 SPSC 开始，因为它的所有权最清楚：只有一个生产者写 tail，只有一个消费者写 head。生产者会读 head 判断是否有空间，消费者会读 tail 判断是否有元素。每个索引只有一个写者，这大幅降低了同步难度。SPSC 的关键不是“没有锁”，而是用所有权设计减少共享写。

环形队列保留一个空槽可以区分满和空。若 `next_tail==head`，队列满；若 `head==tail`，队列空。生产者写入元素后 release 发布新的 tail；消费者 acquire 读取 tail 后，能看到生产者写入的元素。消费者取出元素后 release 发布新的 head；生产者 acquire 读取 head 后，知道槽位已经可复用。对于简单整数元素，这个模型很直观；对于复杂对象，还要处理构造、析构和异常安全。

Listing 12.1 SPSC ring buffer 的核心实现

```

1 class SpscIntQueue {
2 public:
3     explicit SpscIntQueue(std::int32_t capacity)
4         : buffer_(static_cast<std::size_t>(capacity + 1), 0) {}
5
6     bool try_push(std::int32_t value) {
7         const std::uint64_t tail =
8             this->tail_.load(std::memory_order_relaxed);
9         const std::uint64_t next_tail = this->next(tail);
10        const std::uint64_t head =
11            this->head_.load(std::memory_order_acquire);
12        if (next_tail == head) {
13            return false;
14        }
15        this->buffer_[static_cast<std::size_t>(tail)] = value;
16        this->tail_.store(next_tail, std::memory_order_release);
17        return true;
18    }
19
20    std::optional<std::int32_t> try_pop() {
21        const std::uint64_t head =
22            this->head_.load(std::memory_order_relaxed);
23        const std::uint64_t tail =
24            this->tail_.load(std::memory_order_acquire);
25        if (head == tail) {
26            return std::nullopt;
27        }
28        const std::int32_t value =
29            this->buffer_[static_cast<std::size_t>(head)];
30        this->head_.store(this->next(head), std::memory_order_release);
31        return value;
32    }
33
34 private:
35     std::uint64_t next(std::uint64_t index) const {
36         const std::uint64_t size =
37             static_cast<std::uint64_t>(this->buffer_.size());

```

```

38     return (index + 1) % size;
39 }
40
41 std::vector<std::int32_t> buffer_;
42 alignas(64) std::atomic<std::uint64_t> head_{0};
43 alignas(64) std::atomic<std::uint64_t> tail_{0};
44 };

```

这段代码故意只支持 SPSC。若两个生产者同时调用 `try_push`，它们都会写 `tail` 和同一槽位，算法不再正确；若两个消费者同时 `try_pop`，它们都会写 `head`，也不正确。接口文档必须写清这个约束。无锁结构最忌讳把适用场景藏起来，让调用者误用。

`head` 和 `tail` 分开放置，是为了减少生产者和消费者频繁写同一 `cache line`。这里使用 `std::uint64_t` 是为了让索引范围清楚；容量参数仍需检查，生产代码还要处理容量溢出。教学代码展示核心同步，工程代码还要补足参数验证、移动禁用、析构约束和测试。

12.11 SPSC 的线性化点

SPSC `try_push` 的线性化点是 `release` 写 `tail`。写入 `buffer` 只是准备数据；`tail` 更新后，消费者 `acquire` 看到新 `tail`，元素才对消费者可见。`try_pop` 的线性化点可以看作读取元素并 `release` 更新 `head` 的组合：在返回元素前，它已经把槽位归还给生产者。更严格地说，对于整数槽位，消费者读取元素后操作结果已经确定，`head` 更新发布空间可用。

线性化点帮助分析失败路径。若 `try_push` 发现队列满返回 `false`，它没有写入 `buffer`，也没有更新 `tail`，操作没有生效。若 `try_pop` 发现空返回 `nullopt`，它没有更新 `head`。测试要覆盖这些失败路径，因为失败路径在有界队列中不是异常，而是正常背压。

12.12 从 SPSC 到 MPSC

MPSC 增加多个生产者。此时 `tail` 不再只有一个写者，生产者之间必须争用写入位置。常见做法是使用 `fetch_add` 或 `CAS` 为每个生产者预留槽位，然后再处理槽位可见性。问题立刻变复杂：生产者 A 预留了槽位但还没写完，生产者 B 写完后能否让消费者看到后面的槽位？消费者如何知道某个槽位的数据已经准备好？队列满时已经预留的位置如何处理？

这就是为什么 MPSC/MPMC 队列常为每个槽位维护序号或状态。`tail` 只表示预留进度不够，还要知道槽位是否已提交。队列算法难度不是来自环形数组本身，而是来自多写者之间的提交顺序、可见性和容量回收。教材应明确告诉读者：不要把 SPSC 代码简单加 `CAS` 就当 MPSC。

12.13 MPMC 队列的槽位序号直觉

一种经典 MPMC 有界队列会让每个槽位带一个 `sequence`。生产者根据 `tail` 找到槽位，检查 `sequence` 是否表示槽位空闲；成功 `CAS tail` 后写入数据，再更新 `sequence` 表示可读。消费者根据 `head` 找到槽位，检查 `sequence` 是否表示槽位可读；成功 `CAS head` 后读取数据，再更新 `sequence` 表示空闲。`sequence` 用于区分同一个数组槽位在不同轮次中的状态，避免只靠 `head/tail` 造成混淆。

这个模型比 SPSC 难很多。每个端点都有多个写者，`CAS` 失败是正常路径；槽位 `sequence` 也要选择正确内存序；容量必须是固定的或 `carefully` 管理；队列关闭和阻塞等待又是额外层次。生产系统通常会优先使用成熟库，或者先用 `mutex bounded queue`，只有在明确瓶颈和使用场景后才自研 MPMC。

12.14 ABA 和内存回收的关系

ABA 不只是“值变回去了”的小问题，它经常和内存回收连在一起。节点 A 被 `pop` 后释放，分配器可能把同一地址重新分配给新节点。另一个线程手里仍有旧指针，`CAS` 看到地址相同，以为结构没变，实际上对象生命周期已经换了。版本标签能区分同一地址的不同代；`hazard pointer` 和 `epoch` 能防止节点过早释放；垃圾回收语言则把释放延后到不可达之后。

C++ 无锁结构必须把“从数据结构中摘除”和“释放内存”分开。摘除只说明新操作不会再从结构入口到达该节点，不说明旧操作没有持有指针。回收协议就是证明旧操作不再持有指针的机制。

没有回收协议的无锁链表或栈，在轻测试下可能正常，在压力下可能 use-after-free。

12.15 Hazard pointer 的操作流程

Hazard pointer 的基本流程是：线程在读取共享指针后，把该指针发布到自己的 hazard 槽位；然后重新读取共享指针确认它没有变化；确认后才能安全解引用。删除节点的线程不能立即释放，而是把节点放入 retired 列表；当 retired 列表足够大时，扫描所有 hazard 槽位，把没有被任何线程保护的节点释放。

这套机制的成本很明确：读路径需要发布 hazard，释放路径需要扫描，线程数越多扫描成本越高。它的优点是精确，不需要等待所有线程离开全局 epoch。它适合指针访问较复杂、需要较及时回收的结构。教学时不一定要完整实现，但必须让读者知道无锁结构的真正难点在这里。

12.16 Epoch 回收的操作流程

Epoch 回收把时间分成阶段。线程进入无锁操作时宣布自己处于当前 epoch，退出时宣布离开。删除节点时不立即释放，而是记录删除发生的 epoch。只有当所有线程都已经离开旧 epoch，旧 epoch 中删除的节点才可以释放。它把单个节点的精确保护变成批量延迟回收。

Epoch 的读路径通常比 hazard pointer 更轻，但如果某个线程长时间不退出临界区，回收会被阻塞，内存占用可能增长。它适合操作短、线程行为可控的场景，不适合线程可能长期停顿在临界区的场景。系统设计必须考虑线程被抢占、阻塞、崩溃或取消时对回收的影响。

12.17 无锁不等于低尾延迟

lock-free 保证系统整体前进，不保证每个线程快速完成。高竞争 CAS 循环可能让某些线程反复失败，尾延迟很差。wait-free 才提供单线程有限步完成保证，但实现通常更复杂。对于服务系统，尾延迟可能比平均吞吐更重要，因此 lock-free 结构不一定比有锁结构更适合请求路径。

mutex 在高竞争下可能进入内核睡眠，平均吞吐下降但节省 CPU；lock-free CAS 在高竞争下可能烧满 CPU，不断失败重试。哪种更好取决于等待时间、核心预算、能耗、尾延迟和业务语义。教材不能把 lock-free 简化成“更高级所以更快”。

12.18 本章落到贯穿项目

Compute Systems Engine 的无锁路线应分三步。第一步，实现并测试 SPSC ring buffer，只用于单生产者单消费者管线，例如一个 I/O 模拟线程向一个计算线程发送批次。第二步，比较 SPSC 和 mutex bounded queue 在目标场景下的吞吐和等待行为，确认收益来自减少锁和固定内存，而不是改变语义。第三步，再研究 MPMC 或工作窃取 deque，并在实现前写出不变量、线性化点、内存序和回收策略。

项目不应急着把所有队列替换成无锁队列。线程池全局队列最开始用 mutex 更容易证明关闭语义；SPSC 用于清晰管线；MPMC 用于明确的多生产者多消费者热点；work stealing deque 用于本地队列和窃取。不同结构服务不同场景。

12.19 本章习题

习题与作业

习题一：为本章 SPSC ring buffer 写出不变量、满空判断、push 线性化点和 pop 线性化点。说明为什么它不能被多个生产者同时调用。

习题二：把 SPSC ring buffer 扩展到支持容量检查和关闭语义。只要求设计状态机，不要求完整代码。说明关闭后 producer 和 consumer 分别应看到什么返回值。

习题三：写一份 ABA 场景时间线，至少包含两个线程、两个节点、一次 pop、一次释放和一次地址复用。然后分别说明版本标签、hazard pointer、epoch 如何缓解。

习题四：选择一个成熟无锁队列实现或论文，阅读其不变量和内存回收策略。报告必须说明它适用 SPSC、MPSC 还是 MPMC，不能只说“这是无锁队列”。

实验：阅读 Compute Systems Labs 的 bounded queue，写出 mutex 版本保护的不变量。再设计一个 SPSC ring buffer 的接口，说明哪些状态可以用 relaxed，哪些位置需要 acquire/release。

第零部分

并行算法与运行时

第 13 章

归约、扫描与分区

并行算法的核心是处理依赖。归约、扫描和分区是许多系统的基础操作：数据库聚合、日志统计、图计算、排序、渲染和科学计算都会用到它们。

13.1 归约

归约把多个输入合并成一个输出。并行归约的基本策略是局部归约加全局合并。它的性能取决于输入划分、局部性、合并成本和结果类型。整数加法简单，浮点加法要考虑精度和顺序，字符串拼接则可能有大量分配。

归约优化通常先避免共享写。每个线程写自己的 `partial`，最后串行或树形合并。树形合并可以减少合并深度，也更适合分布式系统。

归约的第一步是写出代数性质。操作是否有单位元，是否满足结合律，是否满足交换律，是否允许改变顺序。整数最小值、最大值、按位或通常容易并行；浮点求和需要说明误差模型；哈希合并需要说明冲突处理；对象聚合需要说明生命周期和分配。没有这些语义前提，谈并行归约是不完整的。

归约的第二步是写出内存布局。`partial` 结果应避免 `false sharing`，通常每个 worker 一个对齐槽位。若 `partial` 很大，可以按 NUMA 节点或缓存块组织，避免最后合并阶段变成单线程长尾。归约不是只有“线程本地加一下”这一种形式，它可以是线性合并、二叉树合并、分层合并或流水线合并。

13.2 扫描

扫描也叫 `prefix sum`，输出每个位置之前元素的累计结果。它比归约更难，因为每个输出都依赖前缀。并行扫描通常分两阶段：先对每个块做局部扫描，得到块总和；再扫描块总和；最后把块前缀加回块内结果。

扫描是很多算法的 `building block`，例如 `stream compaction`、并行分配输出位置、基数排序和图前沿构造。理解扫描，可以帮助读者把“有顺序依赖”的问题拆成局部和全局两层。

扫描比归约更能体现并行算法设计。表面上，每个输出都依赖前面所有输入，似乎完全串行；但把输入分块后，每个块内部可以先做局部扫描，块总和再做一次小规模扫描，最后把块前缀加回去。这个套路说明，很多“看起来串行”的依赖，可以拆成局部依赖和块间摘要。分布式系统里的很多计算也是这个形状：节点内先处理，节点间交换摘要，再回填结果。

13.3 分区

分区把元素按谓词或 `key` 分到不同区域。并行分区要先统计每个线程每个桶的数量，再计算全局偏移，最后写入输出。这与扫描密切相关。直接让多个线程 `push` 到共享 `vector`，会引入锁或原子热点。

分区的关键是先分配写入位置，再并行写入。以 `copy_if` 为例，朴素并行写法可能让所有线程对输出 `vector` 做 `push`，需要锁或原子。更好的写法是每个线程先统计自己保留多少元素，对这些计数做扫描得到每个线程的输出起点，然后每个线程写入自己的连续区间。这样写入没有冲突，输出也保持可解释顺序。

13.4 负载均衡

静态划分适合每个元素成本相近的场景。若元素成本差异大，静态划分会让某些线程早早结束，其他线程仍在工作。动态调度、任务队列和工作窃取可以改善负载均衡，但会增加调度开销。

负载均衡不是越动态越好。静态划分没有调度开销，局部性好，适合规则数组和均匀工作。动态队列能处理不均匀工作，但会增加同步和队列争用。工作窃取减少全局争用，但实现更复杂。设计时要估算任务成本方差：如果每个块成本接近，静态块划分更好；如果成本长尾明显，动态调度

才值得。

13.5 并行算法的正确性模板

本册要求每个并行算法都写出四个正确性问题。第一，串行语义是什么，reference 怎样实现。第二，并行切分后，每个 worker 负责哪些输入和输出。第三，局部结果怎样合并，合并顺序是否影响结果。第四，是否存在共享写、重复写、漏写或越界写。性能优化只能在这些问题回答之后进行。

这套模板能防止“并行版本跑得快但其实错”。很多并行 bug 不会在小输入暴露，因为线程交错少、边界简单、缓存状态稳定。测试必须覆盖空输入、极小输入、刚好一个块、多个块、不能整除块大小、重复 key、倾斜 key 和高线程数。

实验：在 Compute Systems Labs 中扩展 parallel `count_if`。先让每个线程本地计数，再合并。随后实现输出压缩：先统计、扫描偏移，再写输出数组。

第 14 章

任务运行时与工作窃取

线程是执行资源，任务是工作单位。把线程和任务分离，是构建可扩展运行时的关键。线程池、任务队列、工作窃取和任务图，是现代并行运行时的核心结构。

14.1 线程池

线程池创建固定数量或可控数量的工作线程，任务提交到队列，由工作线程取出执行。这样避免频繁创建线程，也便于控制并发度。线程池接口看似简单，难点在关闭、异常传播、任务等待、队列阻塞和避免死锁。

如果线程池里的任务再等待同一个线程池中的子任务，而工作线程数量不足，就可能发生饥饿。运行时设计必须考虑任务依赖，而不只是一个全局队列。

线程池的最小语义包括三件事：提交任务是否可能失败，关闭后队列中任务是否继续执行，调用 `wait` 或析构时是否等待所有任务完成。没有这些语义，线程池只是一个会运行函数的对象，不是可靠运行时。教材实现应先把这些边界写清，再谈性能。

14.2 Future 和等待依赖

Future 把“将来会完成的结果”变成对象。它适合表达异步任务，但如果 worker 线程在执行任务时同步等待另一个 future，就可能形成线程池内阻塞。更好的方式是依赖表达给运行时，让运行时在依赖满足后再调度后续任务，而不是占着 worker 等。

这也是任务图运行时的价值：等待不再消耗工作线程，依赖边决定任务何时 ready。构建系统和数据流引擎都大量依赖这种模型。

Future 的危险是把异步伪装成同步。用户看到 `future.get()` 很方便，但如果在 worker 线程里调用它，就可能把 worker 变成等待者。运行时可以通过 work helping 缓解：等待时主动执行其他 ready 任务；也可以要求任务不能阻塞等待，只能注册 continuation。两种设计都要明确写进接口语义。

14.3 任务粒度

任务太大，负载不均；任务太小，调度成本高。合理粒度通常需要根据输入规模、核心数、缓存块和调度开销选择。递归拆分算法常设 cutoff，小于某个规模就顺序执行。

任务粒度可以通过一个简单不等式判断：每个任务的计算时间应显著大于一次提交、入队、出队和唤醒的固定成本。若任务只做几十条指令，调度成本会吞掉收益；若任务运行几百毫秒，负载不均会造成长尾。实践中常把任务切到缓存友好的块大小，再用 benchmark 调整 cutoff。

14.4 工作窃取

工作窃取让每个线程拥有本地双端队列。线程优先从本地取任务，空闲时从其他线程窃取任务。这减少全局队列争用，并改善缓存局部性。常见策略是 owner 从底部 push/pop, stealer 从顶部 steal。

工作窃取的正确性和性能都依赖队列设计。Chase-Lev deque 是经典方案，但实现复杂，涉及原子、内存序和边界竞争。教学时先理解模型，再进入源码。

工作窃取还有一个重要性质：正常情况下 worker 访问自己的本地 deque，只有空闲时才访问别人的 deque。这把高频操作留在本地，低频窃取才跨线程同步。相比所有 worker 抢一个全局队列，它减少了共享热点。这个结构和 NUMA、分布式本地优先原则一致：先消费近处工作，再去远处拿工作。

14.5 本地性和窃取方向

工作窃取通常让 owner 按 LIFO 处理本地任务，让 stealer 从另一端 FIFO 窃取较老任务。LIFO 有利于缓存局部性，因为新产生的子任务往往访问相近数据；窃取较老任务则倾向拿到较大工作块，减少窃取频率。这个设计体现了运行时对局部性和负载均衡的折中。

14.6 任务图

很多计算不是简单队列，而是任务之间有依赖。任务图把节点表示任务，边表示依赖。运行时在依赖满足后调度任务。构建系统、数据处理引擎和图计算框架都大量使用任务图思想。

任务图需要维护 ready 计数。每个任务记录还有多少前驱未完成；当前驱完成时，后继计数减一；计数变为零，任务进入 ready 队列。这个模型避免 worker 阻塞等待依赖。它的难点在于任务失败、取消、异常传播和图生命周期：一个任务失败后，后继任务是跳过、取消还是继续运行，必须在运行时语义里说明。

14.7 取消和关闭

运行时必须定义关闭语义。提交中的任务是否接受，队列里的任务是否执行完，正在运行的任务是否可取消，异常如何传播，这些都要明确。没有关闭语义的线程池很容易在程序退出时丢任务、死锁或访问已销毁对象。

14.8 运行时观测指标

任务运行时不能只测总时间。还要观察队列长度、每个 worker 执行任务数、空闲时间、窃取次数、任务平均耗时、最长任务、等待依赖数量和关闭耗时。没有这些指标，负载不均和调度开销很难定位。一个任务图慢，可能是关键路径太长，不是 worker 不够；一个线程池慢，可能是任务太小，不是锁太慢。

这些指标也会指导分布式计算。单机里 worker 空闲，对应集群里 executor 空闲；单机里队列积压，对应集群里 stage backlog；单机里任务倾斜，对应分布式里的热点 key。运行时思维是从多线程走向分布式框架的桥。

14.9 最小线程池的状态机

线程池最小实现通常包含任务队列、worker 列表、停止标志、未完成任务计数和条件变量。状态机比代码更重要。open 状态允许 submit；shutting down 状态拒绝新任务但继续执行队列中任务；stopped 状态表示 worker 已退出。若没有这些状态，析构、等待和异常路径会变得模糊。

Listing 14.1 线程池状态的概念模型

```
1 enum class RuntimeState : std::int32_t {
2     Open,
3     ShuttingDown,
4     Stopped,
5 };
6
7 struct RuntimeCounters {
8     std::uint64_t submitted = 0;
9     std::uint64_t completed = 0;
10    std::uint64_t failed = 0;
11 };
```

submit 在 Open 状态下把任务加入队列并通知 worker；在 ShuttingDown 或 Stopped 状态下应失败或抛出明确异常。**shutdown** 把状态从 Open 改成 ShuttingDown，并唤醒所有 worker；worker 在队列为空且状态不是 Open 时退出；最后一个 worker 退出后，线程池进入 Stopped。这个流程必须能重复调用而不破坏状态，即 shutdown 应尽量幂等。

未完成任务计数用于 **wait**。提交任务时增加，任务完成或失败时减少，计数变为零时通知等待

者。异常不能让计数泄漏，否则 `wait` 会永远阻塞。生产级线程池还需要保存异常并传给 `future` 或错误回调；教学实现至少要保证异常不会让 `worker` 线程静默终止后破坏计数。

14.10 任务对象和生命周期

任务不是单纯的 `std::function<void()>`。它可能携带输入范围、优先级、placement、依赖计数、取消标志、异常存储和指标字段。使用 `std::function` 便于教学，但可能分配内存、类型擦除成本较高，也不方便表达任务元数据。高性能运行时通常会使用自定义 `task object`、对象池或 intrusive queue。

任务生命周期至少有五个阶段：构造、提交、ready、running、completed。任务图还会有 waiting 状态，因为依赖未满足。取消和失败会引入 canceled、failed 等状态。每个状态允许哪些操作必须定义清楚。例如 running 任务是否能被取消？completed 任务的结果保存多久？shutdown 后 waiting 任务怎么处理？这些语义比 worker 循环更难。

14.11 全局队列线程池的 baseline

全局队列线程池是最好的 baseline。它容易实现，容易证明关闭语义，适合作为 correctness reference。它的缺点也清楚：所有 worker 竞争同一个队列锁；任务入队出队集中；任务局部性弱；负载高时队列可能成为瓶颈。正因为缺点明确，它才适合作为后续 work stealing 的对照。

教材实现应先写全局队列版本，并记录指标：每个 worker 执行任务数、队列最大长度、worker 空转次数、submit 等待时间。若这些指标显示全局队列不是瓶颈，就不应急着实现复杂 work stealing。若 worker 大量争用队列锁、队列长度波动大、任务很小且频繁，才有证据推进到本地队列。

14.12 工作窃取 deque 的所有权

工作窃取 deque 的核心是所有权不对称。owner 线程高频 push/pop 本地端，stealer 低频从另一端 steal。这个不对称让常见路径更轻，但边界竞争很复杂，尤其是 deque 里只剩一个任务时，owner pop 和 stealer steal 可能竞争同一个元素。Chase-Lev 算法之所以复杂，正是为了解决这些边界和内存序。

学习时可以先用 mutex 保护每个 worker deque，实现工作窃取策略，再用测量判断策略收益。等策略正确后，再研究无锁 deque。这样分层能避免把“调度策略是否有效”和“无锁 deque 是否正确”混在一起。运行时工程最怕同时引入新策略和新同步算法，出了问题很难定位。

14.13 关键路径和并行度

任务图的性能由两个量共同决定：总工作量和关键路径长度。总工作量是所有任务耗时之和，关键路径是依赖图中最长路径。线程数再多，也不能把关键路径压短到依赖允许之外。一个任务图如果关键路径很长，增加 worker 只会让更多线程空闲；如果总工作量大且依赖宽，worker 才有足够 ready 任务可执行。

因此任务图优化不只是让 worker 更多，还要改变依赖结构。把串行合并改成树形合并，可以缩短关键路径；把过大的任务拆分，可以增加并行度；把过小任务合并，可以降低调度开销；把热点 key 单独处理，可以减少长尾。任务图分析把算法结构和运行时调度连接起来。

14.14 Continuation 代替阻塞等待

阻塞等待会占住 worker，continuation 则把“前置任务完成后做什么”注册给运行时。前置任务完成时，运行时把后续任务加入 ready 队列。这样等待不占线程，依赖由计数器和队列表达。异步 I/O、构建系统和数据流引擎都大量使用 continuation 思想。

Continuation 的代价是控制流不如同步代码直观，异常传播和取消更复杂。结构化并发试图把异步任务组织成有作用域的树，让父任务能等待子任务完成并统一处理取消。第二册后续不需要完整实现现代 coroutine runtime，但要让读者理解：高性能运行时的很多复杂性来自“不要让线程闲等，但还要保持语义清楚”。

14.15 背压进入运行时

任务队列不能无限增长。无限队列把生产速度和消费速度的差异藏进内存，最终可能造成延迟爆炸或 OOM。有界队列把容量作为系统契约：生产者太快时必须等待、失败、丢弃、降级或转移。背压不是性能失败，而是系统保护机制。

线程池 submit 如果使用有界队列，应定义队列满时语义。阻塞 submit 会把背压传给调用者；try submit 返回 false 让调用者决定；带超时 submit 可以避免永久等待；丢弃策略适合可丢指标，不适合必须执行的任务。每种策略都影响上层业务。运行时不能只说“队列满了就等”，要说明谁等、等多久、能否取消。

14.16 运行时指标的低开销实现

指标本身不能成为瓶颈。每个任务完成时更新全局原子计数器，在高频小任务场景下可能形成共享写热点。更好的做法是每个 worker 写自己的 `WorkerStats`，周期性汇总。队列长度可以在队列锁内更新最大值，也可以采样。窃取次数由 stealer 本地记录，最后合并。指标精度和成本要匹配用途。

若指标用于调度决策，例如根据队列长度做负载均衡，就需要更实时、更一致；若指标只是实验报告，可以延迟汇总。不要把观测性和控制路径混为一谈。实验指标可以近似，协议状态不能近似。

14.17 本章落到贯穿项目

Compute Systems Engine 的运行时路线应分四步。第一步，全局队列线程池：实现 submit、wait、shutdown，队列用 mutex 和条件变量，保证关闭不丢任务。第二步，任务指标：记录每个 worker 执行任务数、空转次数、队列最大长度和任务耗时摘要。第三步，任务图：实现 ready count，支持归约中的树形合并和并行扫描。第四步，工作窃取：先用 mutex deque 验证调度策略，再决定是否实现无锁 deque。

每一步都要保留前一步作为对照。全局队列版本是 work stealing 的 reference；顺序执行是任务图的 reference；固定任务粒度是动态调度的 reference。没有 reference，运行时一旦出错就很难判断是调度策略、同步结构还是任务语义的问题。

14.18 本章习题

习题与作业

习题一：为一个最小线程池写状态机。状态至少包括 Open、ShuttingDown、Stopped。列出每个状态下 submit、wait、shutdown 和 worker loop 的行为。

习题二：设计线程池指标结构，要求避免所有 worker 高频写同一个 cache line。说明哪些指标需要精确，哪些可以近似或延迟汇总。

习题三：把并行归约的最后合并从线性合并改成树形合并。画出任务图，计算关键路径长度如何变化，说明任务数增加带来的调度成本。

习题四：设计一个工作窃取实验。先用 mutex deque 实现策略，不使用无锁 deque。比较全局队列和本地队列在任务粒度不同、负载不均不同情况下的 worker 空闲时间和窃取次数。

实验：设计一个线程池接口，列出 submit、shutdown、wait 的语义。不要急着实现完整工作窃取，先实现固定线程数和全局队列，并写出关闭时不丢任务的测试。

第 15 章

I/O、异步与背压

计算系统不只做 CPU 运算。磁盘、网络、管道、日志和用户请求都会引入 I/O。I/O 的延迟和吞吐特征与 CPU 完全不同，因此运行时必须区分计算等待和 I/O 等待。

15.1 同步 I/O 和异步 I/O

同步 I/O 调用会让当前线程等待结果。少量线程处理少量 I/O 时简单可靠；大量并发连接或慢设备场景下，同步等待会浪费线程。异步 I/O 把提交和完成分离，让线程可以处理其他工作。

事件循环、回调、future、协程和 `io_uring` 都是组织异步 I/O 的方式。选择哪种方式，要看系统需要的并发数、延迟要求、代码复杂度和平台支持。

异步 I/O 的核心不是“换一种语法”，而是把等待从工作线程上移走。同步调用让当前线程停在系统调用或库调用里；异步模型把请求提交出去，完成时再处理结果。这样可以用较少线程管理大量等待。但异步也会引入状态机、取消、超时、错误传播和资源生命周期问题。若并发度很低，同步 I/O 可能更简单可靠。

15.2 背压

背压是系统告诉上游“我处理不过来”的机制。没有背压，队列会无限增长，内存耗尽，延迟失控。背压可以通过 bounded queue、令牌桶、限流、拒绝请求、降级和批处理实现。

并发系统里 bounded queue 的意义不只是节省内存，更是把过载变成可观察、可控制的状态。阻塞、返回错误或丢弃，都比默默无限排队更可管理。

背压必须一路传到真正的生产者。只在中间队列限长，但上游仍然无限接收请求，过载会换一个地方堆积。一个服务端如果 worker 队列满了，可以选择拒绝新请求、降低读取速度、返回重试、丢弃低优先级任务或触发降级。选择哪种方式取决于业务语义，但不能没有选择。

15.3 批处理

批处理用一次调度或一次 I/O 处理多个元素，摊薄固定成本。网络发送、日志写入、数据库提交和向量计算都常用批处理。批太小浪费固定成本，批太大增加延迟和缓存压力。

批处理要控制两个阈值：数量阈值和时间阈值。只按数量凑批，在低流量时可能让请求等太久；只按时间刷批，在高流量时可能批太小。很多系统使用“达到 N 条或等待 T 时间就发送”的策略。批处理还要考虑失败语义：一批里部分成功时如何重试，是否会重复执行，是否需要幂等 ID。

15.4 CPU 与 I/O 的边界

I/O 密集任务不应占满计算线程池。常见设计是分离 I/O 线程、计算线程和后台维护线程，或者使用异步运行时统一调度但明确资源限制。否则慢 I/O 会阻塞计算任务，计算任务也会延迟 I/O 完成处理。

15.5 队列是系统的压力表

队列长度、入队速率、出队速率和等待时间，是理解系统压力的基本指标。队列持续增长说明下游处理能力不足或上游没有背压；队列经常为空说明下游可能在等上游；队列长度锯齿状变化可能是批处理或周期性调度造成的。只看 CPU 使用率可能误判，因为系统可能 CPU 很空但卡在 I/O，也可能 CPU 很满但队列仍然增长。

有界队列实验要记录三种结果：生产者是否阻塞，消费者是否空闲，丢弃或拒绝是否发生。这样才能判断背压策略是否真的控制住过载。

实验：用 bounded queue 模拟生产者和消费者。让生产速度高于消费速度，观察队列大小和阻塞行为。解释这为什么是背压，而不是简单的性能问题。

第零部分

分布式计算：把单机原则扩展到集群

第 16 章

分布式计算模型

分布式计算把计算扩展到多台机器，但同时引入网络通信和故障。共享内存消失后，线程之间的读写变成消息；锁和原子不再直接可用；时间、顺序和失败都变得不可靠。

16.1 网络不是共享内存

单机内存访问通常以纳秒计，网络往返通常以微秒到毫秒计。单机 cache miss 已经很贵，跨网络请求更贵。分布式系统要尽量减少同步往返，使用批处理、流水线、数据本地性和异步通信。

网络还可能丢包、重排、延迟抖动或断开。应用层必须处理超时、重试、重复请求和部分失败。一个请求超时，不代表对方没有执行；重试可能造成重复副作用，因此幂等性非常重要。

16.2 RPC 的成本

RPC 把远程调用包装成类似本地函数调用的形式，但它不是本地函数。参数要序列化，消息要进入内核或用户态网络栈，经过网卡和交换机，到达对端后再反序列化和调度执行。一次 RPC 的固定成本远高于一次函数调用，因此分布式系统要避免细粒度频繁 RPC。

批处理、流水线和异步调用是常见优化。批处理减少每条消息固定开销，流水线隐藏往返延迟，异步调用避免线程阻塞。但它们都会增加错误处理复杂度：部分成功、乱序返回、超时重试和取消都要定义清楚。

RPC 还会隐藏背压问题。本地函数调用如果慢，调用者自然等在当前线程；远程调用如果被异步提交，调用者可能继续制造更多请求，直到队列、连接池或对端服务被压垮。因此 RPC 客户端必须有并发上限、超时、重试预算和排队策略。没有这些边界，异步 RPC 只是把阻塞变成不可控积压。

16.3 时间和顺序

不同机器没有完美同步时钟。日志时间戳可以帮助观察，但不能单独作为一致性依据。分布式系统常用逻辑时钟、版本号、任期号和日志索引表达顺序关系。

顺序要区分因果顺序和物理时间顺序。请求 A 触发请求 B，B 因果上发生在 A 之后；但两个机器的墙钟时间可能显示 B 的时间戳更早。协议通常不依赖墙钟证明因果，而是使用请求 ID、版本号、日志索引、任期或向量时钟等逻辑信息。日志时间戳主要用于观测和排查，不应被误用成强一致证据。

16.4 幂等性

幂等操作执行一次和执行多次效果相同。分布式系统中，客户端超时后常常不知道服务器是否执行了请求。若请求不是幂等的，重试可能造成重复扣款、重复创建或重复写入。常见做法是给请求分配唯一 ID，服务器记录已处理 ID，并在重复请求时返回同一结果。

幂等设计要落到存储结构。只在客户端生成请求 ID 不够，服务器还要把请求 ID、执行结果和业务状态放在同一个原子提交边界里。否则服务器可能已经修改业务状态，但还没记录请求 ID 就崩溃，重试后仍然重复执行。工程上常用去重表、条件写、事务日志或状态机日志解决这个问题。

16.5 故障模型

机器可能宕机，进程可能重启，网络可能分区，磁盘可能慢，消息可能重复。多数工程系统采用 crash-stop 或 crash-recovery 模型，不处理任意恶意错误。明确故障模型，是协议设计的第一步。

16.6 数据本地性

分布式计算中，移动计算到数据附近通常比移动大量数据更划算。MapReduce、Spark、分布式数据库和对象存储都围绕数据分片、任务调度和网络 shuffle 做权衡。

数据本地性是 NUMA 原则在集群尺度上的版本。单机里远端 NUMA 访问慢，集群里跨机拉取数据更慢；单机里先在线程本地 partial 再合并，集群里先在 map 端 combine 再 shuffle；单机里热点 cache line 会拖慢所有核心，集群里热点 key 会拖慢整个 stage。前面章节的思想在这里不是类比游戏，而是同一原则换了成本单位。

16.7 CAP 的正确理解

CAP 不是说系统只能在一致性、可用性、分区容忍性里任选两个。网络分区是分布式系统必须面对的现实；在分区发生时，系统需要在继续响应请求和保持强一致之间做选择。正常情况下，系统仍然要同时追求低延迟、高可用和足够一致。工程讨论要具体到故障场景，而不是只背缩写。

16.8 超时、重试和重试风暴

超时不是失败证明，只是本地等待时间超过阈值。请求可能在网络上，可能在对端队列里，可能已经执行但响应丢失，也可能确实失败。重试可以提高成功率，也可能放大过载。如果一个服务已经慢了，所有客户端同时重试，会制造重试风暴，让系统更慢。

健康的重试策略通常包括指数退避、抖动、最大重试次数、总超时预算、幂等保证和熔断。重试预算尤其重要：一个上游请求如果调用十个下游，每个下游都重试三次，总请求量可能迅速膨胀。分布式系统的性能问题常常来自这些控制路径，而不是用户函数本身。

核心思想

分布式系统的性能问题往往是正确性问题的影子。重试、复制、超时、共识和恢复都会改变吞吐和延迟。

实验：在本机用多个进程模拟客户端和服务端。人为加入随机延迟和超时，观察重试如何造成重复请求。设计一个幂等请求 ID 机制。

第 17 章

分片、复制与共识

分布式系统通过分片扩展容量，通过复制提高可用性，通过共识在故障中维护关键状态一致。三者共同构成大多数存储和计算平台的基础。

17.1 分片

分片把数据按 key、范围或哈希分到不同节点。好的分片要负载均衡、支持扩容、减少跨分片事务。热点 key 会破坏均衡，需要拆分、缓存、限流或特殊路由。

分片后的查询可能需要 scatter-gather：向多个分片发请求，再合并结果。它类似并行归约，但网络延迟和节点故障让合并更复杂。

分片策略要匹配访问模式。哈希分片适合点查和均匀 key，范围分片适合范围扫描和排序，但容易出现热点区间。虚拟分片把逻辑分片数量做得比物理节点多，便于迁移和均衡。热点 key 不能只靠增加节点解决，因为所有请求仍然打到同一个逻辑位置；需要缓存、拆 key、限流、异步化或改变业务模型。

17.2 再分片

系统运行一段时间后，数据量和热点会变化，需要再分片。再分片要迁移数据、更新路由、处理迁移期间的读写，并保证失败后可恢复。简单哈希分片在节点数变化时可能移动大量 key，一致性哈希或虚拟分片可以减少迁移范围。

再分片最难的是迁移期间的一致性。客户端可能按旧路由写，控制面可能已经发布新路由，数据可能正在复制。常见做法是让分片有状态机：准备迁移、双写或转发、追平增量、切换路由、清理旧副本。每一步都要能失败重试。没有状态机的再分片脚本，往往在中途失败后留下难以解释的双写和漏写。

17.3 复制

复制保存多个副本，提高可用性和读吞吐。同步复制延迟高但数据更安全，异步复制延迟低但可能丢失最新写入。主从复制简单，multi-leader 和 leaderless 复制更复杂，需要处理冲突。

复制协议必须说明写入何时算成功。如果 leader 写本地日志就返回，延迟低但 leader 崩溃可能丢数据；如果多数副本确认后返回，延迟高但故障恢复更稳；如果所有副本确认后返回，可用性和尾延迟都受最慢副本影响。工程系统通常让不同数据选择不同级别：元数据强一些，缓存和统计弱一些。

17.4 读写仲裁

Quorum 系统常用 W 个写副本和 R 个读副本满足 $R+W>N$ ，从而让读集合和写集合有交集。这个模型帮助理解一致性和可用性权衡。 W 大提高写安全性但增加写延迟， R 大提高读新鲜度但增加读延迟。实际系统还要处理慢副本、hinted handoff、read repair 和版本冲突。

17.5 一致性

强一致性让读者看到最新写入，但通常需要更多同步。最终一致性允许副本暂时不同，换取可用性和低延迟。选择一致性模型要看业务语义：计费、库存和元数据通常更偏强一致；缓存、推荐和日志统计可以接受延迟收敛。

17.6 共识

Raft 和 Paxos 这类共识协议用于在故障中让多个节点就日志顺序达成一致。共识不是让所有操作都变快，而是让关键状态在故障中仍然可恢复。共识路径应尽量小而清晰，不应把所有大数据流

量都压进共识日志。

共识适合维护小而关键的顺序状态，例如元数据、配置、leader 选举、分片归属和事务提交点。把大对象、热数据流和高频统计全部放进共识日志，通常会让系统吞吐受限。高性能分布式系统常把控制面和数据面分开：控制面用共识保证关键决策一致，数据面用分片、复制、批处理和校验保证吞吐。

17.7 Leader、任期和日志

以 Raft 的直觉看，系统在一个任期内选出 leader，客户端写入先进入 leader 日志，leader 复制给 follower，多数确认后提交。任期号防止旧 leader 在网络恢复后继续破坏新状态，日志索引和任期共同描述顺序。理解这些概念，能帮助读者读懂很多分布式存储的元数据路径。

17.8 读路径和租约

写路径需要顺序，读路径也要定义新鲜度。强一致读可能需要联系 leader 或多数副本，延迟较高；从 follower 读延迟低，但可能读到旧数据；租约读通过时间约束减少通信，但依赖时钟和租约安全边界。选择读路径时要问：业务是否允许旧读，旧到什么程度，故障切换时如何避免旧 leader 继续服务强一致读。

17.9 复制和备份

复制和备份经常被混为一谈。复制解决的是在线可用性：某个副本不可用时，系统还能从其他副本继续服务。备份解决的是历史恢复：数据被误删、程序 bug 写坏、勒索软件破坏或运维误操作后，系统能回到过去某个正确版本。复制通常追求低延迟和快速同步，备份通常追求隔离、保留周期和可验证恢复。一个系统可以有三个副本，仍然必须有备份。

常见误区

不要把复制等同于备份。复制会快速传播错误删除和错误写入；备份强调历史恢复。两者解决的问题不同。

实验：设计一个三副本 key-value 存储的写入流程。分别画出同步多数写、异步主从写和读修复流程，并说明各自的延迟和失败行为。

第 18 章

分布式计算工程实践

分布式计算工程把算法、运行时、存储、网络 and 观测性结合起来。真正的系统不仅要在正常路径跑得快，还要在机器慢、网络抖动、任务失败和数据倾斜时保持可诊断、可恢复。

18.1 Shuffle

Shuffle 把数据按 key 重新分布，是很多分布式计算框架的核心成本。它包含序列化、压缩、网络传输、落盘、排序、合并和反序列化。一个 group-by 或 join 的成本，常常主要来自 shuffle，而不是用户函数。

优化 shuffle 的方向包括减少数据量、提前聚合、压缩、分区均衡、避免热点 key、使用本地磁盘顺序写和控制并发拉取。它是分布式版的数据布局问题。

Shuffle 的基本流程可以拆成 map 端分区、局部排序或缓冲、落盘、网络拉取、reduce 端合并。每一步都有独立瓶颈：序列化消耗 CPU，压缩节省网络但增加计算，落盘受磁盘带宽影响，拉取并发过高会压垮对端，key 倾斜会让少数 reduce 任务变成长尾。优化 shuffle 不能只调一个参数，要先确认慢在哪一步。

18.2 检查点和容错

长任务必须考虑失败。检查点保存中间状态，让失败后从最近点恢复，而不是从头开始。检查点太频繁会拖慢正常路径，太稀疏会增加恢复成本。流式计算还要处理 exactly-once 或 at-least-once 语义。

检查点设计要说明一致性边界。批处理任务可以在 stage 边界保存输出，失败后重跑某个分区；流式任务需要同时保存输入 offset、算子状态和输出提交位置。若只保存算子状态，不保存已经输出到哪里，恢复后可能重复输出；若只保存 offset，不保存状态，聚合结果会丢失。所谓 exactly-once 往往不是“消息真的只执行一次”，而是通过幂等提交、事务输出或去重让外部效果等价于一次。

18.3 观测性

没有观测性，分布式系统无法维护。日志、指标、trace、profile 和事件审计要能回答：请求经过哪些节点，在哪里等待，重试了几次，队列多长，哪个分片慢，哪个任务倾斜，哪次部署引入变化。

观测性要围绕问题设计。延迟问题需要端到端 trace 和每段耗时；吞吐问题需要输入速率、处理速率和队列长度；倾斜问题需要按 key、分片、worker 的分布；可靠性问题需要错误码、重试、超时和恢复路径；资源问题需要 CPU、内存、磁盘、网络 and GC 或分配器指标。只打印一行“任务失败”没有工程价值。

18.4 从单机原则到集群原则

第二册前面讲的原则在分布式系统中仍然成立。局部性对应数据本地调度；锁竞争对应热点 key；false sharing 对应多个任务争用同一分片；背压对应限流和队列控制；归约对应 map-side combine 和树形聚合；缓存一致性对应复制一致性和失效策略。系统规模变大，原则没有消失，只是成本单位从 cache line 变成网络消息和磁盘块。

18.5 一个贯穿工程案例

考虑一个日志统计系统：输入日志按文件分片放在多台机器上，每个 worker 读取本地分片，解析出 key，先做本地 hash 聚合，再把 partial 按 key 分区 shuffle 到 reduce worker，最后合并并写出结果。这个系统把第二册几乎所有原则串在一起。本地解析要考虑 CPU 和内存布局，本地聚合要避免锁热点，shuffle 要处理网络和背压，reduce 要处理 key 倾斜，输出要保证幂等，失败后要能从检查点恢复。

如果某个 key 特别热，单机里它像一个热点锁，分布式里它像一个热点分片。解决方法也相似：先把热点拆成多个局部 partial，再在后续阶段合并。若网络拉取过多，像单机任务队列无背压一样，系统会排队爆炸。若 worker 失败后重跑，输出必须用任务 ID 和分区 ID 保证幂等提交。这样的案例比孤立讲术语更能训练系统思维。

核心思想

分布式计算不是单机计算的替代，而是单机计算原则在通信、故障和复制约束下的扩展。

综合实验：设计一个小型分布式日志统计系统。输入按文件分片，worker 本地统计，shuffle 按 key 聚合，driver 合并结果。写清数据格式、失败重试、幂等输出、指标和瓶颈。

附录 A

实验路线

第二册的实验围绕 Compute Systems Labs 展开。第一组实验是硬件与测量基础，包括 `lscpu` 拓扑记录、缓存层级观察、TLB 与缺页计数、`perf stat`、顺序归约和并行归约。第二组实验是数据布局，包括 AoS、SoA、false sharing、顺序访问、随机访问和预取效果。第三组实验是单机高性能计算，包括多累加器、SIMD 自动向量化、Roofline 分析和典型内核模式。

第四组实验是多线程同步，包括 mutex counter、atomic counter、bounded queue、条件变量、信号量和线程亲和性。第五组实验是无锁结构，包括 SPSC ring buffer、CAS 循环、ABA 现象、版本标签和内存回收方案分析。第六组实验是并行算法，包括 reduce、scan、partition、thread-local histogram 和任务粒度比较。第七组实验是运行时，包括线程池、任务队列、工作窃取模型、异步 I/O 和背压。第八组实验是分布式计算，包括本机多进程 RPC 模拟、超时重试、幂等请求、分片、复制和 shuffle。

每个实验必须记录环境、输入、命令、输出、正确性检查和结论。性能数据没有上下文就不能进入正文。若实验受手机、Termux 或虚拟化环境限制，也要明确记录限制。

附录 B

源码阅读索引

本附录保存后续源码专题的入口。

Linux 内核源码用于理解调度、NUMA、页表、透明大页、perf 事件、futex、epoll、内存分配和网络栈。阅读时不要从全局搜索函数名开始，而要围绕问题进入：线程为什么迁移、缺页为什么发生、futex 怎样把用户态锁和内核等待连接起来。

glibc 和 musl 用于阅读 pthread、mutex、condition variable、malloc 和系统调用封装。重点观察用户态 fast path 和进入内核 slow path 的分界。

Intel TBB、oneTBB 或 LLVM OpenMP runtime 用于阅读任务调度、线程池、工作窃取和并行算法运行时。阅读时关注任务粒度、队列结构、窃取策略和 shutdown 语义。

Folly、Abseil 和 Boost.Lockfree 用于阅读并发容器、原子封装、hazard pointer、RCU 风格回收和高性能队列。阅读时先搞清进度保证和内存回收，不要只看 CAS。

Redis、RocksDB、Kafka 和 Spark 用于阅读分布式计算和存储工程：事件循环、后台线程、LSM compaction、分区、复制、日志、shuffle、背压和故障恢复。阅读时抓数据流和失败路径，不要只列模块名。

术语表

算术强度 操作数与数据移动字节数的比值，用来判断一个内核更可能受计算吞吐还是内存带宽限制。

Roofline 用计算峰值、内存带宽和算术强度估算性能上界的模型。

SIMD 一条指令处理多个数据 lane 的执行方式，是现代 CPU 单核吞吐的重要来源。

乱序执行 CPU 在保持单线程可观察结果不变的前提下，提前执行已经准备好的指令以隐藏延迟。

分支预测 CPU 在分支结果确定前预测执行路径，预测失败会清空错误路径上的工作。

cache line 缓存一致性和缓存填充的基本粒度，常见大小为 64 字节。

TLB 地址翻译缓存，用于缓存虚拟页到物理页的映射。

NUMA 非统一内存访问结构，不同核心访问不同内存节点的成本不同。

false sharing 多个线程写不同变量，但变量落在同一 cache line 上，导致一致性流量和扩展性下降。

内存序 原子操作对可见性和重排施加的约束，例如 relaxed、acquire、release 和 `memory_order_seq_cst`。

lock-free 一类进度保证，表示系统整体能持续前进，但不保证每个线程有限步完成。

背压 下游处理不过来时向上游施加的流量控制机制。

共识 分布式系统中多个节点在故障下对日志顺序或状态达成一致的协议问题。

索引

instruction

add, 9